Unity 6000.5 · Entities 6.5

Unity DOTS Introduction

Entities, Netcode, ECS workflows, jobs, debugging, optimization, and migration.

Youngmin Cho (SillyToolValley)

English Edition · 2026

Unity DOTS Introduction

A field guide to Entities 6.5, Netcode for Entities, ECS workflows, optimization, and migration.

English edition generated from the Unity DOTS Introduction repository on 2026-05-19.

Author: Youngmin Cho (SillyToolValley) / UCI (Unity Certified Instructor)

Unity DOTS Introduction

A field guide to Entities 6.5, Netcode for Entities, ECS workflows, optimization, and migration.

This book packages the Unity DOTS guideline repository into a single reading flow. It keeps the source material practical: setup, SubScene and Baker workflows, ECS component and system patterns, job scheduling, structural-change safety, EntityCommandBuffer usage, Netcode for Entities, optimization, debugging, migration, and glossary references.

The target stack is Unity 6000.5+, Entities 6.5.0, Netcode for Entities 6.5.0, and the DOTS core packages bundled with the editor.

How to read this book

- New DOTS users should read Part I, then Chapters 4-15 in Part II before moving into Netcode.
- Existing Entities 1.x users can skim the core chapters and jump to Part IV for migration notes.
- Performance work belongs after the main ECS workflow chapters, because chunk layout and profiling make more sense once the data model is familiar.

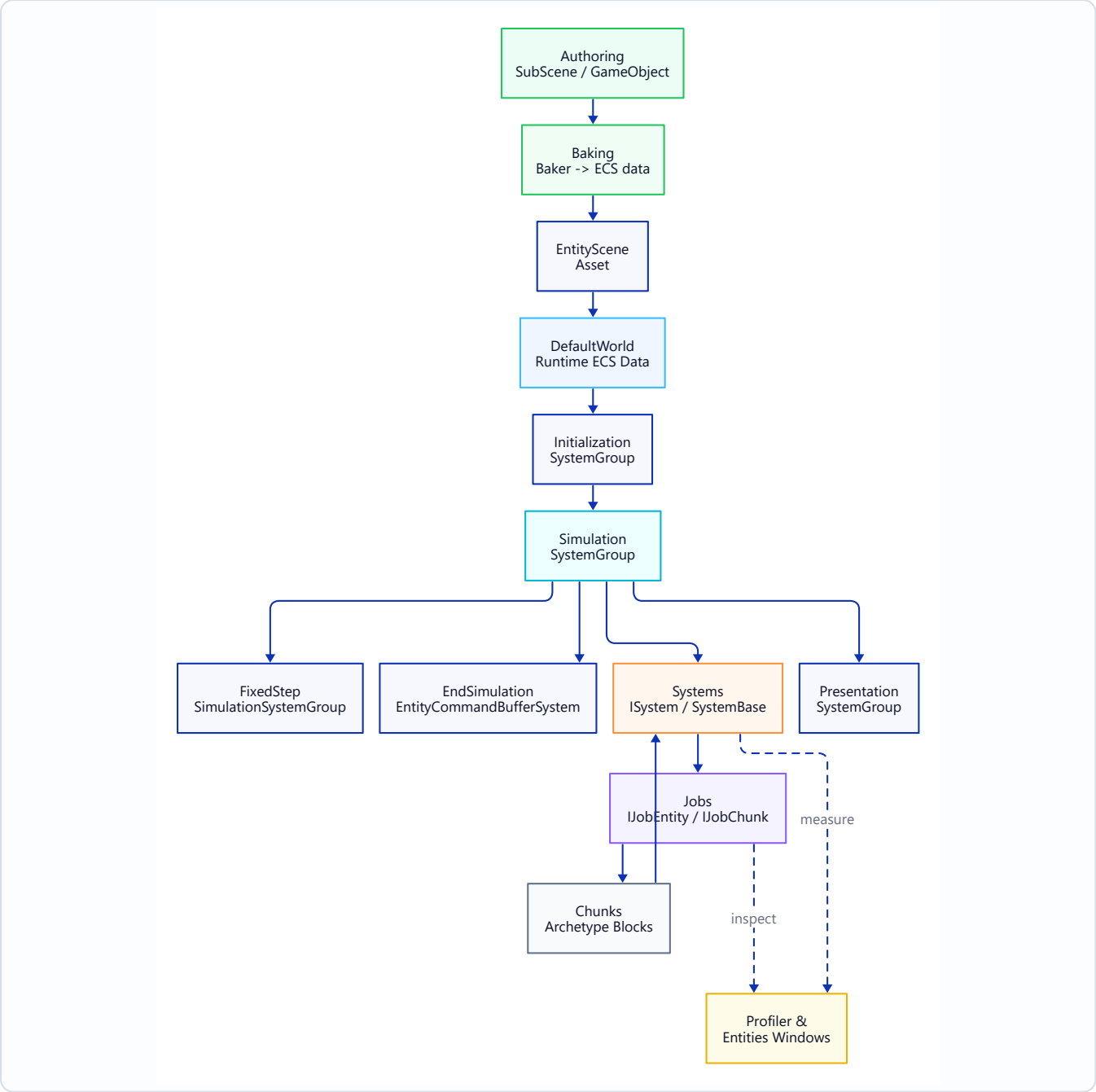

Figure: Unity DOTS Architecture Flow.

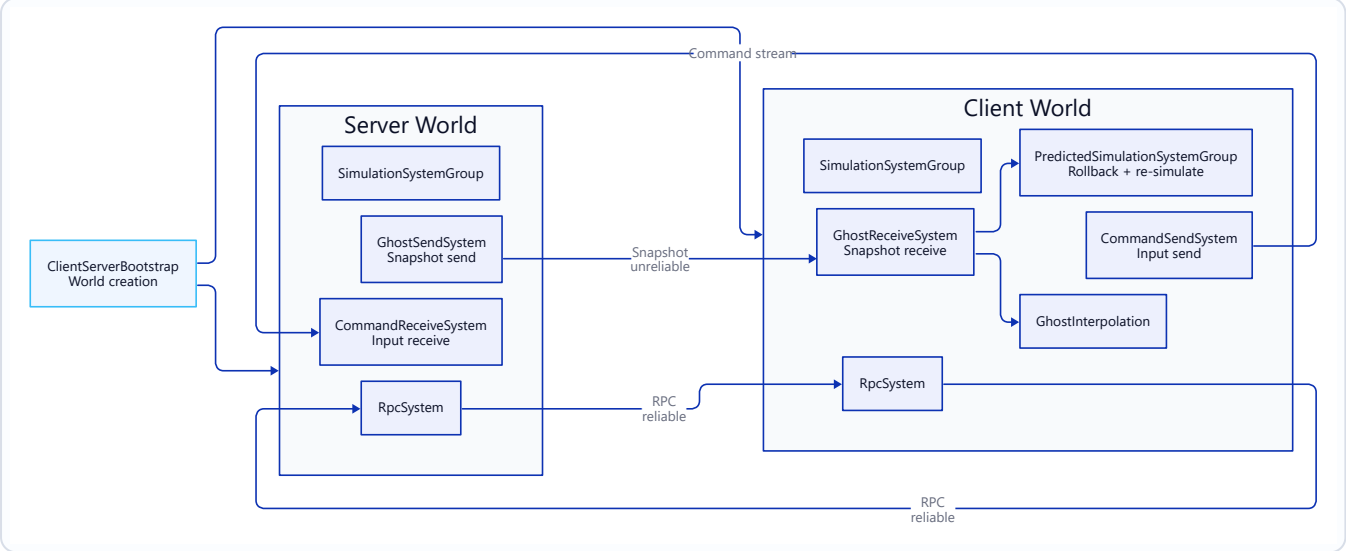

Figure: Netcode for Entities Architecture Flow.

Table of Contents

Section	Topic
Chapter 15	IJobEntity vs IJobChunk
Chapter 16	Structural Change & Safety
Chapter 17	EntityCommandBuffer · Deferred Entity
Chapter 18	ParallelWriter · Deterministic Playback
Chapter 19	Netcode Client-Server World & Bootstrap
Chapter 20	Netcode Network Connection & Approval
Chapter 21	Netcode Ghost Snapshot & Synchronization
Chapter 22	Netcode Prediction & Rollback
Chapter 23	Netcode Command Stream & Input
Chapter 24	Netcode RPC
Chapter 25	Netcode Ghost Optimization · Importance · Relevancy
Chapter 26	Netcode Physics Integration & Lag Compensation
Chapter 27	Netcode Profiler & Debugging
Part III. Optimization and Debugging	
Chapter 28	Chunk Layout & TypeManager
Chapter 29	Systems · Entity Inspector · Query Window
Chapter 30	Profiler · Bottleneck Analysis
Chapter 31	Managed Object Reference Audit
Part IV. Migration	
Chapter 32	Entities 1.x → 6.5 Overview
Chapter 33	Package Manager → Core Package
Chapter 34	Managed Object References → UnityObjectRef
Chapter 35	foreach → IJobEntity Migration
Chapter 36	IAspect Deprecation — Migrating Off Aspects
Appendix. Changelogs	
Appendix A	Entities 1.4 → 6.5 Key Changes
Appendix B	Netcode for Entities 1.4 → 6.5 Key Changes
Reference. Glossary	
Glossary	Glossary

Part I. Getting Started

Chapter 1. Environment Setup — Unity 6000.5 + Entities 6.5.0

1. Overview

From **Unity 6000.4+**, Entities is a **Core Package** — it ships *with* the Editor and its version is pinned to the Editor version, so you no longer pick an Entities version by hand. It is **not** auto-added to new projects, though: you still add it once per project through the Package Manager (Unity Registry). This guide walks through installing the Editor, creating a project, adding Entities, and confirming it is active.

> **Project Environment reference:** Unity 6000.5.0b2 · Entities 6.5.0 · Collections / Mathematics / Entities Graphics (all Core Packages).

---

2. Install Unity 6000.5+

1. Open **Unity Hub**. Install `Unity Hub` first if you don't have it (https://unity.com/download).
2. In Hub, go to **Installs** → **Install Editor**.
3. Pick any **6000.5.x** version (b-series beta or newer). Entities 6.5 is only available on 6000.5+.
4. Modules: leave the defaults. No extra module is required for ECS itself.

> You do **not** need to tick any "DOTS" module when installing the Editor — the Entities package ships with the Editor from 6000.4. You still add it to each project later (see the next steps).

---

3. Create a new project

From **Hub** → **Projects** → **New project**:

Field	Value
Editor Version	6000.5.x

Field	Value
Template	**Universal 3D** (or **3D (Built-in)**, **HDRP** — any works)
Project Name	anything

Click **Create project**. The Editor opens on an empty scene.

> The old dedicated "DOTS template" is no longer required on 6000.4+. Any template can host an Entities workflow.

---

4. Add the Entities package and verify it's active

Open **Window** → **Package Manager**, switch the scope to **Unity Registry**, select **Entities**, and click **Install** (or use **+** → **Add package by name...** and enter `com.unity.entities`). Because Entities is a Core Package you do not pick a version — the Editor installs the one bundled with it (Entities 6.5 on Unity 6000.5). Installing Entities also pulls in **Collections**, **Mathematics**, and **Entities Graphics** as dependencies.

Now switch the scope to **In Project** and confirm **Entities**, **Collections**, **Mathematics**, and **Entities Graphics** are listed. If you prefer to verify via files, open `Packages/manifest.json`: a `com.unity.entities` entry is now present (the dependencies it pulls in are resolved automatically and may not each get their own entry).

Next, open the ECS windows:

- **Window** → **Entities** → **Hierarchy** — shows entities at runtime.
- **Window** → **Entities** → **Systems** — shows registered systems.
- **Window** → **Entities** → **Archetypes** — shows chunk layout.

If all three menus exist, Entities is wired up correctly.

---

5. Smoke test — create a SubScene

1. In the Hierarchy, right-click → **New Sub Scene** → **Empty Scene…**
2. Name it `MainSubScene.unity` and save.
3. Inside the new SubScene, add any primitive (**3D Object** → **Cube**).
4. Save the scene. The Cube is **baked** into an entity as soon as the SubScene is closed (yellow icon) or at build time.
5. Press **Play**. Open **Window** → **Entities** → **Hierarchy** — you should see a Cube entity. The smoke test passes.

> A SubScene is the authoring entry point: its GameObjects are baked to Entities on save and at build. The remaining tutorial in `03_Hello DOTS — First Entity.md` builds on this.

---

6. Troubleshooting

Symptom	Cause / Fix
`Window → Entities` menu is missing	Entities isn't installed yet (add it via **Package Manager** → **Unity Registry** → **Entities**), or the Editor is older than 6000.4 (upgrade it).
`Baker<T>` / `IComponentData` not found	Assembly-definition file excludes `Unity.Entities`. Add it as a reference, or delete the asmdef for a quick test.
SubScene stays as a GameObject and never bakes	The SubScene must be **closed** (the yellow icon) for baked entities to appear. Open/close via the checkbox next to its name in the Hierarchy.
Entities Hierarchy window shows nothing in Play mode	Check that the SubScene is included in the current scene and that **Window** → **Entities** → **Hierarchy**'s "World" dropdown is set to `DefaultGameObjectInjectionWorld`.
Build error: `com.unity.entities` not found	The package isn't installed, or `manifest.json` pins a 1.x version that doesn't exist on this Editor. Install Entities from the Package Manager so the entry resolves to the Editor's Core version. See `Migration/02_Package Manager → Core Package.md`.

Chapter 2. Core Packages Explained

1. Overview

With the **Entities package 6.4.0** release (shipping alongside Unity Editor 6000.4), the major DOTS packages became **Core Packages** — they now ship *with* the Editor (installed from the Editor bundle rather than downloaded from a remote registry) and their version is bound to the Editor. The exact wording from the package changelog is:

> *"Package is now a core package embedded in Unity."*

This affected `com.unity.entities`, `com.unity.collections`, `com.unity.mathematics`, `com.unity.entities.graphics`, and (on its matching 6.5 release) Netcode for Entities. It changes how you install, version, and track them — but it is a **distribution** change, not an API change.

This page explains:

- Which packages are now Core Packages (and which are not).
- Where their version numbers come from.
- What changes in your `manifest.json` and `packages-lock.json`.
- Why the shift happened.

---

2. What is a Core Package?

A **Core Package** is a package that ships **with the Editor**. You still add it to a project through the Package Manager like any other package, but it installs from the Editor bundle instead of a remote registry, you cannot change its version independently of the Editor, and its release cadence is the Editor's. (This is *not* the same as a **Built-in package** — an engine module such as Physics that you only enable or disable.)

Trait	Package Manager package	Core Package
Install step	Add via UPM	Add via Package Manager (installs from the Editor bundle)
Version chosen by	You, in `manifest.json`	Unity, per Editor version

Trait	Package Manager package	Core Package
Release cadence	UPM independent	Editor releases
Appears in `manifest.json`	Yes	Yes — added when you install
Appears in `packages-lock.json`	Yes	Yes, marked as `builtin`
Shown in Package Manager	Under "In Project"	Under "Unity Registry" to install, then "In Project"

---

3. The DOTS package map on Unity 6000.5+

Package	On Unity 6000.5+	Purpose
Entities	**Core Package** (6.5)	ECS runtime: Entity, Component, System, World
Collections	**Core Package**	Native container types (`NativeList<T>`, `NativeHashMap<K,V>`, etc.)
Mathematics	**Core Package**	Burst-optimized `float3`, `quaternion`, `int4`, etc.
Entities Graphics	**Core Package**	Renders entity-based graphics through the SRP
Netcode for Entities	**Core Package** (6.5)	Client-server networking for entities
Unity Physics	Package Manager	Stateless, deterministic physics for entities
Character Controller	Package Manager	ECS-based character controller
Burst Compiler	Built-in engine feature	Compiles jobs and systems to tight native code
Job System	Built-in engine feature	Multithreaded job scheduling

> Note: Job System and Burst were never UPM packages — they are part of the Engine. The shift in 6000.4 is specifically about the *DOTS packages on top of them*.

---

4. Version numbers after the transition

Core Package versions **align with the Editor**:

Editor	Core Entities
Unity 6000.4	Entities 6.4
Unity 6000.5	Entities 6.5

The old 1.x line still exists for projects that cannot yet move to 6000.4+. The two tracks are separate; you upgrade to 6.x by upgrading the Editor.

See `Changelog/Entities 1.4 → 6.5 Key Changes.md` for Entities transition notes and `Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md` for Netcode-specific changes.

---

5. Effect on your project files

`manifest.json`

When you add Entities, a `com.unity.entities` entry appears here. The packages it pulls in (`com.unity.collections`, `com.unity.mathematics`, `com.unity.entities.graphics`) are resolved as dependencies and usually don't need their own entries. If you upgrade a 1.x project, replace the old pinned versions and let the Package Manager resolve them to the Editor's Core versions — see `Migration/02_Package Manager → Core Package.md`.

`packages-lock.json`

Core Packages appear with `"source": "builtin"` instead of a version URL.

Assembly definitions (.asmdef)

Your `.asmdef` files still reference the assembly names (for example `Unity.Entities`, `Unity.Collections`). Nothing changes here — only the package-distribution layer changed.

---

6. Why the shift?

The stated goal is to let the ECS team ship features faster and more incrementally by coupling DOTS updates to the Editor release instead of an independent package schedule. It also lets the Engine rely on ECS internally, which was awkward while the packages were optional.

---

7. Troubleshooting

Symptom	Cause / Fix
Package Manager only offers `Entities` at an old `1.x` version	You are on a pre-6000.4 Editor. Either upgrade the Editor, or stay on the 1.x line.

Symptom	Cause / Fix
`manifest.json` resolve error after changing the `com.unity.entities` version	Delete `packages-lock.json` and let the Editor regenerate it.
A sample or asset store package targets `com.unity.entities@1.x`	On 6000.5+ the 1.x API has mostly been superseded; check the Migration folder for equivalents.
Two Editor installs report different Entities versions	Expected. Version is per Editor install.

Chapter 3. Hello DOTS — First Entity

1. What you will build

A cube that rotates on its Y axis, driven by an ECS system. In one sitting you will touch every core concept: **SubScene**, **authoring → baker**, **`IComponentData`**, **`ISystem`**, and the **Entities Hierarchy / Systems** windows. At the end you will swap the main-thread system for an **`IJobEntity`** to see how parallelism enters.

Prerequisite: [`01_Environment Setup.md`](01_Environment Setup (Unity 6000.5 + Entities 6.5.0).md) — a Unity 6000.5 project with a confirmed SubScene and Entities menu.

---

2. Create the SubScene

1. Open your empty scene.
2. In the **Hierarchy**, right-click → **New Sub Scene** → **Empty Scene...**
3. Save it as `HelloDots.unity` inside `Assets/Scenes/`.
4. Inside the new SubScene, right-click → **3D Object** → **Cube**. Rename it to `RotatingCube`.
5. Save the scene (`Ctrl/⌘ + S`). Close the SubScene by clicking the checkbox next to its name in the Hierarchy — the icon goes yellow, which means its contents are being baked to entities.

> From this point on, modifications to `RotatingCube`'s components trigger a re-bake automatically when you reopen/close the SubScene.

---

3. Define the component — `RotationSpeed`

Create `Assets/Scripts/RotationSpeed.cs`:

```
using Unity.Entities;

public struct RotationSpeed : IComponentData
{
    public float RadiansPerSecond;
}
```

Notes:

- `IComponentData` on an **unmanaged** `struct` is the default pick — it lives directly in chunks and is Burst-friendly.
- Keep the fields blittable (primitives, `Unity.Mathematics` types, other unmanaged structs).

---

4. Define the authoring + baker

Create `Assets/Scripts/RotationSpeedAuthoring.cs`:

```
using Unity.Entities;
using Unity.Mathematics;
using UnityEngine;

public class RotationSpeedAuthoring : MonoBehaviour
{
    public float DegreesPerSecond = 90f;

    class Baker : Baker<RotationSpeedAuthoring>
    {
        public override void Bake(RotationSpeedAuthoring authoring)
        {
            // Dynamic — this entity will have its transform written each frame.
            var entity = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(entity, new RotationSpeed
            {
                RadiansPerSecond = math.radians(authoring.DegreesPerSecond)
            });
        }
    }
}
```

Attach the authoring to the cube

1. Select `RotatingCube` in the SubScene.
2. **Inspector → Add Component → Rotation Speed Authoring**.
3. Leave `DegreesPerSecond = 90`.

As soon as the SubScene is closed (yellow icon), the baker runs and the cube is turned into an entity with a `RotationSpeed` component on it.

Verify the bake

- **Window → Entities → Hierarchy** — you should see `RotatingCube` listed.

- Click it. The **Inspector** on the right shows its components — look for `RotationSpeed { RadiansPerSecond: ~1.5708 }` (π/2).

---

5. Write the system — `RotationSystem`

Create `Assets/Scripts/RotationSystem.cs`:

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, speed) in
                 SystemAPI.Query<RefRW<LocalTransform>, RefRO<RotationSpeed>>())
        {
            transform.ValueRW = transform.ValueRO.RotateY(speed.ValueRO.RadiansPerSecond * dt);
        }
    }
}
```

Notes:

- `partial struct` + `ISystem` is the **unmanaged system** form. It can be Burst-compiled and runs without GC overhead.
- `SystemAPI.Query<RefRW<A>, RefRO<B>>()` is the modern iteration API — this replaces the deprecated `Entities.ForEach`.
- `LocalTransform.RotateY(radians)` returns a rotated copy; we write it back with `transform.ValueRW = ...`.

Press Play

The cube rotates. In **Window** → **Entities** → **Systems** you can see `RotationSystem` listed under the simulation group with a per-frame timing.

---

6. Inspect what you built

Useful windows once you are in Play mode:

Window	What it shows
Entities Hierarchy	Live list of entities. Click any entity to see its chunk + components.
Systems	All running systems, grouped by their `SystemGroup`. Timing per system is visible here.
Archetypes	Chunks grouped by component signature. Useful for spotting fragmentation.

Take a moment to confirm:

1. The `RotatingCube` entity exists and has `LocalTransform` + `RotationSpeed`.
2. `RotationSystem` is present under `SimulationSystemGroup` and accumulates frame time.

---

7. Swap to `IJobEntity` (parallelism)

Replace `RotationSystem.cs` with a job-based version to see how parallelism plugs in:

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct RotateJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in RotationSpeed speed)
    {
        transform = transform.RotateY(speed.RadiansPerSecond * DeltaTime);
    }
}

[BurstCompile]
public partial struct RotationSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new RotateJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

Notes:

- `IJobEntity.Execute(ref A, in B)` is source-generated into a chunk-level job. The parameters are the components you touch.
- `.ScheduleParallel()` distributes chunks across worker threads. For a single cube you will not see a speedup, but the shape of the code scales to thousands of entities without change.
- The system's dependency is handled for you — `state.Dependency` is combined automatically when you call `ScheduleParallel()` from an `ISystem.OnUpdate`.

---

8. What to read next

If you want to...	Read
Learn how authoring maps to entities in detail	`DOTS Workflows/01_Baker Pattern & SubScene.md`
Spawn many entities at runtime	`DOTS Workflows/02_Spawner Example.md`
Understand Entity / Component / Chunk	`DOTS Workflows/03_ECS Core Concepts.md`
Choose between `ISystem` and `SystemBase`	`DOTS Workflows/08_System — ISystem vs SystemBase.md`

---

9. Troubleshooting

Symptom	Cause / Fix
Cube does not appear in **Entities Hierarchy**	The SubScene is still open (white icon, not yellow). Close it via the checkbox in the Hierarchy.
Cube appears but has no `RotationSpeed` component	`RotationSpeedAuthoring` was not attached, or the SubScene wasn't re-baked after you added it. Toggle the SubScene checkbox off and on.
Cube does not rotate	`RotationSystem` is not present in the Systems window. Most common causes: missing `partial` keyword on the struct, or the script contains a compile error.
Compile error: `TransformUsageFlags does not exist`	`using Unity.Entities;` is missing from the authoring file.
Compile error: `math.radians does not exist`	`using Unity.Mathematics;` is missing.
`SystemAPI.Query<...>` not found	Missing `using Unity.Entities;`, or the struct is not marked `partial`.

Symptom	Cause / Fix
Inspector shows `LocalTransform` missing on the entity	The baker called `GetEntity(TransformUsageFlags.None)` instead of `Dynamic`. Change the flag.
`ScheduleParallel()` throws "race condition on LocalTransform"	Another system writes to `LocalTransform` in the same frame without ordering. Add `[UpdateBefore]`/`[UpdateAfter]` or use a shared `EntityCommandBuffer`.

Part II. DOTS Workflows

Chapter 4. Baker Pattern & SubScene

1. Overview

Entities cannot be authored directly in the Hierarchy. Instead, you author **GameObjects** inside a **SubScene** and a **Baker** converts them into entities when the SubScene is closed or built.

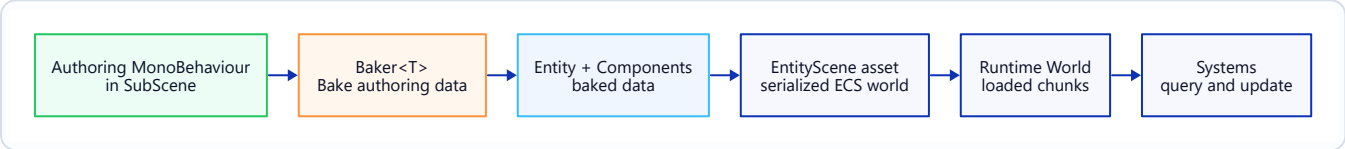

Figure: Baker and SubScene Pipeline.

---

2. SubScene lifecycle

A SubScene has three visible states:

Icon in Hierarchy	State	What it means
(white checkbox)	**Open**	Its GameObjects are editable; still GameObjects, no baking.
(yellow checkbox)	**Closed**	Bakers have run; its contents exist as entities only.
⏳	**Baking**	Incremental bake in progress (brief).

Create a SubScene

- **Hierarchy** → **right-click** → **New Sub Scene** → **Empty Scene...** — saves a `.unity` asset and adds a `SubScene` component to the parent GameObject.
- Open/close via the checkbox next to the SubScene name. Closing triggers a bake.

Incremental baking

When the SubScene is open and you edit a GameObject, only the Bakers that touched the modified authoring run again. Avoid expensive work in `Bake()` for this reason.

---

3. Baker<T> API

A Baker maps one authoring MonoBehaviour to one or more entities + components.

```
public class EnemyAuthoring : MonoBehaviour
{
    public float MaxHealth = 100f;
    public GameObject ProjectilePrefab;

    class Baker : Baker<EnemyAuthoring>
    {
        public override void Bake(EnemyAuthoring authoring)
        {
            // Primary entity for this GameObject.
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Health { Value = authoring.MaxHealth });

            // Bake a GameObject reference into an Entity reference.
            AddComponent(self, new ProjectilePrefabRef
            {
                Value = GetEntity(authoring.ProjectilePrefab, TransformUsageFlags.Dynamic)
            });
        }
    }
}
```

Key APIs:

API	Purpose
`GetEntity(TransformUsageFlags)`	Entity for the current authoring GameObject.
`GetEntity(Object, TransformUsageFlags)`	Entity corresponding to another GameObject/prefab (registers a prefab entity).
`CreateAdditionalEntity(TransformUsageFlags)`	A second entity linked to the same authoring (e.g. sub-parts).
`AddComponent<T>(entity, value)`	Attach component data.
`AddBuffer<T>(entity)` / `SetBuffer<T>`	Attach a `DynamicBuffer<T>`.
`AddSharedComponent<T>`	Attach an `ISharedComponentData`.
`DependsOn(asset)`	Declare an asset dependency so the Baker re-runs when the asset changes.

---

4. `TransformUsageFlags`

Baking only adds `LocalTransform` / `LocalToWorld` / `Parent` when you ask for them via `TransformUsageFlags`. Unused transform components make chunks larger for no reason, so pick the **narrowest** flag that still works.

Flag	Adds	Typical use
`None`	Nothing	Pure data entities (config, singletons, logic-only).
`ManualOverride`	Nothing; you manage transforms yourself	Custom transform layouts.
`Renderable`	`LocalToWorld`	Static mesh that never moves.
`Dynamic`	`LocalTransform`, `LocalToWorld` (+ `Parent` if parented)	Anything that moves or is written by a system.
`WorldSpace`	`LocalTransform`, `LocalToWorld`; prevents parenting	Entities that ignore hierarchy.
`NonUniformScale`	Adds `PostTransformMatrix`	Non-uniform scale (uncommon in DOTS).

> `Dynamic` is the safe default for anything that a system will move or rotate.

---

5. Baking systems (advanced)

A Baker produces components from a single authoring. A **Baking System** runs **after** all Bakers and sees the full baked world — useful for cross-entity post-processing (resolving references, computing derived data).

```
[WorldSystemFilter(WorldSystemFilterFlags.BakingSystem)]
public partial struct EnemyWaveLinkSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs during bake only. Query entities tagged earlier by Bakers
        // and wire them together.
    }
}
```

Use this when a Baker doesn't know everything it needs (e.g. counts, cross-references). Keep it deterministic — baking must produce the same result on every run.

---

6. Example — authoring a projectile spawner

Authoring:

```
using Unity.Entities;
using UnityEngine;

public class ProjectileSpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
    public float Interval = 0.5f;

    class Baker : Baker<ProjectileSpawnerAuthoring>
    {
        public override void Bake(ProjectileSpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new ProjectileSpawner
            {
                Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                Interval = authoring.Interval,
                TimeLeft = authoring.Interval
            });
        }
    }
}
```

Component:

```
using Unity.Entities;

public struct ProjectileSpawner : IComponentData
{
    public Entity Prefab;
    public float Interval;
    public float TimeLeft;
}
```

A runtime system can then instantiate `Prefab` on a timer. See `02_Spawner Example.md`.

---

7. Troubleshooting

Symptom	Cause / Fix
Entity appears but is missing a component	The Baker didn't run (authoring was added after the SubScene closed) — toggle the SubScene to re-bake.
`GetEntity(prefab, ...)` returns `Entity.Null`	The referenced GameObject isn't a prefab asset or scene object reachable from the Baker. Ensure it's a prefab or in the same SubScene.

Symptom	Cause / Fix
Baker runs every frame in the Editor	`Bake()` has non-deterministic logic (e.g. `Random`, `Time.time`). Bakers must be pure functions of the authoring.
Entity has `LocalToWorld` but not `LocalTransform`	`TransformUsageFlags.Renderable` was used. Switch to `Dynamic` if the entity will move.
Changes to an authoring field don't re-bake	The Baker doesn't read that field or didn't call `DependsOn` for an external asset.
Baking is slow	Heavy work inside `Bake()`. Move cross-entity logic into a Baking System or a runtime one-shot initializer.

Chapter 5. Spawner Example

1. Overview

A spawner is a single entity that creates more entities — at startup, on a timer, or on an event. This page builds one end to end, covering:

- Authoring the spawner in a SubScene.
- Storing a **prefab entity** reference (baked from a GameObject prefab).
- Instantiating it at runtime through an **EntityCommandBuffer** so structural changes happen safely.

Reading `01_Baker Pattern & SubScene.md` first will make this easier.

---

2. Data model

```
using Unity.Entities;
using Unity.Mathematics;

public struct Spawner : IComponentData
{
    public Entity  Prefab;       // baked prefab entity
    public float3  SpawnPoint;
    public float   Interval;
    public float   TimeLeft;
    public int     RemainingCount;  // stop when 0
}
```

Two design choices worth calling out:

- `Prefab` is an **`Entity`**, not a `GameObject`. Bakers convert the GameObject prefab into a prefab entity; the runtime only sees entities.
- `TimeLeft` and `RemainingCount` live on the spawner entity so the system stays stateless.

---

3. Authoring + Baker

```
using Unity.Entities;
using Unity.Mathematics;
using UnityEngine;
```

```
public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
    public float      Interval = 0.5f;
    public int        Count    = 100;

    class Baker : Baker<SpawnerAuthoring>
    {
        public override void Bake(SpawnerAuthoring authoring)
        {
            var self = GetEntity(TransformUsageFlags.Dynamic);

            AddComponent(self, new Spawner
            {
                Prefab        = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic),
                SpawnPoint    = authoring.transform.position,
                Interval      = authoring.Interval,
                TimeLeft      = 0f,            // fires immediately on first update
                RemainingCount = authoring.Count
            });
        }
    }
}
```

Drop `SpawnerAuthoring` on an empty GameObject in the SubScene, assign a Prefab, set a count. Close the SubScene — the spawner is now an entity.

---

4. Runtime system

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnerSystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<Spawner>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        // Use the EndSimulation ECB so structural changes flush at a safe point.
        var ecb = SystemAPI
            .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
            .CreateCommandBuffer(state.WorldUnmanaged);

        foreach (var (spawnerRW, spawnerEntity) in
```

```
                SystemAPI.Query<RefRW<Spawner>>().WithEntityAccess())
        {
            ref var spawner = ref spawnerRW.ValueRW;

            if (spawner.RemainingCount <= 0)
            {
                ecb.DestroyEntity(spawnerEntity);   // done — remove the spawner
                continue;
            }

            spawner.TimeLeft -= dt;
            if (spawner.TimeLeft > 0f) continue;

            spawner.TimeLeft = spawner.Interval;
            spawner.RemainingCount--;

            var instance = ecb.Instantiate(spawner.Prefab);
            ecb.SetComponent(instance, LocalTransform.FromPosition(spawner.SpawnPoint));
        }
    }
}
```

What matters:

- `RequireForUpdate<Spawner>()` skips the system entirely when no spawner exists.
- The `EndSimulationEntityCommandBufferSystem` singleton provides a shared ECB that flushes at end-of-frame — safer than mutating through `EntityManager` mid-loop.
- `ecb.Instantiate(prefab)` creates a **deferred entity**; `ecb.SetComponent(instance, …)` schedules component writes on it. The `instance` handle is only valid inside this ECB's playback. See `14_EntityCommandBuffer · Deferred Entity.md`.

---

5. Scaling up — parallel spawner

If you need to spawn thousands per frame from many spawners, convert to an `IJobEntity` with a parallel ECB writer:

```
[BurstCompile]
public partial struct SpawnJob : IJobEntity
{
    public float                                DeltaTime;
    public EntityCommandBuffer.ParallelWriter  ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
                 Entity entity,
                 ref Spawner spawner)
    {
        if (spawner.RemainingCount <= 0)
        {
            ECB.DestroyEntity(chunkIndex, entity);
            return;
```

```
        }

        spawner.TimeLeft -= DeltaTime;
        if (spawner.TimeLeft > 0f) return;

        spawner.TimeLeft = spawner.Interval;
        spawner.RemainingCount--;

        var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
        ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
    }
}
```

And in the system:

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();

new SpawnJob { DeltaTime = SystemAPI.Time.DeltaTime, ECB = ecb }
    .ScheduleParallel();
```

The `chunkIndex` sort key is what keeps parallel playback deterministic. See `15_ParallelWriter · Deterministic Playback.md`.

---

6. Troubleshooting

Symptom	Cause / Fix
Nothing spawns	`Spawner` entity doesn't exist — open the SubScene, verify `SpawnerAuthoring` is attached and the SubScene is closed (yellow icon).
Prefab spawns at the world origin	The ECB never set `LocalTransform`. Either add `ecb.SetComponent(instance, LocalTransform.FromPosition(...))` or ensure the prefab already has a transform with the desired position.
Prefab is visible but not moving	Prefab was baked with `TransformUsageFlags.Renderable` instead of `Dynamic`.
`SystemAPI.GetSingleton<...ECBS.Singleton>()` throws	The ECB system isn't registered — happens in very stripped worlds. Fall back to `state.EntityManager` + manual ECB, or register the ECB system in your bootstrap.
Spawner never stops	`RemainingCount` decremented against a cached `Spawner` instead of `spawnerRW.ValueRW`. Always write through `RefRW.ValueRW`.
`ecb.SetComponent` on a deferred instance is ignored	Using a different ECB for `Instantiate` and `SetComponent`. Both calls must share the same ECB instance.

Chapter 6. ECS Core Concepts

1. The five nouns

Entities ECS boils down to five nouns:

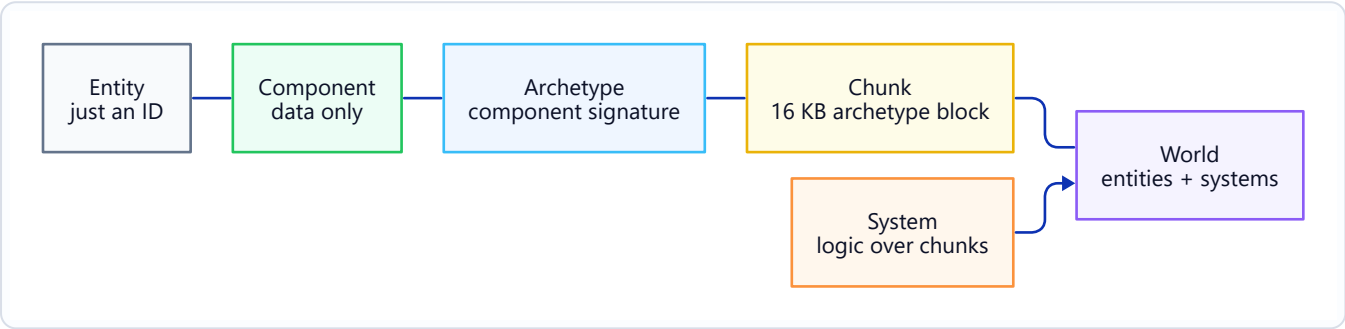

Figure: ECS Core Nouns.

Internalising this diagram is most of what it takes to read existing DOTS code.

---

2. Entity

An **entity** is just a 64-bit handle: `struct Entity { int Index; int Version; }`. It has no data and no methods beyond identity — all data lives in components.

```
Entity e = state.EntityManager.CreateEntity();   // fresh, no components
state.EntityManager.AddComponent<Health>(e);     // now it has data
state.EntityManager.DestroyEntity(e);            // handle becomes stale
```

The `Version` field invalidates old references: if you destroy and recreate at the same `Index`, the old `Entity` value no longer matches.

---

3. Component

A **component** is a plain data container. It carries no behaviour. Code lives in systems.

```
public struct Health : IComponentData
{
    public float Value;
```

```
    public float Max;
}
```

There are several component variants (unmanaged, managed, buffer, shared, cleanup, tag, chunk, enableable). See `05_Component Types.md`.

---

4. Archetype

An **archetype** is a unique set of component types that some entities share.

- An entity with `{ LocalTransform, Health, Enemy }` belongs to archetype A.
- An entity with `{ LocalTransform, Health, Enemy, Burning }` belongs to archetype B — a different one, because adding `Burning` changed the component set.

Moving between archetypes (adding/removing a component) is a **structural change** and physically moves the entity to a new chunk. See `13_Structural Change & Safety.md`.

---

5. Chunk

Entities in the same archetype are packed into **16 KB chunks**. Within a chunk, each component type forms a contiguous array — great for cache locality.

Fact	Implication
Chunks are 16 KB	Each component eats a slice; the more (or larger) components in the archetype, the fewer entities per chunk.
Data is laid out column-wise (SoA) per component	`IJobChunk` can iterate one component array at a time and vectorise with Burst.
Adding/removing a component moves the entity	Structural change. Query iterators assume the archetype is stable while iterating.
Chunks track component change versions	Systems can skip chunks that didn't change since last run (`EntityQuery.SetChangedVersionFilter`).

You can inspect live chunk layout in **Window** → **Entities** → **Archetypes**.

---

6. System

A **system** is a piece of logic that runs each frame, typically over a query of entities.

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var health in SystemAPI.Query<RefRW<Health>>())
        {
            health.ValueRW.Value = math.min(health.ValueRW.Value + dt, health.ValueRW.Max);
        }
    }
}
```

Systems are registered automatically and live inside a **SystemGroup**. See `08_System — ISystem vs SystemBase.md` and `09_System Group & Update Order.md`.

---

7. World

A **World** is a container that owns:

- An **EntityManager** (creates, destroys, queries entities).
- A set of **systems** and their SystemGroups.
- The chunks themselves (via the EntityManager).

The default runtime world is `DefaultGameObjectInjectionWorld`. Most games have one world in production; multi-world setups show up in Netcode (server / client) or in editor tooling. See `16_Netcode Client-Server World & Bootstrap.md` for the Netcode world split.

```
World world = World.DefaultGameObjectInjectionWorld;
EntityManager em = world.EntityManager;

Entity e = em.CreateEntity(typeof(Health));
em.SetComponentData(e, new Health { Value = 100, Max = 100 });
```

---

8. EntityManager

The `EntityManager` is the main-thread API for structural changes: create, destroy, add/remove components, instantiate prefab-entities, copy worlds, etc. Structural changes through `EntityManager` are immediate and can invalidate any in-flight query iterators or job references.

For deferred structural changes from jobs or mid-iteration, use an **EntityCommandBuffer** — see `14_EntityCommandBuffer · Deferred Entity.md`.

---

9. Putting it together

```
World
└── EntityManager
    └── Archetype { LocalTransform, Health, Enemy }
        └── Chunk  (up to ~N entities)
            └── Entity { Index, Version }
                ├── LocalTransform
                ├── Health
                └── Enemy (tag)
```

A system that queries `{ LocalTransform, Health }` touches every chunk whose archetype **contains both**. That includes the archetype above — plus any other archetype that also has `Enemy` plus extra components.

---

10. Troubleshooting

Symptom	Cause / Fix
`Entity` lookups return stale data	The entity was destroyed and recreated — the `Version` no longer matches. Re-resolve the entity.
Query matches nothing but the entity clearly has the component	Component was added in a different world, or the component is enableable and currently disabled.
`Archetype` count explodes	Too many unique component combinations (often from per-entity tags with high cardinality). Consolidate using shared components or enableable flags.
Chunk contains only 1-2 entities	Archetype is oversized — remove components the entity doesn't need, or split into multiple entities/archetypes.
`InvalidOperationException : … invalidated … during` iteration	A structural change happened mid-query. Use an ECB, or batch changes after the loop.

Chapter 7. Entity References — Entity · EntityPrefabReference · UnityObjectRef

1. Overview

DOTS code uses a small set of reference types. They are easy to mix up because they all point at "something", but they live at different points in the authoring-to-runtime pipeline.

Type	Namespace	What it references	Typical use
`Entity`	`Unity.Entities`	An entity inside one ECS `World`.	Runtime relationships and component fields.
`EntityPrefabReference`	`Unity.Entities.Serialization`	A baked entity prefab asset that can be loaded later.	Streaming and content workflows.
`UnityObjectRef<T>`	`Unity.Entities`	A `UnityEngine.Object` asset/object through an unmanaged wrapper.	Referencing meshes, materials, textures, audio clips, and authored assets from ECS data.

This page is only about DOTS / Entities reference types. Engine-wide `UnityEngine.Object` identifier APIs are outside the scope of this manual.

---

2. `Entity`

`Entity` is the runtime ECS handle. It is an unmanaged value that identifies a row of component data inside one `World`.

```
using Unity.Entities;

public struct Target : IComponentData
{
    public Entity Value;
}
```

Important rules:

- An `Entity` value is only meaningful inside the `World` that created it.
- Destroying an entity invalidates the old handle.

- Entity indices can be reused; use `EntityManager.Exists(entity)` before dereferencing a stored handle you did not just query.
- Do not save raw `Entity` values to disk or send them across independent worlds as stable IDs.

```
if (state.EntityManager.Exists(target.Value))
{
    var transform = state.EntityManager.GetComponentData<LocalTransform>(target.Value);
}
```

---

3. Entity references in baking

When a Baker converts authoring objects into ECS data, use `GetEntity` to convert a referenced authoring object or prefab into an `Entity` reference.

```
using Unity.Entities;
using UnityEngine;

public class SpawnerAuthoring : MonoBehaviour
{
    public GameObject Prefab;
}

public struct Spawner : IComponentData
{
    public Entity Prefab;
}

public class SpawnerBaker : Baker<SpawnerAuthoring>
{
    public override void Bake(SpawnerAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new Spawner
        {
            Prefab = GetEntity(authoring.Prefab, TransformUsageFlags.Dynamic)
        });
    }
}
```

The result is a runtime ECS reference. It is not a managed `GameObject` reference.

---

4. `EntityPrefabReference`

`EntityPrefabReference` is for prefab content that should be loaded through the Entities content pipeline instead of being directly embedded as an `Entity` reference in the current scene.

```
using Unity.Entities;
using Unity.Entities.Serialization;
using UnityEngine;

public class ProjectileLibraryAuthoring : MonoBehaviour
{
    public GameObject ProjectilePrefab;
}

public struct ProjectileLibrary : IComponentData
{
    public EntityPrefabReference ProjectilePrefab;
}

public class ProjectileLibraryBaker : Baker<ProjectileLibraryAuthoring>
{
    public override void Bake(ProjectileLibraryAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.None);
        AddComponent(entity, new ProjectileLibrary
        {
            ProjectilePrefab = new EntityPrefabReference(authoring.ProjectilePrefab)
        });
    }
}
```

Use `EntityPrefabReference` when the prefab belongs to a content-management or streaming workflow. Use a plain `Entity` prefab reference when the prefab is baked into the same SubScene and is available with the scene.

---

5. `UnityObjectRef<T>`

`UnityObjectRef<T>` lets unmanaged ECS data reference a `UnityEngine.Object` without turning the component into a managed component.

```
using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}
```

Bake it from authoring data:

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
```

```
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>
{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}
```

Dereference `.Value` only on the main thread or in managed presentation code:

```
Mesh mesh = visual.Mesh.Value;
Material material = visual.Material.Value;
```

Rules:

- The stored wrapper is unmanaged and can live inside `IComponentData`.
- The target object is still a managed Unity object; `.Value` is not a Burst job operation.
- The referenced object must still be loaded when you dereference it.

---

6. Choosing the right reference

Need	Use
Point from one entity to another in the same world	`Entity`
Store a prefab baked into the same SubScene	`Entity` prefab reference from `GetEntity(authoring.Prefab, ...)`
Store a prefab for content streaming / deferred loading	`EntityPrefabReference`
Store a mesh, material, texture, audio clip, or ScriptableObject reference in unmanaged ECS data	`UnityObjectRef<T>`
Persist an entity across sessions	A game-owned stable ID, not a raw `Entity`

---

7. Troubleshooting

Symptom	Cause / Fix
Stored `Entity` no longer exists	The entity was destroyed or belongs to another world. Re-resolve it or check `EntityManager.Exists`.
Prefab reference is `Entity.Null` after baking	The Baker did not call `GetEntity` on the prefab, or the prefab was not included in a baked scene/content workflow.
`UnityObjectRef<T>.Value` returns `null`	The referenced Unity object is not loaded or was destroyed. Guard the dereference and reload/re-resolve the asset.
Burst job cannot use a mesh/material reference	`UnityObjectRef<T>` storage is unmanaged, but `.Value` touches managed Unity objects. Move dereferencing to main-thread presentation code.
Save file stores raw `Entity` values	Raw entity handles are world-local and not stable. Store a game-owned ID or content key instead.

Chapter 8. Component Types

1. Overview

A "component" in ECS is any data container attached to an entity. Entities exposes several flavours with different storage, lifetime, and cost profiles. Picking the right one is half the work of designing an ECS system.

Quick map:

Interface / Form	Storage	Structural change on mutate?	Typical use
`IComponentData` (unmanaged `struct`)	In chunk, SoA	No (on value change)	Default component
`IComponentData` (managed `class`)	Side table	No	Reference types (meshes, GameObject refs)
`IBufferElementData`	In chunk + heap overflow	No	Per-entity dynamic arrays
`ISharedComponentData`	Per-chunk	Yes (moves entity to new chunk)	Grouping entities that share config
`ICleanupComponentData`	In chunk	Normal	Entity-destroy cleanup protocol
`ICleanupSharedComponentData`	Per-chunk	Normal	Shared-resource cleanup protocol
Tag (empty `IComponentData`)	Zero-size in chunk	No	Filter marker, no data
Chunk component	Per-chunk	Yes	Metadata computed once per chunk
Enableable (`IEnableableComponent`)	In chunk + bit mask	No	Turn behaviour on/off per entity

The rest of this page expands each row.

---

2. Unmanaged `IComponentData`

The default. Lives directly in the chunk as tightly-packed SoA data. Burst-friendly.

```
public struct Velocity : IComponentData
{
    public float3 Value;
}
```

Rules:

- Must be a `struct` — no reference fields (no `class`, no `string`, no `List<T>`).
- Allowed field types: primitives, `Unity.Mathematics`, other unmanaged structs, `Entity`, `FixedString*`, `BlobAssetReference<T>`, `UnityObjectRef<T>`.
- Read via `RefRO<T>`, write via `RefRW<T>` from `SystemAPI.Query`.

---

3. Managed `IComponentData`

Lives in a side table indexed by entity. Works with reference types but costs a GC allocation and breaks Burst compatibility for that component.

```
public class EnemyAI : IComponentData
{
    public AIModel Brain;      // a managed class
    public List<Entity> Allies;
}
```

Use when you genuinely need a managed reference (mesh, texture, authored ScriptableObject). Prefer `UnityObjectRef<T>` inside an unmanaged component when you just need a weak reference to a `UnityEngine.Object`.

---

4. `IBufferElementData` — dynamic buffers

A `DynamicBuffer<T>` is a per-entity resizable array.

```
public struct Waypoint : IBufferElementData
{
    public float3 Position;
}

var buffer = state.EntityManager.AddBuffer<Waypoint>(entity);
buffer.Add(new Waypoint { Position = new float3(1, 0, 0) });
```

Storage: up to `InternalBufferCapacity` elements live in-chunk; beyond that the buffer spills to the heap. Tune capacity with `[InternalBufferCapacity(N)]`.

---

5. `ISharedComponentData` — grouping

Shared components carry a **value that groups chunks**. Two entities with the same `ISharedComponentData` value end up in the same chunk; different values produce different chunks.

```
public struct RenderMeshGroup : ISharedComponentData
{
    public int MaterialID;
}
```

Consequences:

- Changing the value moves the entity to another chunk → **structural change**, expensive.
- Great for data that doesn't change frequently but varies per group (material sets, team IDs, bake regions).
- Values are stored in a managed table if the type contains references (`ISharedComponentData` can be managed).

---

6. Cleanup components — `ICleanupComponentData`

Mark an entity so that, when it is "destroyed," it stays alive with only its cleanup components until your system processes them.

```
public struct ConnectionCleanup : ICleanupComponentData
{
    public int ConnectionHandle;
}

// DestroyEntity(e) removes all normal components but leaves ConnectionCleanup
// behind so you can close the socket in a dedicated system, then destroy the
// cleanup component explicitly.
```

The sibling `ICleanupSharedComponentData` applies the same protocol to shared resources.

Use cases: releasing native resources, un-registering from external systems, staged destruction across frames.

---

7. Tag components

A **tag** is an empty `IComponentData`:

```
public struct Enemy : IComponentData {}
```

Zero bytes in a chunk, queryable like any other component. Ideal for "is a type of X" markers. Overusing tags (hundreds of unique ones) fragments archetypes — use enableable components or shared components for high-cardinality states.

---

8. Chunk components

A chunk component carries **one value per chunk**, not per entity.

```
state.EntityManager.AddChunkComponentData<RegionBounds>(query, new RegionBounds { ... });
```

Typical use: precomputed data that applies to all entities in the chunk (bounds, region ID, partition key). Mutating a chunk component causes a structural change.

---

9. Enableable components (`IEnableableComponent`)

See the dedicated page `06_Enableable Component.md` for depth. Summary here: a component that can be toggled on/off per entity **without** a structural change, via a per-chunk bitmask.

```
public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}

state.EntityManager.SetComponentEnabled<Stunned>(entity, false);  // no structural change
```

Queries automatically skip entities whose enableable components are disabled — a cheap, common way to temporarily exclude entities from a system.

---

10. Decision guide

I want to...	Use
Store per-entity numeric data	Unmanaged `IComponentData`
Hold a managed reference per entity	Managed `IComponentData` or `UnityObjectRef<T>`
Store a variable-length list per entity	`IBufferElementData`
Group entities for batching	`ISharedComponentData`
Toggle a component on/off per frame	Enableable component

I want to...	Use
Just mark a type	Tag (empty `IComponentData`)
Precompute data for a whole chunk	Chunk component
Run code when an entity is destroyed	Cleanup component

---

11. Troubleshooting

Symptom	Cause / Fix
`InvalidOperationException: The type T must be unmanaged`	You used a managed type where unmanaged is required. Switch the component or the API (e.g. `ComponentLookup<T>` vs `ManagedAPI.GetComponent<T>`).
Setting a shared component value stutters	Each change is a structural change. Batch with an ECB or accept that this value is rarely mutated.
`DynamicBuffer<T>` spills to the heap	`[InternalBufferCapacity(N)]` is too small for your usage. Increase it or keep sizes bounded.
Entity reappears after `DestroyEntity`	A cleanup component is still attached. Remove it explicitly once cleanup is done.
Too many archetypes	Tag explosion. Convert unique-per-entity "states" to enableable components.
Tag component isn't filtering a query	Tag components have no data, so you can't request a `RefRW<Tag>` / `RefRO<Tag>` in the `SystemAPI.Query<...>` tuple. Apply them via `.WithAll<Tag>()` / `.WithNone<Tag>()` instead.

Chapter 9. Enableable Component

1. Why enableable components exist

Adding or removing a component is a **structural change**: the entity moves to a different chunk, queries must re-resolve, references may be invalidated. If you do this every frame per entity (e.g. toggling "Stunned" on and off), the cost dominates the frame.

An **enableable component** gives you a way to turn behaviour on and off **without** a structural change. The component is always present on the chunk; a bit in a per-chunk mask decides whether each entity "has" it from a query's point of view.

---

2. The interface

Implement both `IComponentData` and `IEnableableComponent`:

```
using Unity.Entities;

public struct Stunned : IComponentData, IEnableableComponent
{
    public float Duration;
}
```

You can also make an `IBufferElementData` enableable.

---

3. Toggling

From a system:

```
// Turn Stunned off without removing the component
state.EntityManager.SetComponentEnabled<Stunned>(entity, false);

// From an ECB (deferred, safe mid-iteration)
ecb.SetComponentEnabled<Stunned>(entity, true);

// From a job with a ComponentLookup
var stunnedLookup = SystemAPI.GetComponentLookup<Stunned>();
stunnedLookup.SetComponentEnabled(entity, false);
```

Reading the current state:

```
bool isStunned = state.EntityManager.IsComponentEnabled<Stunned>(entity);
```

All three paths operate on a bitmask; none of them trigger a structural change.

---

4. Query behaviour

`SystemAPI.Query<...>` and `EntityQuery`-based iteration **skip** entities whose enableable components are currently disabled — by default, you only see entities where every listed component is enabled.

```
foreach (var stunned in SystemAPI.Query<RefRW<Stunned>>())
{
    // only fires for entities where Stunned is enabled
}
```

Override this if you need to see disabled entities too:

```
foreach (var (stunnedRW, entity) in SystemAPI
             .Query<RefRW<Stunned>>()
             .WithEntityAccess()
             .WithOptions(EntityQueryOptions.IgnoreComponentEnabledState))
{
    // fires for ALL entities with the component, enabled or not
}
```

---

5. Example — stun that expires

A single `IJobEntity` cannot receive both `ref Stunned` (a `RefRW<T>`) **and** `EnabledRefRW<Stunned>` for the same component — Entities explicitly disallows wrapping the same component in both forms in one job. Two valid patterns:

5.1 Read + write value, toggle enabled bit via ECB

```
[BurstCompile]
public partial struct StunTickJob : IJobEntity
{
    public float                                DeltaTime;
    public EntityCommandBuffer.ParallelWriter  ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
                 Entity entity,
                 ref Stunned stunned)
    {
        stunned.Duration -= DeltaTime;
        if (stunned.Duration <= 0f)
```

```
            ECB.SetComponentEnabled<Stunned>(chunkIndex, entity, false);
    }
}
```

Playback happens at the next ECB boundary (e.g. `EndSimulationECBS`) — see `14_EntityCommandBuffer · Deferred Entity.md`. No structural change; just a bitmask flip at a known point.

5.2 Toggle the enabled bit directly when you do not need the component value

```
[BurstCompile]
public partial struct StunDisableJob : IJobEntity
{
    void Execute(EnabledRefRW<Stunned> stunnedEnabled)
    {
        stunnedEnabled.ValueRW = false;   // no structural change
    }
}
```

`EnabledRefRW<T>` is the only `Stunned` reference on this job, so it compiles. Use this shape when a system is purely a switcher.

> Want to both read the value **and** toggle the enabled bit on the same entity in one pass? Do it from the main thread via `EntityManager.SetComponentEnabled<T>(entity, false)`, or split across two jobs chained through `state.Dependency`.

---

6. When to prefer enableable over add/remove

Situation	Pick
High-frequency toggling (per-frame, per-event)	**Enableable**
Adds state that isn't always present (per-entity modifier with variable fields)	**Enableable**
Low-frequency archetype change (e.g. adding a permanent capability)	Add/remove the component as usual
Filtering many systems uniformly	**Enableable** (queries pick it up automatically)

General rule: if the "has this component" bit flips multiple times in an entity's lifetime, make it enableable.

---

7. Interaction with other features

Feature	Behaviour
`WithChangeFilter<T>`	Toggling the enable bit also bumps the component's change version.
`ICleanupComponentData`	A cleanup component cannot also be enableable.
Netcode for Entities	Enableable components can be ghost-synced, but toggles must go through the authoritative side.
Serialisation / world export	The enable bits are preserved.

---

8. Troubleshooting

Symptom	Cause / Fix
Query includes entities that should be "off"	Component isn't implementing `IEnableableComponent`, or `WithOptions(EntityQueryOptions.IgnoreComponentEnabledState)` is set.
`SetComponentEnabled<T>` throws "type T does not support being enableable"	The struct only implements `IComponentData` — add `IEnableableComponent`.
`EnabledRefRW<T>` parameter is rejected by source generator	Job or system is missing `partial`, or the component isn't enableable.
Source generator error "cannot have both RefRW and EnabledRefRW for the same component"	Split the logic — use `ref T` **or** `EnabledRefRW<T>` per job, not both on the same component. See §5 for the two patterns.
Change filter fires spuriously	Toggling `Enabled` counts as a change for the purposes of `WithChangeFilter`. Expected — combine with `EnabledRefRO` reads rather than toggling.
ECB `SetComponentEnabled` is ignored	You called it on a deferred entity before the ECB that created it flushed. Both must be in the same ECB cycle.

Chapter 10. Singleton Component

1. What a singleton is (and isn't)

A **singleton** in Entities is just a component that is expected to exist on **exactly one entity** in the world. There is no special `ISingleton` interface — any `IComponentData` becomes a singleton by convention, and `SystemAPI` provides ergonomic accessors that assert the "exactly one" rule.

Typical uses: game state, global config, time/tick sources, network state, central ECB registrations.

```
public struct GameState : IComponentData
{
    public int   Score;
    public float TimeScale;
    public bool  Paused;
}
```

---

2. The accessors

Inside any `ISystem` / `SystemBase`:

```
// Reads
GameState gs      = SystemAPI.GetSingleton<GameState>();
RefRW<GameState> gsRW  = SystemAPI.GetSingletonRW<GameState>();
bool has          = SystemAPI.HasSingleton<GameState>();
bool got          = SystemAPI.TryGetSingleton<GameState>(out var gs2);

// Writes
SystemAPI.SetSingleton(new GameState { Score = 10 });

// Entity handle, if you need it
Entity e          = SystemAPI.GetSingletonEntity<GameState>();

// Buffers work too
DynamicBuffer<Waypoint> buf = SystemAPI.GetSingletonBuffer<Waypoint>();
```

`Get*` variants throw if zero or more than one singleton exists. `TryGet*` returns `false` in those cases. `RW` variants mark the component as changed for change-tracking.

---

3. Creating the singleton

Two common patterns:

3.1 Bake it from a SubScene

```
public class GameStateAuthoring : MonoBehaviour
{
    public float InitialTimeScale = 1f;

    class Baker : Baker<GameStateAuthoring>
    {
        public override void Bake(GameStateAuthoring authoring)
        {
            var e = GetEntity(TransformUsageFlags.None);
            AddComponent(e, new GameState
            {
                Score     = 0,
                TimeScale = authoring.InitialTimeScale,
                Paused    = false
            });
        }
    }
}
```

Attach `GameStateAuthoring` to a single GameObject in the SubScene; the Baker produces exactly one entity with the singleton.

3.2 Create at system startup

```
public void OnCreate(ref SystemState state)
{
    var e = state.EntityManager.CreateEntity(typeof(GameState));
    state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
}
```

Use this for data that shouldn't live in authoring (server-injected config, procedurally generated, etc.).

---

4. Gating a system on the singleton

If a system only makes sense after the singleton exists:

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<GameState>();
}
```

`RequireForUpdate<T>()` tells the system to **skip `OnUpdate` entirely** when the component isn't present. Avoids defensive null checks inside `OnUpdate`.

---

5. Read-only vs read-write performance

- `GetSingleton<T>()` — reads a copy. No change tracking.
- `GetSingletonRW<T>()` — returns a `RefRW<T>` pointing into the chunk. Assigning `.ValueRW` marks the component changed, which is what you want if other systems use `WithChangeFilter<T>`.
- `SetSingleton` — full write, always bumps the version.

Prefer `GetSingleton` in read-heavy systems; switch to `GetSingletonRW` only when you actually mutate.

---

6. Multiple candidates — the "singleton with more than one" trap

`GetSingleton<T>` throws `InvalidOperationException: expected exactly one entity with component T, found N` when two entities end up with the same singleton component. Common causes:

- Two GameObjects in the SubScene both have the authoring script attached.
- A test fixture created the singleton at runtime **and** the SubScene also bakes it.
- A system creates the singleton in `OnCreate` without guarding with `HasSingleton`.

Defensive pattern:

```
public void OnCreate(ref SystemState state)
{
    if (!SystemAPI.HasSingleton<GameState>())
    {
        var e = state.EntityManager.CreateEntity(typeof(GameState));
        state.EntityManager.SetComponentData(e, new GameState { TimeScale = 1f });
    }
}
```

---

7. Buffer singletons

A `DynamicBuffer<T>` can be a singleton too:

```
public struct QueuedEvent : IBufferElementData
{
    public int Type;
    public int Payload;
}

// Access:
DynamicBuffer<QueuedEvent> events = SystemAPI.GetSingletonBuffer<QueuedEvent>();
events.Add(new QueuedEvent { Type = 1 });
```

Useful for global event queues and command streams.

---

8. Troubleshooting

Symptom	Cause / Fix
`InvalidOperationException: expected exactly one entity with component T, found 0`	Singleton entity was never created. Add a baker, or create in `OnCreate` with a `HasSingleton` guard.
`... found 2`	Two entities carry the component. Deduplicate in authoring or cleanup at startup.
`GetSingletonRW` doesn't appear to mark the component changed	You wrote to `.ValueRO` or ignored the `RefRW`. Only `.ValueRW` triggers change tracking.
System runs before the singleton is created	Use `RequireForUpdate<T>` or order your systems so the creator runs first (`[UpdateBefore]`).
Singleton is modified from two systems with no ordering	Add `[UpdateAfter]`/`[UpdateBefore]` or merge the logic. Race conditions on singletons are especially painful to debug.
Netcode: singleton value diverges between client and server	Ghost-sync the singleton, or rebuild it deterministically from synchronised inputs.

Chapter 11. System — ISystem vs SystemBase

1. Overview

Entities has two ways to declare a system: **`ISystem`** (unmanaged `struct`) and **`SystemBase`** (managed `class`). Pick `ISystem` unless you specifically need what `SystemBase` gives you.

	`ISystem`	`SystemBase`
Declared as	`partial struct : ISystem`	`partial class : SystemBase`
Memory	Unmanaged, stack-allocatable	Managed, GC-owned
Burst-compilable	**Yes** (add `[BurstCompile]`)	No
Managed fields	No	Yes
Managed API access	Through `EntityManager` + `ManagedAPI`	Directly
Default choice	✅	Only when you must

---

2. `ISystem` — the default

```
using Unity.Burst;
using Unity.Entities;

[BurstCompile]
public partial struct EnemyAISystem : ISystem
{
    [BurstCompile]
    public void OnCreate(ref SystemState state)
    {
        // Called once when the system is created.
        state.RequireForUpdate<EnemySpawnConfig>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var config = SystemAPI.GetSingleton<EnemySpawnConfig>();
        // ... logic
    }

    [BurstCompile]
    public void OnDestroy(ref SystemState state)
    {
        // Called once when the system is destroyed.
```

```
    }
}
```

Mandatory bits:

- Struct must be `partial` — the source generator adds a hidden wrapper.
- All three methods (`OnCreate`, `OnUpdate`, `OnDestroy`) are optional — omit the ones you don't need.
- `[BurstCompile]` on the struct **and** on each method you want Burst-compiled.

---

3. `SystemBase` — when a managed system is unavoidable

```
using Unity.Entities;

public partial class PlayerInputSystem : SystemBase
{
    private InputActions _input;   // managed field — the reason SystemBase exists

    protected override void OnCreate()
    {
        _input = new InputActions();
        _input.Enable();
    }

    protected override void OnUpdate()
    {
        var move = _input.Gameplay.Move.ReadValue<Vector2>();
        // ... dispatch to components
    }

    protected override void OnDestroy() => _input.Dispose();
}
```

Reach for `SystemBase` when:

- You hold a managed field that must live with the system (Input System objects, UI Toolkit handles, managed services, etc.).
- You need to call managed APIs directly inside `OnUpdate` without marshalling through the `EntityManager`.

Everything in `SystemBase` runs on the main thread without Burst.

---

4. System lifecycle

Both flavours share the same hook names:

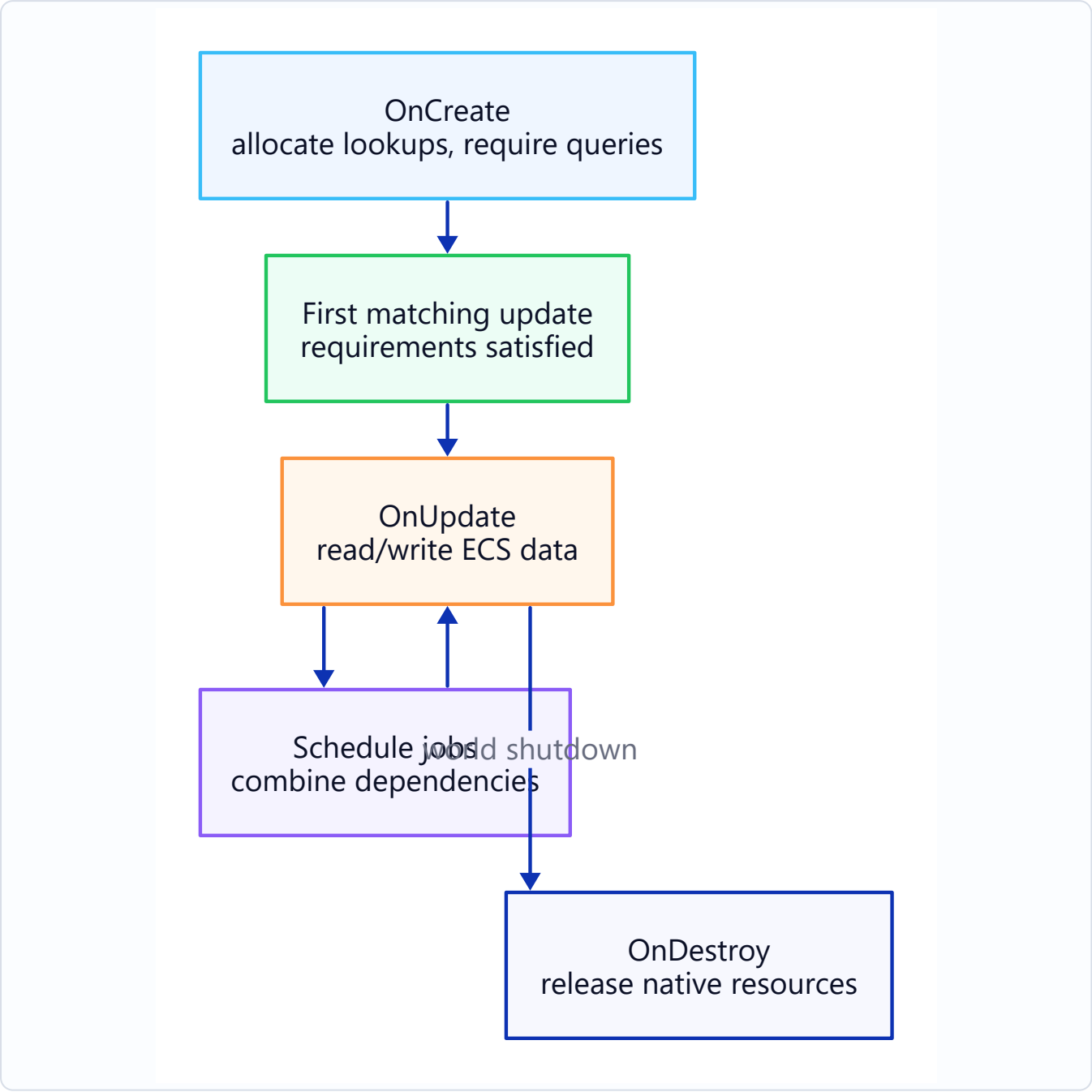

Figure: System Lifecycle.

Hook	When it fires	Typical use
`OnCreate`	Once, at system creation	Cache queries, `RequireForUpdate<T>`, allocate native containers
`OnStartRunning`	Query(s) changed from empty → non-empty	Reset per-session state
`OnUpdate`	Every frame in the parent SystemGroup	Main work

Hook	When it fires	Typical use
`OnStopRunning`	Query(s) changed from non-empty → empty	Flush buffered state
`OnDestroy`	Once, at world teardown	Release native/managed resources

`RequireForUpdate<T>()`

Skip `OnUpdate` entirely when a required component or singleton is absent:

```
public void OnCreate(ref SystemState state)
{
    state.RequireForUpdate<EnemySpawnConfig>();
    state.RequireForUpdate<GameRunning>();
}
```

This is the standard way to gate a system behind "config exists" / "game started."

---

5. Choosing between the two

Rules of thumb:

1. **Start with `ISystem`.** It's Burst-friendly and GC-free.
2. **Switch to `SystemBase` only when a managed field forces your hand.** Even then, consider splitting: keep the managed work in a thin `SystemBase` that pushes results into components, and run the heavy logic in an `ISystem` that reads those components.
3. **Do not mix** by adding a managed field to an `ISystem` struct — it won't compile. The split above is the idiomatic workaround.

Example split:

```
// Managed — bridges Input System into components.
public partial class InputBridgeSystem : SystemBase
{
    private InputActions _input;
    // ... fills a PlayerInput singleton each frame
}

// Unmanaged + Burst — reads the PlayerInput singleton and drives gameplay.
[BurstCompile]
public partial struct PlayerMoveSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state) { /* ... */ }
}
```

---

6. `SystemState` quick reference

Inside an `ISystem`, `ref SystemState state` is your window into the world.

Member	Purpose
`state.EntityManager`	Main-thread structural changes.
`state.World`	The owning World.
`state.Dependency`	The current job dependency handle — combine with your scheduled jobs.
`state.GetEntityQuery(...)`	Build or retrieve a cached query.
`state.RequireForUpdate<T>()`	Gate the system on a component's presence.
`state.Enabled`	Set to `false` to skip the system without destroying it.

In a `SystemBase`, these are members of the class itself (e.g. `this.EntityManager`).

---

7. Troubleshooting

Symptom	Cause / Fix
`error CS0246: The type or namespace name 'Entities' could not be found`	`using Unity.Entities;` missing.
`OnUpdate` never fires	`RequireForUpdate<T>` is gating on a component that doesn't exist yet — verify the component is created elsewhere.
Compile error: "partial struct must be declared partial"	`ISystem` requires `partial` so the source generator can extend the struct.
Burst warning: "method is not burst compiled"	Missing `[BurstCompile]` on the method. The attribute on the struct alone is not enough.
System runs on the wrong world (client vs server with Netcode)	Add `[WorldSystemFilter(...)]` to pick ServerSimulation / ClientSimulation / LocalSimulation.
Managed field can't be added to `ISystem`	Expected — split the managed part into a `SystemBase` as shown above.
System doesn't appear in `Window → Entities → Systems`	Missing `partial`, outside the default world, or excluded by `[DisableAutoCreation]`.

Chapter 12. System Group & Update Order

1. Overview

Every system lives inside a **SystemGroup**. Groups decide when their children run each frame, and ordering attributes decide the sequence **within** a group. Getting the grouping right is how you avoid frame-late writes, race-conditions, and "why does this entity jitter" bugs.

2. Built-in groups

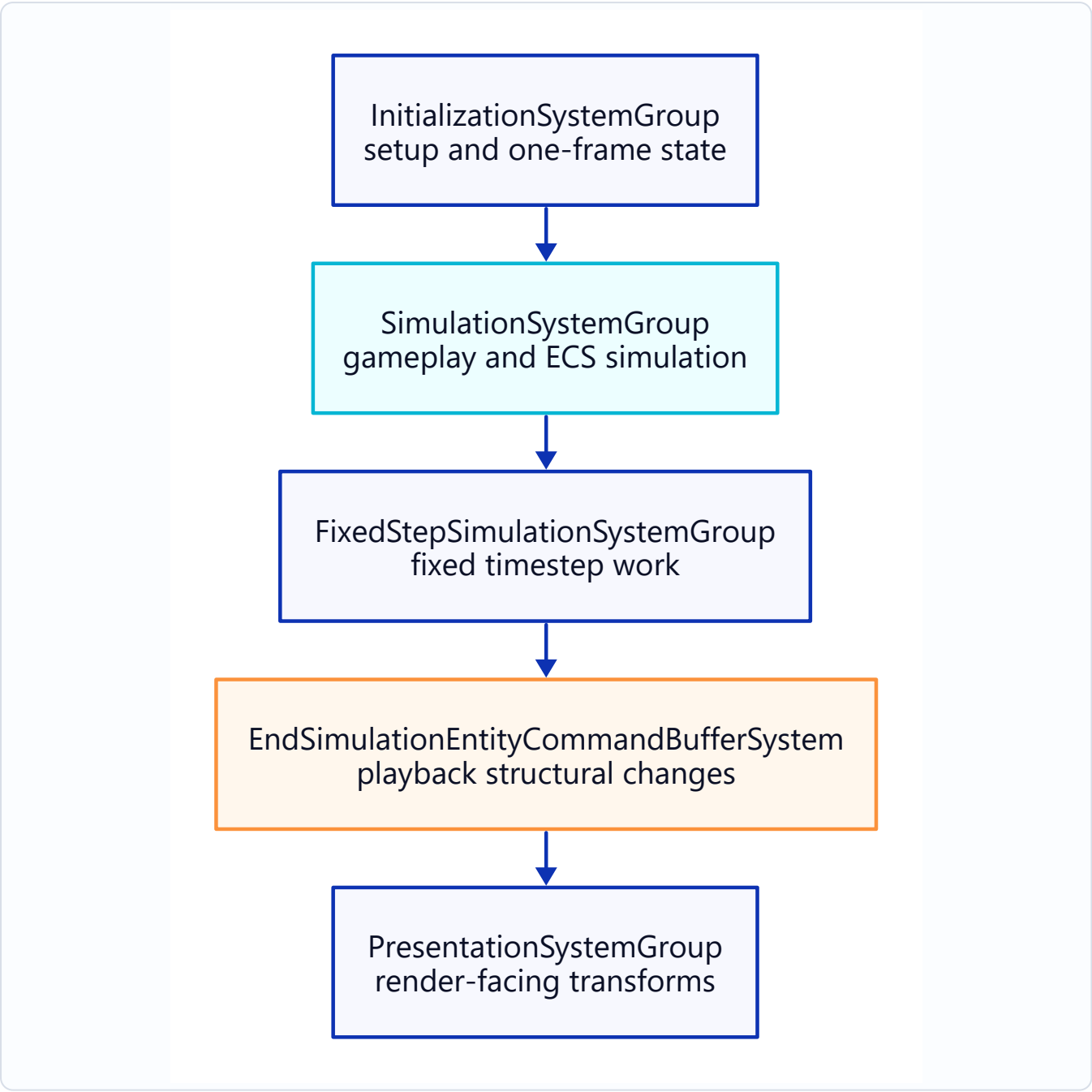

Figure: System Group and Update Order.

Group	When it runs	Typical contents
`InitializationSystemGroup`	Once per frame, before simulation	One-shot initializers, bootstrap tasks
`SimulationSystemGroup`	Once per frame	Most gameplay logic
`FixedStepSimulationSystemGroup`	**Fixed timestep** (default 60 Hz); may run 0, 1, or N times per frame	Physics, deterministic step logic
`PresentationSystemGroup`	Once per frame, after simulation	Render prep, interpolation

Group	When it runs	Typical contents
`BeginSimulationEntityCommandBufferSystem`	First thing in Simulation	Ordered ECB playback (early)
`EndSimulationEntityCommandBufferSystem`	Last thing in Simulation	Ordered ECB playback (late)

Netcode for Entities adds more (`PredictedSimulationSystemGroup`, `GhostSendSystem`, etc.) — they nest inside the ones above. See `16_Netcode Client-Server World & Bootstrap.md` and `19_Netcode Prediction & Rollback.md`.

---

3. Placing a system in a group

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
public partial struct EnemyAISystem : ISystem { /* ... */ }
```

If you omit `[UpdateInGroup]`, the system defaults to `SimulationSystemGroup`.

Nesting

Groups can live inside other groups:

```
[UpdateInGroup(typeof(SimulationSystemGroup), OrderFirst = true)]
public partial class GameplaySystemGroup : ComponentSystemGroup { }

[UpdateInGroup(typeof(GameplaySystemGroup))]
public partial struct MovementSystem : ISystem { /* ... */ }
```

`ComponentSystemGroup` is the base class for custom groups. They act as both systems and containers.

---

4. Ordering within a group

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateAfter(typeof(EnemyAISystem))]
[UpdateBefore(typeof(MovementSystem))]
public partial struct EnemyPathfindingSystem : ISystem { /* ... */ }
```

Attribute	Meaning
`[UpdateBefore(typeof(X))]`	Run before `X` (both must be in the same group).
`[UpdateAfter(typeof(X))]`	Run after `X`.
`OrderFirst = true` (on `[UpdateInGroup]`)	Run as early as possible in the group.
`OrderLast = true`	Run as late as possible.

> Tip: favour `[UpdateAfter]` chains over `OrderFirst/Last` — they read better in code review and survive refactors.

---

5. `[RequireMatchingQueriesForUpdate]`

A newer attribute (Entities 1.x+) that tells a system to **skip `OnUpdate` unless at least one entity matches each of its queries**:

```
[RequireMatchingQueriesForUpdate]
public partial struct EnemyAISystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Only runs when there is at least one entity matching every query the system uses.
    }
}
```

Compare with `state.RequireForUpdate<T>()`, which requires a **specific** component/singleton. `[RequireMatchingQueriesForUpdate]` is more implicit — it keys off whatever queries the system ends up having.

---

6. Custom groups

Reasons to create one:

- Cohesive gameplay phase (e.g. "apply damage before animation").
- A different tick rate (e.g. a custom fixed step at 30 Hz).
- A feature toggle — enable/disable the whole group.

```
using Unity.Entities;

[UpdateInGroup(typeof(SimulationSystemGroup))]
```

```
public partial class DamagePipelineGroup : ComponentSystemGroup
{
    // You can override OnUpdate to customise how the group ticks.
}

[UpdateInGroup(typeof(DamagePipelineGroup), OrderFirst = true)]
public partial struct ApplyDamageSystem : ISystem { /* ... */ }

[UpdateInGroup(typeof(DamagePipelineGroup))]
[UpdateAfter(typeof(ApplyDamageSystem))]
public partial struct DeathSystem : ISystem { /* ... */ }
```

Custom tick rate

```
public partial class SlowTickGroup : ComponentSystemGroup
{
    private float _accum;
    protected override void OnUpdate()
    {
        _accum += (float)SystemAPI.Time.DeltaTime;
        if (_accum < 0.1f) return;      // 10 Hz
        _accum = 0f;
        base.OnUpdate();                // tick children
    }
}
```

---

7. Fixed vs variable timestep

If determinism matters (physics, networked prediction, deterministic playback), put the system in `FixedStepSimulationSystemGroup`:

```
[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
public partial struct PhysicsStepSystem : ISystem { /* ... */ }
```

Gotchas:

- `FixedStepSimulationSystemGroup` may run **multiple times** per frame (or zero times) to catch up to real time.
- Use `SystemAPI.Time.DeltaTime` — it is the fixed step's dt inside this group.
- Anything that reads input or produces presentation should stay in `SimulationSystemGroup` / `PresentationSystemGroup`.

---

8. Inspecting the order at runtime

Window → Entities → Systems shows:

- Every group and its children, in actual run order.
- Per-system timing.
- Enabled / disabled state (systems you toggle at runtime).

If your attribute-declared order doesn't match what you see, there's almost always a circular dependency or a forgotten `[UpdateInGroup]`.

---

9. Troubleshooting

Symptom	Cause / Fix
System runs in the wrong frame phase	Missing or wrong `[UpdateInGroup]`. Default is `SimulationSystemGroup`.
`[UpdateAfter]` ignored	Target system is in a different group. Both must share a group.
Circular dependency error on startup	A chain of `[UpdateBefore]` / `[UpdateAfter]` forms a cycle. The Systems window prints the cycle.
System runs several times per frame unexpectedly	You placed it in `FixedStepSimulationSystemGroup` — that group catches up. Move to `SimulationSystemGroup` if it shouldn't.
Custom group runs forever	`OnUpdate` override never called `base.OnUpdate()`.
System runs but does nothing	`RequireForUpdate<T>` isn't satisfied, or every query is empty. Check **Window → Entities → Query**.

Chapter 13. JobSystem & Burst

1. Overview

DOTS performance rests on two engine features that existed before Entities and still underpin it:

- **Job System** — schedule work onto worker threads with dependency graphs.
- **Burst Compiler** — compile C# jobs to tight native code (SIMD, no GC, no virtual dispatch).

Neither is an Entities-specific concept. `IJobEntity` and `IJobChunk` are thin layers on top of `IJob` + Burst. Understanding the base layer makes every higher-level pattern read clearly.

---

2. Job System — the core types

Interface	Shape	Use when...
`IJob`	`void Execute()`	One-shot single-threaded work.
`IJobParallelFor`	`void Execute(int index)`	Loop from 0 to N in parallel.
`IJobParallelForBatch`	`void Execute(int start, int count)`	Loop in parallel batches (cache-friendlier than per-index).
`IJobChunk`	`void Execute(ArchetypeChunk chunk, int chunkIndex, bool useEnabledMask, in v128 enabledMask)`	Iterate ECS chunks in parallel.
`IJobEntity`	`void Execute([component params])`	ECS entity iteration — source-generated into a chunk job.

All of these are **structs** and all support Burst.

Minimal `IJobParallelFor` example

```
using Unity.Burst;
using Unity.Collections;
using Unity.Jobs;

[BurstCompile]
struct ScaleJob : IJobParallelFor
{
    public NativeArray<float> Data;
    public float Factor;
```

```
    public void Execute(int index) => Data[index] *= Factor;
}

// Scheduling:
var handle = new ScaleJob { Data = arr, Factor = 2f }.Schedule(arr.Length, 64);
handle.Complete();
```

Notes:

- Fields must be blittable.
- Write-access to the same `NativeArray` from multiple jobs requires dependency chaining (below).

---

3. Scheduling and dependencies

```
JobHandle a = jobA.Schedule();
JobHandle b = jobB.Schedule(a);              // b runs after a
JobHandle c = jobC.Schedule(JobHandle.CombineDependencies(a, b));
c.Complete();
```

Rules:

- Every `Schedule()` returns a `JobHandle`. Pass the previous handle in as a dependency if the new job reads or writes shared data.
- **Never skip `.Complete()`** on the final handle before the main thread reads the results (the Safety System enforces this).
- In an `ISystem`, `state.Dependency` is the system-level handle; scheduled jobs automatically combine with it if you use `.Schedule(state.Dependency)` or rely on source-generated helpers like `IJobEntity.ScheduleParallel(...)`.

Inside an `ISystem`

```
[BurstCompile]
public void OnUpdate(ref SystemState state)
{
    var job = new MyJob { Dt = SystemAPI.Time.DeltaTime };
    state.Dependency = job.ScheduleParallel(state.Dependency);
}
```

`IJobEntity.ScheduleParallel()` without an explicit dependency will combine with `state.Dependency` implicitly. Be explicit when chaining multiple jobs.

---

4. Burst — what and why

Burst compiles a subset of C# (`[BurstCompile]`-marked methods, no GC allocations, blittable data) to highly optimised native code via LLVM.

What Burst compiles well

- Struct `IJob*` / `ISystem` methods tagged `[BurstCompile]`.
- Math on `Unity.Mathematics` types (`float3`, `quaternion`, `int4`, etc.).
- Tight loops over `NativeArray<T>`, `NativeList<T>`, `DynamicBuffer<T>`, `ArchetypeChunk` column arrays.

What Burst cannot compile

- Managed references (`class`, `string`, `List<T>`, anything not unmanaged).
- Exceptions (you can catch them, but generally avoid).
- Dynamic dispatch through interfaces (explicit generic instantiation is fine).
- `System.IO` / `Reflection` / most of BCL beyond math/primitives.

Enabling Burst

```
using Unity.Burst;

[BurstCompile]
public partial struct MyJob : IJobEntity
{
    // attribute on the struct marks the type for Burst
    // methods need [BurstCompile] too if you want each compiled
    [BurstCompile]
    void Execute(ref Velocity v) { /* ... */ }
}
```

For systems:

```
[BurstCompile]
public partial struct MySystem : ISystem
{
    [BurstCompile] public void OnCreate(ref SystemState state) { }
    [BurstCompile] public void OnUpdate(ref SystemState state) { }
}
```

> The attribute on the type is **not** enough — each method you want compiled also needs `[BurstCompile]`.

Burst Inspector

Window → Analysis → Burst Inspector shows the LLVM IR, assembly, and vector usage per method. Use it when "why isn't this fast" is the question.

---

5. Native containers

Burst and the Safety System expect **native containers** instead of managed collections:

Managed	Native equivalent
`T[]`	`NativeArray<T>`
`List<T>`	`NativeList<T>`
`Dictionary<K,V>`	`NativeHashMap<K,V>`, `NativeParallelHashMap<K,V>`
`HashSet<T>`	`NativeHashSet<T>`, `NativeParallelHashSet<T>`
`Queue<T>`	`NativeQueue<T>`
`string`	`FixedString32Bytes` / `FixedString64Bytes` / etc.

Allocation:

```
var arr = new NativeArray<int>(1024, Allocator.TempJob);
// ...
arr.Dispose();
```

Lifetime matters: Burst + Safety verifies that containers are disposed and not accessed after a job's lifetime.

---

6. Allocators

Allocator	Lifetime	Typical use
`Allocator.Temp`	Within the current frame, main thread only	Scratch containers in `OnUpdate`
`Allocator.TempJob`	Across one job, ≤4 frames	Containers passed to jobs
`Allocator.Persistent`	Manual `Dispose()` required	System-owned containers that live across frames
`state.WorldUpdateAllocator`	Cleared at the end of the system group update	Preferred for per-system transient data

Prefer `state.WorldUpdateAllocator` (or its Rewindable allocator) inside systems — it avoids manual `Dispose()` bookkeeping.

---

7. Safety System

In the Editor with "Jobs > Leak Detection" and "Jobs > Safety Checks" on, Unity validates that:

- Two jobs don't write the same native container without a dependency edge.
- A main-thread read doesn't race with a running write.
- Native containers aren't used after `Dispose()` or past their allocator's lifetime.

Safety errors surface as exceptions with a clear "job A and job B both access X" message. **Do not disable safety checks in development** — performance wins are small and debugging cost is high.

---

8. Troubleshooting

Symptom	Cause / Fix
Burst warning: "method not burst compiled"	Missing `[BurstCompile]` on the method; or Burst found an unsupported type (check the Burst Inspector for the failing line).
`InvalidOperationException: … writes to … without declaring dependency`	Missing dependency edge — pass the previous `JobHandle` or use `state.Dependency` chaining.
`ObjectDisposedException` on a NativeArray	Disposed before the job finished. Use `arr.Dispose(handle)` or wait on the handle first.
Scheduled job never completes	Missing `.Complete()` on a chain, or a circular dependency.
Performance regression after adding `[BurstCompile]`	Method is being recompiled each domain reload; check Burst cache. Or a Burst-incompatible type slipped in — the method silently fell back to IL.
`NativeHashMap` enumeration throws	Enumeration doesn't play well with concurrent writes — enumerate a `NativeArray<K>` copy instead.
Main-thread stall waiting for a job	Called `.Complete()` too early. Move the completion point as late as possible in the frame.

Chapter 14. IJobEntity · SystemAPI.Query

1. Overview

Two APIs cover 95% of component iteration in Entities 6.5:

API	Where it runs	Use when...
`SystemAPI.Query<...>()`	Main thread, inside `OnUpdate`	The work is trivial (a few hundred entities), or needs main-thread APIs.
`IJobEntity`	Worker threads, via `Schedule` / `ScheduleParallel`	The work is heavier or benefits from parallelism.

They are the recommended replacement for the legacy `Entities.ForEach` (marked obsolete in Entities 1.4; still compiles on 6.5 with deprecation warnings; Unity's changelog says removal is planned for a future major release).

---

2. `SystemAPI.Query<T>`

Iterate matching entities on the main thread. Components are accessed via `RefRO<T>` / `RefRW<T>`.

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
                 SystemAPI.Query<RefRW<Health>, RefRO<Regen>>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRW.Max);
        }
    }
}
```

Rules:

- The system struct must be `partial`.

- `RefRO<T>` for read-only, `RefRW<T>` for read-write. Don't write through `RefRO`.
- Add an `Entity` to the tuple if you need the id: `SystemAPI.Query<RefRW<Health>>().WithEntityAccess()`.

Filters

```
foreach (var health in SystemAPI.Query<RefRW<Health>>()
             .WithAll<Enemy>()
             .WithNone<Dead>()
             .WithAny<Burning, Poisoned>())
{
    // only entities that are Enemy AND not Dead AND (Burning OR Poisoned)
}
```

- `WithAll<T>()` — must have all of the listed components.
- `WithNone<T>()` — must not have any of them.
- `WithAny<T...>()` — must have at least one of them.
- `WithChangeFilter<T>()` — only chunks where component `T` changed since last run.
- `WithEntityAccess()` — also return the `Entity` id.

---

3. `IJobEntity`

A source-generated parallel iteration. The `Execute` signature declares what components you touch.

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;

[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
```

```
    }
}
```

Signature rules:

- `void Execute(...)` — parameters map 1:1 to components.
- `ref Component` — read-write. `in Component` — read-only (no write allowed).
- `Entity entity` as a parameter is allowed — the source generator fills it in.
- Job `struct` must be `partial`.

Schedule variants

Call	Threading
`.Run()`	Main thread, completes before returning. Useful for debugging or small workloads.
`.Schedule()`	Single worker thread.
`.ScheduleParallel()`	Distributes chunks across worker threads.

All three update `state.Dependency` automatically inside an `ISystem.OnUpdate`.

Filters in jobs

```
new HealthRegenJob { DeltaTime = dt }
    .ScheduleParallel(SystemAPI.QueryBuilder()
        .WithAll<Enemy, Regen>()
        .WithNone<Dead>()
        .Build());
```

Or via `[WithAll]`/`[WithNone]`/`[WithAny]` attributes on the job struct itself.

Entity access in a job

```
void Execute(Entity entity, ref Health health) { /* ... */ }
```

Chunk index / `[ChunkIndexInQuery]`

Useful when pairing with a `ParallelWriter` ECB for deterministic playback — pass the chunk index as the `sortKey`. See `15_ParallelWriter · Deterministic Playback.md`.

```
void Execute([ChunkIndexInQuery] int chunkIndex, in Health health, Entity entity)
{
    if (health.Value <= 0)
```

```
        ecb.DestroyEntity(chunkIndex, entity);
}
```

---

4. When to use which

Situation	Pick
Tens to hundreds of entities, trivial math	`SystemAPI.Query`
Thousands+ of entities, independent per-entity work	`IJobEntity.ScheduleParallel`
Per-chunk work (bounds, sums, spatial partitioning)	**`IJobChunk`** — see `12_IJobEntity vs IJobChunk.md`
Need managed API inside the loop	`SystemAPI.Query` from a `SystemBase`
Need deferred structural changes	Either form, but write via `EntityCommandBuffer.ParallelWriter` — see `14_EntityCommandBuffer · Deferred Entity.md`

---

5. Gotchas

- **Don't capture `this` or managed fields in a job.** The source generator will error if you try.
- **`RefRW<T>.ValueRW` triggers change tracking.** Reading through `ValueRW` and never writing still dirties the chunk.
- **`WithChangeFilter<T>` is chunk-level.** If any entity in the chunk writes to `T`, the whole chunk is considered changed.
- **Aspects are obsolete (but not removed).** The `IAspect` grouping interface was deprecated in 1.4 and is still listed as a valid `Execute` parameter kind on the 6.5 API reference. New code should use plain component parameters; existing aspects still compile on 6.5.

---

6. Troubleshooting

Symptom	Cause / Fix
Compile error: **`IJobEntity must be partial`**	Add `partial` to the job struct.
`ScheduleParallel` throws a race-condition error	Two jobs write to the same component without an ordering edge. Ensure `state.Dependency` is combined correctly, or add `[UpdateAfter]` between systems.

Symptom	Cause / Fix
Job doesn't run on Burst	Missing `[BurstCompile]` on the job struct, or a Burst-incompatible type (managed reference, `FixedString`-less `string`) is captured.
`SystemAPI.Query` returns nothing	Filter combination is empty. Inspect the query in **Window** → **Entities** → **Query** to see what it actually matches.
Writing through `RefRO` (silent bug)	Compiler now errors on `refRO.ValueRW = ...`. If it slips through older source-gen versions, switch to `RefRW`.
Entity-count mismatch between `SystemAPI.Query` and an `EntityQuery` you built manually	Different filter set. `SystemAPI.Query` includes enableable-component filtering by default; `EntityQueryBuilder` requires explicit configuration.

Chapter 15. IJobEntity vs IJobChunk

1. The short answer

- **`IJobEntity`** for "do the same thing to each entity."
- **`IJobChunk`** for "do something to each chunk that isn't a simple per-entity map."

`IJobEntity` is generated on top of `IJobChunk` — they run on the same iteration machinery. `IJobChunk` just exposes the full chunk to your code.

---

2. `IJobEntity` — per-entity ergonomics

```
[BurstCompile]
public partial struct DamageTickJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Burn burn)
    {
        health.Value -= burn.DamagePerSecond * DeltaTime;
    }
}
```

Strengths:

- One method, one entity view. Easy to write, easy to read.
- Source-generated plumbing: `Entity`, `[ChunkIndexInQuery]`, `[EntityIndexInQuery]` parameters are filled in for you.
- Filters via `[WithAll]` / `[WithNone]` / `[WithAny]` attributes on the struct, or via a `QueryBuilder` passed to `.Schedule*`.

Weaknesses:

- No cross-entity visibility inside `Execute`. Can't easily "sum over this chunk" without extra state.
- No access to chunk-level metadata (change version, shared component value) from inside `Execute`.

---

3. `IJobChunk` — full-chunk control

```
[BurstCompile]
public partial struct HealthAggregateJob : IJobChunk
{
    [ReadOnly] public ComponentTypeHandle<Health> HealthHandle;
    public NativeArray<float> PerChunkTotal;  // size = chunk count

    public void Execute(in ArchetypeChunk chunk,
                        int unfilteredChunkIndex,
                        bool useEnabledMask,
                        in v128 chunkEnabledMask)
    {
        var healths = chunk.GetNativeArray(ref HealthHandle);
        float total = 0f;
        for (int i = 0; i < chunk.Count; i++)
            total += healths[i].Value;

        PerChunkTotal[unfilteredChunkIndex] = total;
    }
}
```

Strengths:

- Direct access to chunk arrays: `chunk.GetNativeArray(ref handle)`, `chunk.GetBufferAccessor(ref bufferHandle)`.
- `chunk.Has<T>()`, `chunk.GetSharedComponent<T>(...)` for conditional work.
- Sees the enabled-mask and change version per chunk.
- Perfect for reductions, spatial partitioning, batched ECB recording.

Cost:

- More boilerplate: declare `ComponentTypeHandle<T>` in the system, update them each frame with `handle.Update(ref state)`.
- You manage the inner `for` loop yourself — so more places to get wrong.

Setting up the handles in a system

```
[BurstCompile]
public partial struct HealthAggregateSystem : ISystem
{
    private EntityQuery _query;
    private ComponentTypeHandle<Health> _healthHandle;

    public void OnCreate(ref SystemState state)
    {
        _query        = state.GetEntityQuery(ComponentType.ReadOnly<Health>());
        _healthHandle = state.GetComponentTypeHandle<Health>(true);
    }
```

```
    public void OnUpdate(ref SystemState state)
    {
        _healthHandle.Update(ref state);

        var totals = CollectionHelper.CreateNativeArray<float>(
            _query.CalculateChunkCount(), state.WorldUpdateAllocator);

        state.Dependency = new HealthAggregateJob
        {
            HealthHandle   = _healthHandle,
            PerChunkTotal  = totals
        }.ScheduleParallel(_query, state.Dependency);
    }
}
```

`handle.Update(ref state)` refreshes the cached type handle each frame — forget this and Burst will hit stale indices.

---

4. Enabled mask (`useEnabledMask`)

For queries that include an `IEnableableComponent`, `Execute` receives `useEnabledMask = true` and a `v128` mask of enabled bits. You must respect the mask or you'll process disabled entities.

Helper:

```
var enumerator = new ChunkEntityEnumerator(useEnabledMask, chunkEnabledMask, chunk.Count);
while (enumerator.NextEntityIndex(out int i))
{
    // i is the index of the next enabled entity in the chunk
}
```

`IJobEntity` applies the mask for you automatically — another reason to prefer it unless you need chunk-level visibility.

---

5. Decision guide

Situation	Pick
Tick health, translate position, apply input — pure per-entity math	`IJobEntity`
Sum values across all entities in a chunk	`IJobChunk`
Record one ECB command per chunk instead of per entity	`IJobChunk`
Access a shared component value inside the loop	`IJobChunk`

Situation	Pick
Need the chunk's change version	`IJobChunk`
Iterate a `DynamicBuffer<T>` on each entity	Either works; `IJobEntity` is cleaner
Per-entity logic but need the entity's chunk index	`IJobEntity` with `[ChunkIndexInQuery] int chunkIndex`
Scheduling cost matters and the work is tiny	Benchmark — fewer chunks beats fewer entities. `IJobChunk` usually wins at thousands of chunks.

---

6. Interop — using both together

A common pattern: gather per-chunk data with `IJobChunk`, then do per-entity writes with `IJobEntity`, chained through dependencies.

```
var gather = new HealthAggregateJob { ... }
    .ScheduleParallel(query, state.Dependency);

var act = new HealJob { PerChunkTotal = totals }
    .ScheduleParallel(gather);

state.Dependency = act;
```

---

7. Troubleshooting

Symptom	Cause / Fix
`IJobChunk.Execute` reads garbage values	Forgot `handle.Update(ref state)` before scheduling, or the wrong query was used.
`ComponentTypeHandle<T>` not found	Missing `using Unity.Entities;`, or the handle was declared with the wrong access mode. Use `GetComponentTypeHandle<T>(true)` for read-only.
Enabled entities skipped	Mask not respected — use `ChunkEntityEnumerator` as shown. `IJobEntity` avoids this.
Job writes to the same `NativeArray` from two chunks	Provide a per-chunk slot (as in `HealthAggregateJob`) or use `NativeParallelHashMap` with chunk index as key.
Scheduling `IJobChunk` without query argument	`ScheduleParallel(query, ...)` requires the query explicitly. `IJobEntity.ScheduleParallel()` picks it up from attributes/source generation.
`v128` type not found	`using Unity.Burst.Intrinsics;` is missing.

Chapter 16. Structural Change & Safety

1. What counts as a structural change

A **structural change** is any operation that can move entities between chunks or alter the chunk inventory. All of the following are structural:

Operation	Why it's structural
`CreateEntity` / `DestroyEntity`	Allocates or frees a chunk slot.
`AddComponent<T>` / `RemoveComponent<T>`	Entity's archetype changes, so it physically moves to a different chunk.
`SetSharedComponent<T>`	Different shared-component value → different chunk (groups by value).
`AddChunkComponent<T>` / `RemoveChunkComponent<T>`	Changes the chunk's own metadata.
`SetName(entity, ...)`	Touches entity metadata in the EntityManager.
`MoveEntitiesFrom`	Copies entities between worlds.
`Instantiate(prefab)`	Allocates a new entity (may pre-fill an archetype).

Writing a component's **value** via `SetComponentData` / `RefRW<T>.ValueRW` is **not** structural — the component already lives in the chunk; you're just changing bytes.

---

2. Why structural changes are dangerous

When the archetype changes, anything holding a position-based reference into a chunk goes stale:

- In-flight query iterators (`SystemAPI.Query<...>`) may skip or revisit entities.
- `ComponentLookup<T>` entries become invalid for affected entities.
- `ArchetypeChunk` handles in a job point to the **old** chunk.
- Other systems running concurrently observe inconsistent state.

The Safety System catches most of these with an `InvalidOperationException` in the Editor. In a build, they become crashes or corrupt entities.

---

3. Rule of thumb

> **Do not perform structural changes while iterating a query in the same system update.**

Two compliant patterns, in order of preference:

1. **Use an `EntityCommandBuffer`.** Record the operation during iteration, play it back after. See `14_EntityCommandBuffer · Deferred Entity.md`.
2. **Iterate into a `NativeList<Entity>`, then mutate after the loop.** Simpler for small cases; no ECB overhead.

Example of pattern 2:

```
var toKill = new NativeList<Entity>(state.WorldUpdateAllocator);

foreach (var (health, e) in SystemAPI
             .Query<RefRO<Health>>()
             .WithEntityAccess())
{
    if (health.ValueRO.Value <= 0f)
        toKill.Add(e);
}

state.EntityManager.DestroyEntity(toKill.AsArray());
```

---

4. Enableable components — the escape hatch

If you find yourself adding and removing a component every frame, it should almost certainly be an **enableable component** instead — toggling the enable bit is **not** structural. See `06_Enableable Component.md`.

---

5. Jobs and structural changes

Structural changes are **main-thread operations**. You cannot perform them inside a job.

The idiomatic ways to mutate from a job:

Approach	When
`EntityCommandBuffer.ParallelWriter` recorded in the job, played back on the main thread	Per-entity structural changes from parallel code.

Approach	When
`EntityCommandBuffer` (single-thread) recorded in an `IJob` or `IJobChunk`, played back on main thread	Batched / chunk-scoped changes.
Toggle an enableable component via `EnabledRefRW<T>` inside a job	Per-entity "activate/deactivate" without structure.

An ECB captures the operation as a command; playback happens at a known point on the main thread, usually at an `EntityCommandBufferSystem` boundary.

---

6. Combining structural changes and reads

If two systems run back-to-back and system A performs a structural change that system B depends on, make the ordering explicit:

```
[UpdateInGroup(typeof(SimulationSystemGroup))]
[UpdateBefore(typeof(MovementSystem))]
public partial struct SpawnSystem : ISystem { /* uses ECB */ }
```

When the ECB system playback sits between A and B (e.g. `EndSimulationECBS` with a suitable `[UpdateAfter]` chain), the structural change lands before B reads.

---

7. Cost model

Rough ranking for a single change:

```
SetComponentData (value only)          ~ 1x
SetComponentEnabled (enableable bit)   ~ 1x (per-chunk bitmask write)
AddComponent (structural)              ~ 10x   (entity moves chunk)
SetSharedComponent (structural)        ~ 10x
DestroyEntity (structural)             ~ 10x
Per-frame add + remove pattern         pathological
```

Orders of magnitude; exact numbers depend on archetype size, chunk occupancy, and allocator pressure.

---

8. Diagnosing structural churn

Symptoms:

- `Window → Entities → Systems` shows a system with disproportionate timing for its logic.
- Profiler marker `EntityManager.SetArchetype` or `Move Entity` dominates frames.
- Archetype count grows over time.

Tactics:

- Audit `AddComponent` / `RemoveComponent` calls — are any per-frame? Replace with enableable components.
- Inspect shared-component value sets — any accidentally per-entity keys?
- Check the Archetypes window for one-entity chunks; that's fragmentation from over-granular component sets.

---

9. Troubleshooting

Symptom	Cause / Fix
`InvalidOperationException: EntityCommandBuffer has already been played back`	Recording into an ECB after its playback. Each ECB is single-shot — get a fresh one from the ECBS each frame.
`InvalidOperationException: … invalidated … during foreach`	Structural change inside the loop. Move to an ECB or buffer-then-apply.
Entities silently disappear	`DestroyEntity` happened but other systems still hold the old `Entity` handle. Check handles with `EntityManager.Exists`.
Add/Remove pattern churns frame time	Replace with enableable components.
`RemoveComponent` triggers "cleanup component present" error	The entity has a cleanup component. Handle cleanup first (see `05_Component Types.md`).
Shared component value set explodes (hundreds of distinct values)	The value is too granular. Bucket it (e.g. rounded to the nearest N) or store the variable piece as a regular component.

Chapter 17. EntityCommandBuffer · Deferred Entity

1. Why ECB exists

`EntityCommandBuffer` (ECB) records structural changes — create, destroy, add, remove, set — and replays them later on the main thread at a safe point. It lets you mutate the world from:

- Inside a query `foreach` (where direct `EntityManager` calls would invalidate the iterator).
- A job (where main-thread structural changes aren't allowed).
- A parallel job across many chunks (via `ParallelWriter`).

See `13_Structural Change & Safety.md` for the "why this is dangerous" background.

---

2. The ECB lifecycle

Figure: EntityCommandBuffer Lifecycle.

An ECB is **single-use**: after playback it's disposed. You ask the right `EntityCommandBufferSystem` for a new one each frame.

---

3. Built-in ECB systems

System	When it plays back	Typical use
`BeginInitializationEntityCommandBufferSystem`	Start of Initialization group	Early world setup from the previous frame
`EndInitializationEntityCommandBufferSystem`	End of Initialization group	—
`BeginSimulationEntityCommandBufferSystem`	Start of Simulation	Apply pending changes before gameplay runs

System	When it plays back	Typical use
`EndSimulationEntityCommandBufferSystem`	End of Simulation	**Default pick** for spawns/destroys driven by gameplay
`BeginPresentationEntityCommandBufferSystem`	Start of Presentation	Rare; visual-only setup

Picking the right one determines **when other systems observe your changes**. If you spawn a bullet in `SimulationSystemGroup` and want it to move **this same frame**, record into `BeginSimulation` next frame — or into `EndSimulation` if next frame is fine.

---

4. Getting an ECB

Inside any `ISystem.OnUpdate`:

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);
```

The `.Singleton` nested type is a per-system token the source generator exposes for each built-in ECB system. `CreateCommandBuffer` gives you a fresh ECB that will be played back when that system ticks.

For parallel jobs:

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

See `15_ParallelWriter · Deterministic Playback.md` for the parallel playback contract.

---

5. Recording — entity creation

```
Entity e = ecb.CreateEntity();
ecb.AddComponent(e, new Health { Value = 100 });
ecb.AddComponent(e, LocalTransform.FromPosition(0, 0, 0));
```

Or instantiate from a prefab entity:

```
Entity inst = ecb.Instantiate(prefab);
ecb.SetComponent(inst, LocalTransform.FromPosition(spawnPos));
```

Both `CreateEntity` and `Instantiate` return a **deferred entity** (see §6).

---

6. Deferred entities

An entity returned by `ecb.CreateEntity()` or `ecb.Instantiate()` **does not exist yet**. It has a placeholder negative index. You can:

- Pass it to subsequent ECB calls on the **same ECB** — `AddComponent`, `SetComponent`, `AppendToBuffer`.
- Store it in another component via `ecb.SetComponent(other, new ReferenceTo { Entity = inst })`.

You **cannot**:

- Read or write to it via `EntityManager` until the ECB plays back.
- Pass it between different ECBs (the placeholder only resolves inside the owning ECB).

At playback time the ECB system substitutes the real `Entity` handle everywhere the placeholder was used.

---

7. Recording — structural changes

```
ecb.AddComponent(entity, new Stunned { Duration = 2f });
ecb.RemoveComponent<Stunned>(entity);
ecb.SetComponent(entity, new Health { Value = 50 });

ecb.SetComponentEnabled<Stunned>(entity, false);

var buf = ecb.AddBuffer<Waypoint>(entity);
buf.Add(new Waypoint { Position = new float3(1, 0, 0) });
ecb.AppendToBuffer(entity, new Waypoint { Position = new float3(2, 0, 0) });

ecb.DestroyEntity(entity);
```

All of these are queued commands; none take effect until playback.

---

8. ECB from an `IJobEntity`

```
[BurstCompile]
public partial struct KillOnHealthZeroJob : IJobEntity
```

```
{
    public EntityCommandBuffer ECB;

    void Execute(Entity entity, in Health health)
    {
        if (health.Value <= 0)
            ECB.DestroyEntity(entity);
    }
}

// In OnUpdate:
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged);

new KillOnHealthZeroJob { ECB = ecb }.Schedule();
```

For `.ScheduleParallel()`, swap to `.AsParallelWriter()` and thread a `sortKey` through — covered in the next page.

---

9. Standalone (manual) ECBs

Sometimes you need playback you control yourself — e.g. inside a single `OnUpdate`:

```
var ecb = new EntityCommandBuffer(Allocator.Temp);
// ... record ...
ecb.Playback(state.EntityManager);
ecb.Dispose();
```

Use this for self-contained, one-off mutation patterns. For anything that crosses system boundaries or runs in a job, use the built-in ECBS above.

---

10. Determinism

ECB commands play back in **record order** on single-writer ECBs. On a `ParallelWriter`, playback order depends on the `sortKey` you provide — see `15_ParallelWriter · Deterministic Playback.md`.

For Netcode / replay systems, deterministic playback is non-negotiable. Always pass a stable sort key (chunk index, entity-in-query index) and avoid recording in main-thread code paths that run in nondeterministic order.

---

11. Troubleshooting

Symptom	Cause / Fix
`InvalidOperationException: ECB has already been played back`	Reused across frames. Get a fresh ECB per OnUpdate via `CreateCommandBuffer`.
Deferred entity handle is `Entity.Null` after playback	You referenced it across two different ECBs. Keep both the create and the subsequent set on the same ECB.
Spawned entity doesn't appear this frame	Recorded into `EndSimulation` ECBS — playback is at end of frame. Switch to `BeginSimulation` next frame to observe it before most systems.
`ecb.SetComponent` throws "component does not exist"	Target entity doesn't have the component yet (`AddComponent` first) or playback hasn't happened.
`Dispose()` called on an ECB managed by ECBS	Don't call `Dispose()` on ECBs from `CreateCommandBuffer` — the ECBS handles lifetime. Only dispose standalone (`new EntityCommandBuffer(Allocator.Temp)`) ones.
Parallel ECB playback reorders commands	`sortKey` isn't deterministic. Use `[ChunkIndexInQuery]` or a stable query-provided index.

Chapter 18. ParallelWriter · Deterministic Playback

1. Why there is a separate parallel writer

`EntityCommandBuffer` is single-writer. Two threads recording into the same ECB would race on its internal chain. `EntityCommandBuffer.ParallelWriter` solves this by giving each caller a **thread-local chain** and requiring a **sort key** per command.

At playback time the ECB system:

1. Flattens all thread-local chains.
2. Sorts commands by `sortKey` (stable sort).
3. Executes them on the main thread.

The result is deterministic as long as your sort keys are deterministic.

---

2. Getting a ParallelWriter

```
var ecb = SystemAPI
    .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
    .CreateCommandBuffer(state.WorldUnmanaged)
    .AsParallelWriter();
```

`AsParallelWriter()` returns a `struct EntityCommandBuffer.ParallelWriter` you can copy into an `IJobEntity`, `IJobChunk`, or `IJobParallelFor`.

---

3. The sortKey contract

Every parallel ECB call takes a `sortKey` as its first argument:

```
ecb.Instantiate(sortKey, prefab);
ecb.SetComponent(sortKey, entity, value);
ecb.DestroyEntity(sortKey, entity);
ecb.AddComponent(sortKey, entity, component);
```

Rules for a **good** sort key:

- **Deterministic.** Same input → same key, no matter how the scheduler distributes work.

- **Unique enough.** Two commands on the same entity should not collide unless you want playback order to be ambiguous.
- **Cheap.** An `int` you already have, not a hash.

In practice, the go-to choices are:

Source	How to get it	When to use
`[ChunkIndexInQuery]` on `IJobEntity`	Source generator fills it in	99% of cases
`unfilteredChunkIndex` in `IJobChunk.Execute`	Provided by the API	`IJobChunk` jobs
`[EntityIndexInQuery]` on `IJobEntity`	Source generator	When commands per entity must be ordered individually
A manually-computed deterministic index	You compute it	Rare; custom job shapes

---

4. Full example — parallel spawn

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

[BurstCompile]
public partial struct SpawnParallelJob : IJobEntity
{
    public float                                DeltaTime;
    public EntityCommandBuffer.ParallelWriter  ECB;

    void Execute([ChunkIndexInQuery] int chunkIndex,
                 Entity spawnerEntity,
                 ref Spawner spawner)
    {
        if (spawner.RemainingCount <= 0)
        {
            ECB.DestroyEntity(chunkIndex, spawnerEntity);
            return;
        }

        spawner.TimeLeft -= DeltaTime;
        if (spawner.TimeLeft > 0f) return;

        spawner.TimeLeft = spawner.Interval;
        spawner.RemainingCount--;

        var inst = ECB.Instantiate(chunkIndex, spawner.Prefab);
        ECB.SetComponent(chunkIndex, inst, LocalTransform.FromPosition(spawner.SpawnPoint));
    }
}
```

```
[BurstCompile]
public partial struct SpawnerParallelSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var ecb = SystemAPI
            .GetSingleton<EndSimulationEntityCommandBufferSystem.Singleton>()
            .CreateCommandBuffer(state.WorldUnmanaged)
            .AsParallelWriter();

        new SpawnParallelJob
        {
            DeltaTime = SystemAPI.Time.DeltaTime,
            ECB       = ecb
        }.ScheduleParallel();
    }
}
```

Passing `chunkIndex` to both `Instantiate` and the subsequent `SetComponent` makes sure the two commands sort to the same position and therefore the deferred-entity reference from `Instantiate` resolves to the right target.

---

5. Determinism — what it guarantees, what it doesn't

Guaranteed: Given the same ECB contents, playback produces identical resulting state — regardless of which worker thread recorded which command.

Not guaranteed automatically:

- **Entity IDs.** Two runs might produce entities with different `Index` values if prior commands differed.
- **Chunk ordering across systems.** If system A produces entities in parallel and system B reads them in order of `ChunkIndexInQuery`, the iteration order can still depend on archetype creation history.

If you need bit-for-bit determinism across runs (replays, Netcode rollback), combine:

1. Parallel ECB with deterministic sort keys.
2. Fixed-step systems inside `FixedStepSimulationSystemGroup`.
3. No reliance on `Entity.Index`; key on gameplay IDs you control.

---

6. Sort-key patterns

6.1 Per-chunk (the default)

Use `[ChunkIndexInQuery] int chunkIndex`. Collisions are fine if every entity in the chunk generates an identical command (e.g. a whole chunk being marked stunned).

6.2 Per-entity

Use `[EntityIndexInQuery] int entityIndex` when you need strict per-entity ordering (e.g. recording per-entity damage events that must play back in entity order).

6.3 Custom deterministic index

Very rarely, you build a flat index from inputs yourself — e.g. `int sortKey = baseOffset + localIndex;`. Only do this when the common two don't fit; custom indices are a frequent source of determinism bugs.

---

7. Avoid these patterns

- **Using `JobsUtility.ThreadIndex`** as a sortKey. Not deterministic.
- **Using `Random` to build the key.** Not deterministic.
- **Mixing ECBs.** Record into two different ECBs and you lose the total ordering.
- **Recording to a per-system ECB from multiple systems.** One ECB, one recorder (or one parallel-writer wrapping one ECB).

---

8. Troubleshooting

Symptom	Cause / Fix
Deferred entity resolves to the wrong target	`Instantiate` and the follow-up `SetComponent` used different sort keys. Keep them the same.
Commands fire in different order each run	Non-deterministic sort key. Switch to `[ChunkIndexInQuery]`.
"JobEntity must have partial" compile error	Add `partial` to the job struct.

Symptom	Cause / Fix
`ECB.SetComponent` in a parallel job throws "wrong thread"	Using the non-parallel `EntityCommandBuffer` inside a parallel job. Call `.AsParallelWriter()` first and use the parallel API.
Netcode replay desyncs	A parallel ECB somewhere doesn't use a deterministic sort key, or an `IJobEntity` captured a non-deterministic value. Audit the chain.
`AsParallelWriter()` on a disposed ECB	ECB has already been played back — get a fresh one this frame.

Chapter 19. Netcode Client-Server World & Bootstrap

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Netcode for Entities uses a **client-server model** and separates client and server logic into different ECS worlds.

World	Role
Server World	Authoritative simulation.
Client World	Local player simulation, prediction, interpolation, and presentation.
Thin Client World	Dummy client used for development and load testing.

When a client device also hosts the server, the project is running a **client-hosted server** configuration.

---

2. World types and system filtering

2.1 `WorldSystemFilter`

Use `WorldSystemFilter` to declare which Netcode world a system can run in.

```
using Unity.Entities;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerOnlySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        // Runs only in the server world.
    }
}
```

Flag	Meaning
`LocalSimulation`	Local world without Netcode systems.
`ServerSimulation`	Server simulation world.

Flag	Meaning
`ClientSimulation`	Client simulation world.
`ThinClientSimulation`	Thin client simulation world.

2.2 System-group filtering

Some Netcode groups only exist in specific worlds, so placing a system in the group also filters the world.

```
using Unity.Entities;
using Unity.NetCode;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct MyInputSystem : ISystem
{
    // GhostInputSystemGroup exists in Client and Local worlds.
    // The system is excluded from Server worlds.
}
```

System group	Worlds where it exists
`GhostInputSystemGroup`	Client, Local
`PredictedSimulationSystemGroup`	Client, Server
`PresentationSystemGroup`	Client only; not Server or Thin Client

---

3. Bootstrap process

`ClientServerBootstrap` creates Server and Client worlds at runtime.

3.1 Default behavior

The default bootstrap creates both a Server World and a Client World on startup. It does **not** automatically connect them, so the game still controls when to listen and connect.

3.2 Custom bootstrap

```
using Unity.NetCode;

public class GameBootstrap : ClientServerBootstrap
{
    public override bool Initialize(string defaultWorldName)
    {
        // Create only the default local world.
        CreateLocalWorld(defaultWorldName);
```

```
        return true;
    }
}
```

You can then create worlds from a play button, lobby flow, test harness, or command line.

```
using Unity.NetCode;

void OnPlayButtonClicked()
{
    var serverWorld = ClientServerBootstrap.CreateServerWorld("ServerWorld");
    var clientWorld = ClientServerBootstrap.CreateClientWorld("ClientWorld");

    AutomaticThinClientWorldsUtility.NumThinClientsRequested = 4;
    AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds();
}
```

`AutomaticThinClientWorldsUtility` was added in the 1.5.0 line and is used to create or clean up Thin Client worlds at runtime.

---

4. Tick-rate configuration

Configure server simulation speed with the `ClientServerTickRate` singleton.

Property	Default	Meaning
`SimulationTickRate`	60	Server simulation ticks per second.
`NetworkTickRate`	Same as `SimulationTickRate`	Snapshot send rate in ticks per second.
`MaxSimulationStepsPerFrame`	4	Maximum simulation steps per frame.
`MaxSimulationStepBatchSize`	4	Number of ticks that can be batched with scaled `deltaTime`.
`TargetFrameRateMode`	`BusyWait`	`BusyWait`, `Sleep`, or `Auto`.

```
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

public class TickRateAuthoring : MonoBehaviour
{
    public int SimulationTickRate = 60;
    public int NetworkTickRate = 30;

    class Baker : Baker<TickRateAuthoring>
    {
        public override void Bake(TickRateAuthoring authoring)
```

```
        {
            var entity = GetEntity(TransformUsageFlags.None);
            AddComponent(entity, new ClientServerTickRate
            {
                SimulationTickRate = authoring.SimulationTickRate,
                NetworkTickRate = authoring.NetworkTickRate,
            });
        }
    }
}
```

Lowering `NetworkTickRate` below `SimulationTickRate` saves bandwidth. In that setup, `GhostSendSystem` load is spread across multiple simulation ticks with round-robin scheduling.

---

5. Server and client update flow

```
Server World
└─ Fixed timestep, 60 Hz default
   └─ SimulationSystemGroup
      ├─ GhostSendSystem
      └─ CommandReceiveSystem

Client World
├─ Dynamic timestep
│  └─ SimulationSystemGroup
│     ├─ GhostReceiveSystem
│     └─ CommandSendSystem
├─ Fixed timestep, same as the server
│  └─ PredictedSimulationSystemGroup
│     └─ Prediction code and rollback loop
└─ PresentationSystemGroup
   └─ Rendering and interpolation
```

The server uses a fixed timestep for deterministic authoritative simulation. The client uses a dynamic timestep for normal simulation and presentation, while prediction code runs in the fixed predicted loop.

---

6. World migration

Use `DriverMigrationSystem` when a world transition must preserve connection state.

```
using Unity.Entities;
using Unity.NetCode;

public World MigrateServerWorld(World sourceWorld)
{
    DriverMigrationSystem migrationSystem = null;
    foreach (var world in World.All)
```

```
    {
        if ((migrationSystem = world.GetExistingSystemManaged<DriverMigrationSystem>()) != null)
            break;
    }

    var ticket = migrationSystem.StoreWorld(sourceWorld);
    sourceWorld.Dispose();

    var newWorld = ClientServerBootstrap.CreateServerWorld("NewServerWorld");
    var newMigrationSystem = newWorld.GetExistingSystemManaged<DriverMigrationSystem>();
    newMigrationSystem.LoadWorld(ticket);
    return newWorld;
}
```

---

7. Related docs

- `03_ECS Core Concepts.md` — World, EntityManager, archetype, and chunk basics.
- `09_System Group & Update Order.md` — base ECS system groups that Netcode extends.
- `17_Netcode Network Connection & Approval.md` — listen, connect, approve, and enter in-game state.

---

8. Troubleshooting

Symptom	Cause / Fix
A system runs on the server when it should not	Add a `WorldSystemFilter` or move it into the correct Netcode system group.
Tick-rate settings do not apply	Check that `ClientServerTickRate` was baked into a loaded SubScene.
Thin Clients are not created	Confirm `AutomaticThinClientWorldsUtility.BootstrapThinClientWorlds()` is called after setting `NumThinClientsRequested`.
Client and server connect automatically	Check whether `AutoConnectPort` is set in the bootstrap.
Presentation code errors in Play mode	Rendering systems are running in a server world. Add a `ClientSimulation` filter or move them to `PresentationSystemGroup`.

Chapter 20. Netcode Network Connection & Approval

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Netcode for Entities represents every network connection as an **entity**. The connection entity carries `NetworkStreamConnection`; when the connection ends, the entity is destroyed.

The connection is not considered "in game" until both sides add `NetworkStreamInGame`. Without that component, the server does not send snapshots and the client does not send commands.

---

2. Core connection components

Component	Meaning
`NetworkStreamConnection`	Stores the transport handle.
`NetworkId`	Unique ID assigned by the server after approval.
`NetworkStreamInGame`	Required to exchange snapshots and command data.
`CommandTarget`	Entity that receives player commands; maintained by game code unless `AutoCommandTarget` is used.
`NetworkStreamRequestDisconnect`	Request to disconnect; do not call the driver disconnect directly.
`NetworkStreamIsReconnected`	Reconnection tracking component added in the 1.6.1 line.

---

3. Three connection methods

3.1 Manual connection with `NetworkStreamDriver`

```
using Unity.Entities;
using Unity.Networking.Transport;
using Unity.NetCode;

// Server.
var serverDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
```

```
serverDriver.ValueRW.Listen(NetworkEndpoint.AnyIpv4.WithPort(7979));

// Client.
var clientDriver = SystemAPI.GetSingletonRW<NetworkStreamDriver>();
clientDriver.ValueRW.Connect(
    state.EntityManager,
    NetworkEndpoint.Parse("127.0.0.1", 7979));
```

3.2 `AutoConnectPort` in bootstrap

```
using Unity.NetCode;

public class AutoConnectBootstrap : ClientServerBootstrap
{
    public override bool Initialize(string defaultWorldName)
    {
        AutoConnectPort = 7979;
        CreateDefaultClientServerWorlds();
        return true;
    }
}
```

The server listens on a wildcard address, and the client connects to loopback.

3.3 Request components

```
using Unity.Entities;
using Unity.NetCode;

var listenReq = serverWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestListen));
serverWorld.EntityManager.SetComponentData(listenReq,
    new NetworkStreamRequestListen { Endpoint = serverEndpoint });

var connectReq = clientWorld.EntityManager.CreateEntity(
    typeof(NetworkStreamRequestConnect));
clientWorld.EntityManager.SetComponentData(connectReq,
    new NetworkStreamRequestConnect { Endpoint = serverEndpoint });
```

---

4. Connection state machine

4.1 Client state

```
Connecting → Handshake → [Approval] → Connected → Disconnected
```

State	Meaning
Connecting	Raised once after `Connect`.

State	Meaning
Handshake	Transport connection established; protocol handshake pending.
Approval	Exists only when connection approval is enabled.
Connected	Server assigned `NetworkId`.
Disconnected	Disconnect or timeout; `DisconnectReason` is set.

4.2 Server state

```
Handshake → [Approval] → Connected → Disconnected
```

State	Meaning
Handshake	Client accepted; `NetworkProtocolVersion` exchange.
Approval	Exists only when connection approval is enabled.
Connected	Connection approved and `NetworkId` assigned.
Disconnected	Client disconnected.

4.3 Listening for connection events

```
using Unity.Burst;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateAfter(typeof(NetworkReceiveSystemGroup))]
[BurstCompile]
public partial struct ConnectionEventListener : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var events = SystemAPI.GetSingleton<NetworkStreamDriver>()
            .ConnectionEventsForTick;

        foreach (var evt in events)
        {
            Debug.Log($"[{state.WorldUnmanaged.Name}] {evt.ToFixedString()}!");
        }
    }
}
```

`ConnectionEventsForTick` is only valid inside `SimulationSystemGroup`; reading it elsewhere can miss or duplicate events.

---

5. Connection approval

Connection approval lets the server validate a whitelist, password, token, matchmaking ticket, or similar gate before assigning `NetworkId`.

5.1 Approval flow

```
Client                          Server
  │                               │
  ├─ Connect ──────────────────► │
  │                              ├─ Handshake
  │◄──── Protocol Handshake ────┤
  │                               │
  ├─ IApprovalRpcCommand ──────► │  Approval RPC only
  │                              ├─ Validate
  │                              ├─ Add ConnectionApproved
  │◄──── NetworkId assigned ────┤
         Connected
```

5.2 Enable approval

```
using Unity.Entities;
using Unity.NetCode;

using var drvQuery = serverWorld.EntityManager
    .CreateEntityQuery(ComponentType.ReadWrite<NetworkStreamDriver>());

drvQuery.GetSingletonRW<NetworkStreamDriver>().ValueRW
    .RequireConnectionApproval = true;
```

Set the same approval requirement on the client side.

5.3 Approval RPC

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

public struct ApprovalRpc : IApprovalRpcCommand
{
    public FixedString512Bytes Token;
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (connection, entity) in
            SystemAPI.Query<RefRW<NetworkStreamConnection>>()
```

```
                .WithNone<NetworkId>()
                .WithNone<ApprovalSent>()
                .WithEntityAccess())
        {
            var rpcEntity = ecb.CreateEntity();
            ecb.AddComponent(rpcEntity, new ApprovalRpc { Token = "MY_TOKEN" });
            ecb.AddComponent<SendRpcCommandRequest>(rpcEntity);
            ecb.AddComponent<ApprovalSent>(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerApprovalSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (req, rpc, entity) in
            SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ApprovalRpc>>()
                .WithEntityAccess())
        {
            var connEntity = req.ValueRO.SourceConnection;

            if (rpc.ValueRO.Token.Equals("MY_TOKEN"))
                ecb.AddComponent<ConnectionApproved>(connEntity);
            else
                ecb.AddComponent<NetworkStreamRequestDisconnect>(connEntity);

            ecb.DestroyEntity(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}
```

Approval timeout is controlled by `ClientServerTickRate.HandshakeApprovalTimeoutMS`; the default is 5000 ms.

---

6. Entering in-game state

After approval, both client and server must add `NetworkStreamInGame` before gameplay data is exchanged.

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ServerGoInGameSystem : ISystem
{
```

```
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (id, entity) in
            SystemAPI.Query<RefRO<NetworkId>>()
                .WithNone<NetworkStreamInGame>()
                .WithEntityAccess())
        {
            ecb.AddComponent<NetworkStreamInGame>(entity);
            // Spawn the player entity and set CommandTarget as needed.
        }

        ecb.Playback(state.EntityManager);
    }
}

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation |
    WorldSystemFilterFlags.ThinClientSimulation)]
public partial struct ClientGoInGameSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (id, entity) in
            SystemAPI.Query<RefRO<NetworkId>>()
                .WithNone<NetworkStreamInGame>()
                .WithEntityAccess())
        {
            ecb.AddComponent<NetworkStreamInGame>(entity);
        }

        ecb.Playback(state.EntityManager);
    }
}
```

---

7. Data-stream buffers

Buffer	Direction	Data
`IncomingRpcDataStreamBuffer`	Receive	RPC data.
`IncomingCommandDataStreamBuffer`	Receive	Command data.
`IncomingSnapshotDataStreamBuffer`	Receive on client	Snapshot data.
`OutgoingRpcDataStreamBuffer`	Send	RPC data.
`OutgoingCommandDataStreamBuffer`	Send on client	Command data.

---

8. Host migration

Host Migration preserves client connection state when the server changes.

Item	Meaning
Activation	Enabled by default from the 1.7.0 line; older versions required `ENABLE_HOST_MIGRATION`.
Reconnection tracking	`NetworkStreamIsReconnected` is added to both connection entities after reconnect.
Prespawn Ghosts	Prespawn Ghost IDs are preserved across Host Migration.

The 1.7.0 line substantially refactored Host Migration API names, so earlier samples need migration before use.

---

9. Related docs

- `16_Netcode Client-Server World & Bootstrap.md` — how Netcode worlds are created.
- `18_Netcode Ghost Snapshot & Synchronization.md` — why `NetworkStreamInGame` gates snapshot exchange.
- `21_Netcode RPC.md` — normal RPCs and approval RPCs.

---

10. Troubleshooting

Symptom	Cause / Fix
Connection succeeds but Ghosts do not appear	Add `NetworkStreamInGame` on both client and server.
Data still does not exchange after approval	Confirm `ConnectionApproved` was added to the connection entity.
Connection drops when the app loses focus	Set `Application.runInBackground = true`.
Protocol version mismatch	Use identical client/server builds and Ghost configurations.
Approval times out	Adjust `HandshakeApprovalTimeoutMS` or debug the approval RPC path.

Chapter 21. Netcode Ghost Snapshot & Synchronization

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

A **Ghost** is a networked entity simulated on the server and replicated to clients. The server periodically sends Ghost state as snapshots, and each client applies those snapshots through interpolation or prediction.

Mechanism	Use
Ghost	Eventual consistency over unreliable packets with built-in replication optimization.
RPC	One-shot reliable event.

---

2. Ghost vs RPC

Use Ghost for	Use RPC for
Spatial entity state near a client.	High-level game-flow events.
Data that needs client prediction.	One-shot commands such as chat or setting changes.
State that should still exist after a disconnect/reconnect cycle.	Events with no persistent replicated state.

---

3. Ghost authoring

Add `GhostAuthoringComponent` to the prefab that should be replicated.

Property	Meaning
Name	Ghost identifier.
Importance	Snapshot priority when bandwidth is constrained.
Supported Ghost Mode	`All`, `Interpolated`, or `Predicted`.

Property	Meaning
Default Ghost Mode	`Interpolated`, `Predicted`, or `OwnerPredicted`.
Optimization Mode	`Dynamic` by default, or `Static` for objects that rarely change.
MaxSendRate	Maximum replication rate in Hz, capped by `NetworkTickRate`.
UseSingleBaseline	Single baseline delta compression option from the 1.5.0 line.

3.1 Ghost modes

Mode	Meaning
Interpolated	Client interpolates between server snapshots. Visuals are smooth but delayed.
Predicted	Client simulates ahead to reduce input latency. CPU cost is higher.
OwnerPredicted	Owning client predicts; other clients interpolate.

---

4. Serialization attributes

4.1 `GhostField`

Use `GhostField` on component fields that should be serialized.

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

public struct MyComponent : IComponentData
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.Interpolate)]
    public float3 Position;

    [GhostField(Quantization = 100)]
    public float Health;

    [GhostField]
    public int Score;

    [GhostField(SendData = false)]
    public float LocalOnly;
}
```

`GhostField` property	Meaning
`Quantization`	Float-to-int multiplier; `1000` keeps three decimal places.
`Smoothing`	`Clamp`, `Interpolate`, or `InterpolateAndExtrapolate`.

`GhostField` property	Meaning
`MaxSmoothingDistance`	Distance threshold that disables interpolation for teleports.
`Composite`	Treat the struct as one change bit.
`SendData`	Set false to exclude the field from serialization.
`SubType`	Selects a specialized serialization rule.

4.2 `GhostEnabledBit`

Use `GhostEnabledBit` to replicate the enabled state of an enableable component.

```
using Unity.Entities;
using Unity.NetCode;

[GhostEnabledBit]
public struct MyEnableableComp : IComponentData, IEnableableComponent
{
    [GhostField] public int Value;
}
```

4.3 `GhostComponent`

Use `GhostComponent` to control prefab variants, child-entity serialization, and replication scope.

```
using Unity.Entities;
using Unity.NetCode;

[GhostComponent(
    PrefabType = GhostPrefabType.All,
    SendTypeOptimization = GhostSendType.AllClients,
    SendToOwner = SendToOwnerType.SendToNonOwner)]
public struct EnemyData : IComponentData
{
    [GhostField] public float Health;
}
```

Property	Options
`PrefabType`	`InterpolatedClient`, `PredictedClient`, `Client`, `Server`, `AllPredicted`, `All`.
`SendTypeOptimization`	None, `Interpolated`, `Predicted`, `All`.
`SendToOwner`	`SendToOwner`, `SendToNonOwner`, `All`.
`SendDataForChildEntity`	Replicate child-entity component data; slower than root entity data.

---

5. Buffer serialization

For `IBufferElementData`, every public field must have `[GhostField]`.

```
using Unity.Entities;
using Unity.NetCode;

[InternalBufferCapacity(8)]
public struct MyBuffer : IBufferElementData
{
    [GhostField] public int Value;
    [GhostField] public float Score;
}
```

Buffers are stricter than components: a buffer element needs every public field annotated, not just one serialized field.

---

6. Ghost component variants

Use a variant to define an alternate serialization schema without modifying the original component type.

```
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[GhostComponentVariation(typeof(LocalTransform), "Transform - 2D")]
[GhostComponent(PrefabType = GhostPrefabType.All)]
public struct Transform2dVariant
{
    [GhostField(Quantization = 1000, Smoothing = SmoothingAction.InterpolateAndExtrapolate)]
    public float3 Position;
}
```

Variant	Use
`ClientOnlyVariant`	Component exists only on clients.
`ServerOnlyVariant`	Component exists only on the server.
`DontSerializeVariant`	Disable serialization entirely.

---

7. Snapshot system

7.1 Partial snapshots

If a snapshot would exceed the MTU, Netcode sends higher-importance Ghosts first. Remaining Ghosts are sent in later ticks, so world state streams progressively.

7.2 `NetworkTickRate`

When `NetworkTickRate` is lower than `SimulationTickRate`, `GhostSendSystem` work is distributed over multiple simulation ticks.

```
SimulationTickRate = 60, NetworkTickRate = 30
Tick 1: GhostSendSystem sends a snapshot
Tick 2: GhostSendSystem skips
Tick 3: GhostSendSystem sends a snapshot
...
```

---

8. Ghost prefab registration workflow

```
1. Create a Ghost prefab
   ├─ Add GhostAuthoringComponent
   └─ Add the required authoring components

2. Place or reference it from a SubScene
   ├─ Put the prefab inside the SubScene, or
   └─ Let GhostCollection register the prefab

3. Bake
   ├─ Baker analyzes GhostField attributes
   └─ Source Generator emits serialization code

4. Runtime
   ├─ Server instantiates the Ghost entity
   ├─ GhostSendSystem sends snapshots
   └─ Client automatically spawns the Ghost
```

8.1 Prespawned Ghosts

Ghosts placed in a SubScene become **Prespawned Ghosts**:

- Server and client automatically match the same entity when the SubScene loads.
- No separate spawn message is needed; snapshot sync is enough.
- Prespawn Ghost IDs are preserved during Host Migration in the 1.6.1+ line.

Static world objects such as buildings and terrain are usually best as Prespawned Ghosts with Static Optimization Mode.

---

9. Checklist

- ☐ Add `GhostAuthoringComponent` to the Ghost prefab.
- ☐ Mark replicated fields with `[GhostField]`.
- ☐ Choose Ghost Mode: `Predicted`, `Interpolated`, or `OwnerPredicted`.
- ☐ Add `NetworkStreamInGame` so snapshot exchange can begin.
- ☐ Tune `Quantization` for bandwidth and precision.
- ☐ Enable `SendDataForChildEntity` only when child-entity replication is required.

---

10. Related docs

- `22_Netcode Ghost Optimization · Importance · Relevancy.md` — bandwidth, relevancy, and send-rate controls.
- `19_Netcode Prediction & Rollback.md` — predicted Ghost simulation.
- `../Changelog/Netcode for Entities 1.4 → 6.5 Key Changes.md` — Ghost serialization changes across versions.

---

11. Troubleshooting

Symptom	Cause / Fix
Ghost does not appear on the client	Add `NetworkStreamInGame`, and confirm the Ghost prefab is registered.
Float values jitter	`Quantization` is too low or smoothing is missing.
Teleport creates interpolation artifacts	Set `MaxSmoothingDistance` or use a custom smoothing path.
Protocol version mismatch	Client and server Ghost schemas differ.
Buffer does not serialize	Every public buffer element field must have `[GhostField]`.

Chapter 22. Netcode Prediction & Rollback

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Client prediction runs the same simulation code on the client before the authoritative server result arrives. This reduces input latency and makes player-owned movement feel immediate.

Prediction has a real CPU cost: when an authoritative snapshot arrives, the client can roll back to an older tick and re-simulate forward to the current predicted tick.

---

2. Core components and systems

Component / system	Meaning
`PredictedGhost`	Added to predicted Ghosts on clients and to all Ghosts on the server.
`Simulate`	Enableable tag set on entities that should simulate for the current prediction tick.
`PredictedSimulationSystemGroup`	Fixed-timestep group that runs the prediction loop.
`NetworkTime`	Holds server tick, prediction tick, and related timing data.

---

3. Client prediction flow

```
1. Receive a server snapshot
   └ Apply the latest authoritative state to predicted entities

2. Find the oldest applied tick
   └ Use the oldest tick across affected predicted entities

3. Roll back and re-simulate
   └ PredictedSimulationSystemGroup repeats from oldest tick to target tick
   └ TimeData and NetworkTime update for each tick

4. Render
   └ Present the predicted result
```

High latency multiplies the cost. A 300 ms connection can require roughly 20+ frames of re-simulation depending on tick rate and prediction settings.

---

4. Writing prediction systems

4.1 Use the `Simulate` tag

At the start of the prediction loop, Netcode disables `Simulate` on predicted Ghosts and then enables it only for entities that should run on the current tick.

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct MovementSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 5f;
        var dt = SystemAPI.Time.DeltaTime;

        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<Simulate>())
        {
            var move = new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical);
            transform.ValueRW.Position += move * speed * dt;
        }
    }
}
```

Always include `.WithAll<Simulate>()` in prediction queries. Without it, the system can re-simulate entities for ticks that are already done.

4.2 Server-side behavior

On the server, the prediction loop runs exactly once per server tick:

- `TimeData` contains normal server tick data.
- `Simulate` is always enabled.
- The same system code can run on both client and server.

---

5. Remote-player prediction

If another player's command data is serialized, that remote player can also be predicted.

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct RemotePlayerMovement : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        foreach (var (transform, input) in
            SystemAPI.Query<RefRW<LocalTransform>, RefRO<PlayerInput>>()
                .WithAll<PredictedGhost, Simulate>())
        {
            transform.ValueRW.Position +=
                new float3(input.ValueRO.Horizontal, 0, input.ValueRO.Vertical)
                * SystemAPI.Time.DeltaTime;
        }
    }
}
```

With `IInputComponentData`, the input system provides the input for the current tick automatically.

---

6. Prediction smoothing

Prediction smoothing reduces visible correction when the server result differs from the predicted result.

```
using Unity.NetCode;
using Unity.Transforms;

GhostPredictionSmoothing.RegisterSmoothingAction<LocalTransform>(
    state.EntityManager,
    CustomSmoothing.Action);
```

```
using Unity.Burst;
using Unity.Collections.LowLevel.Unsafe;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[BurstCompile]
public static unsafe class CustomSmoothing
{
    public static readonly PortableFunctionPointer<
        GhostPredictionSmoothing.SmoothingActionDelegate> Action =
```

```
        new(SmoothingAction);

    [BurstCompile(DisableDirectCall = true)]
    private static void SmoothingAction(
        void* currentData, void* previousData, void* userData)
    {
        ref var current = ref UnsafeUtility.AsRef<LocalTransform>(currentData);
        ref var backup = ref UnsafeUtility.AsRef<LocalTransform>(previousData);

        var dist = math.distance(current.Position, backup.Position);
        if (dist > 0)
        {
            current.Position = math.lerp(backup.Position, current.Position, 0.5f);
        }
    }
}
```

---

7. Prediction switching

Prediction is expensive, so a Ghost can switch dynamically between `Predicted` and `Interpolated`.

7.1 Switch prerequisites

- Supported Ghost Modes must be `All`.
- Current mode must not be `OwnerPredicted`.
- The Ghost must not already be switching.

7.2 Queueing switches

```
using Unity.NetCode;

ref var switchQueues = ref SystemAPI
    .GetSingletonRW<GhostPredictionSwitchingQueues>()
    .ValueRW;

switchQueues.ConvertToPredictedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
    TransitionDurationSeconds = 1f,
});

switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
{
    TargetEntity = entity,
    TransitionDurationSeconds = 0f,
});
```

Interpolated and Predicted Ghosts run on different timelines, often separated by roughly two ping intervals. `SwitchPredictionSmoothing` interpolates position and rotation over the configured duration to avoid visible teleports.

---

8. Prediction performance controls

Technique	Effect
Prediction Switching	Switch distant or low-importance Ghosts to Interpolated.
Restrict GhostField write access	Use read-only access for fields that do not change in the prediction loop.
Physics batching	Use `MaxPredictionStepBatchSizeRepeatedTick` for repeated physics re-simulation.
Tick-rate tuning	Lower `SimulationTickRate` reduces re-simulation count, trading off responsiveness.

---

9. Related docs

- `20_Netcode Command Stream & Input.md` — input data used by prediction systems.
- `22_Netcode Ghost Optimization · Importance · Relevancy.md` — prediction switching as an optimization strategy.
- `23_Netcode Physics Integration & Lag Compensation.md` — predicted physics behavior.

---

10. Troubleshooting

Symptom	Cause / Fix
Prediction does not run	Check that the Ghost mode is Predicted and that prediction queries include `Simulate`.
Heavy jitter	Register prediction smoothing and confirm quantization matches gameplay needs.
Entity teleports on server correction	Set `MaxSmoothingDistance` or add custom smoothing.
Prediction CPU is too high	Use prediction switching, reduce tick rate, and batch physics re-simulation where appropriate.
Entity simulates more than once per tick	A prediction query is missing `.WithAll<Simulate>()`.

Chapter 23. Netcode Command Stream & Input

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Once a client is in-game, it sends a continuous **command stream** to the server. The stream includes input data and snapshot ACK data; it continues even when the player is not actively pressing input so the connection remains synchronized.

---

2. `ICommandData` vs `IInputComponentData`

	`ICommandData`	`IInputComponentData`
Use	Low-level command control.	High-level input workflow.
Buffer management	Manual, via `AddCommandData`.	Automatic through generated code.
Tick mapping	Manual, via `GetDataAtTick`.	Automatic; current tick input is provided.
`InputEvent`	Not supported.	Supported for one-shot events.
Maximum payload	1024 bytes.	1024 bytes.
Best fit	Custom command pipelines.	Normal player input.

---

3. `IInputComponentData`

3.1 Define input

```
using Unity.NetCode;

public struct PlayerInput : IInputComponentData
{
    public int Horizontal;
    public int Vertical;
    public InputEvent Jump;
}
```

`InputEvent` preserves one-shot events such as `GetKeyDown` so they register exactly once on the server even if packet delivery drops a redundant command packet.

3.2 Gather input on the client

```
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[UpdateInGroup(typeof(GhostInputSystemGroup))]
public partial struct GatherInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    public void OnUpdate(ref SystemState state)
    {
        bool jump = Input.GetKeyDown(KeyCode.Space);
        bool left = Input.GetKey(KeyCode.A);
        bool right = Input.GetKey(KeyCode.D);

        foreach (var input in
            SystemAPI.Query<RefRW<PlayerInput>>()
                .WithAll<GhostOwnerIsLocal>())
        {
            input.ValueRW = default;
            if (jump) input.ValueRW.Jump.Set();
            if (left) input.ValueRW.Horizontal -= 1;
            if (right) input.ValueRW.Horizontal += 1;
        }
    }
}
```

3.3 Process input on server and prediction loop

```
using Unity.Burst;
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[UpdateInGroup(typeof(PredictedSimulationSystemGroup))]
public partial struct ProcessInputSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<PlayerInput>();
    }

    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        var speed = 3f;
        var dt = SystemAPI.Time.DeltaTime;
```

```
        foreach (var (input, transform) in
            SystemAPI.Query<RefRO<PlayerInput>, RefRW<LocalTransform>>()
                .WithAll<Simulate>())
        {
            if (input.ValueRO.Jump.IsSet)
            {
                // Jump logic; runs once on the input tick.
            }

            transform.ValueRW.Position +=
                new float3(input.ValueRO.Horizontal, 0, 0) * speed * dt;
        }
    }
}
```

The prediction loop can execute this system many times while re-simulating. `InputEvent.IsSet` is true only for the correct tick.

---

4. `ICommandData`

4.1 Define a command

```
using Unity.Mathematics;
using Unity.NetCode;

public struct MyCommand : ICommandData
{
    public NetworkTick Tick { get; set; }
    public int Action;
    public float2 Direction;
}
```

4.2 Add and read command data

```
using Unity.Entities;
using Unity.NetCode;

DynamicBuffer<MyCommand> buffer = default;
NetworkTick networkTick = default;

buffer.AddCommandData(new MyCommand
{
    Tick = networkTick,
    Action = 1,
    Direction = new float2(1, 0),
});

if (buffer.GetDataAtTick(networkTick, out var cmd))
{
```

```
    // Use cmd.Action and cmd.Direction.
}
```

---

5. Command-send mechanism

```
Command packet
├─ Tick, Command
├─ Delta(Tick - 1)
├─ Delta(Tick - 2)
└─ Delta(Tick - 3)

The last three inputs are redundantly sent with delta compression.
Maximum payload: 1024 bytes.
```

`CommandSendPacketSystem` flushes at `SimulationTickRate`, and `CommandReceiveSystem` receives the command data on the server and forwards it to the target entity.

---

6. `AutoCommandTarget`

If a Ghost has `AutoCommandTarget`, commands are sent automatically.

Requirement	Meaning
Ghost has owner enabled	Enabled on `GhostAuthoringComponent`.
Support Auto Command Target enabled	Enabled on `GhostAuthoringComponent`.
Client is the owner	Server sets `GhostOwner.NetworkId` to the client's `NetworkId.Value`.
Ghost is Predicted or OwnerPredicted	Command data targets prediction.
`AutoCommandTarget.Enabled` is true	Can be disabled at runtime.

Without `AutoCommandTarget`, set `CommandTarget` on the connection entity manually.

```
using Unity.Entities;
using Unity.NetCode;

Entity connectionEntity = default;
Entity playerEntity = default;

state.EntityManager.SetComponentData(connectionEntity,
    new CommandTarget { targetEntity = playerEntity });
```

---

7. Identifying locally owned entities

7.1 `GhostOwnerIsLocal`

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

foreach (var (input, transform) in
    SystemAPI.Query<RefRW<PlayerInput>, RefRW<LocalTransform>>()
        .WithAll<GhostOwnerIsLocal>())
{
    // Local-owned entity only.
}
```

7.2 Manual owner check

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Transforms;

var localId = SystemAPI.GetSingleton<NetworkId>().Value;

foreach (var (owner, transform) in
    SystemAPI.Query<RefRO<GhostOwner>, RefRW<LocalTransform>>())
{
    if (owner.ValueRO.NetworkId == localId)
    {
        // Local-owned entity.
    }
}
```

---

8. Forced input latency

`ClientTickRate.ForcedInputLatencyTicks` adds intentional input delay.

Setting	Meaning
`ForcedInputLatencyTicks`	Additional input latency in ticks; default is 0.
Use case	Give all players the same input delay in genres such as fighting games.
Effect	Included automatically in `MaxPredictAheadTimeMS` calculations.

8.1 `InputTargetTick` vs `ServerTick`

In input systems, use `NetworkTime.InputTargetTick` rather than `NetworkTime.ServerTick` when forced input latency is active.

```
using Unity.NetCode;

var inputTick = SystemAPI.GetSingleton<NetworkTime>().InputTargetTick;
var serverTick = SystemAPI.GetSingleton<NetworkTime>().ServerTick;
```

`InputTargetTick` and `ServerTick` differ when `ForcedInputLatencyTicks` is non-zero. Input should target `InputTargetTick`.

---

9. Related docs

- `19_Netcode Prediction & Rollback.md` — prediction systems that consume input.
- `17_Netcode Network Connection & Approval.md` — `NetworkStreamInGame` and connection entities.
- `21_Netcode RPC.md` — reliable one-shot messages, not continuous input.

---

10. Troubleshooting

Symptom	Cause / Fix
Server receives no input	Add `NetworkStreamInGame`, set `CommandTarget`, or satisfy `AutoCommandTarget` requirements.
Jump fires more than once	Use `InputEvent.Set()` / `.IsSet` instead of a normal bool.
Payload-size errors	Keep command structs at or below 1024 bytes.
Input missing during prediction	Prefer `IInputComponentData` automatic tick mapping, or check `GetDataAtTick`.
Server code runs in `GhostInputSystemGroup`	That group is client-side; put server logic in `PredictedSimulationSystemGroup`.

Chapter 24. Netcode RPC

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

An RPC is a **reliable one-shot message** between client and server. RPC serialization and deserialization code is generated automatically, and RPC handling still fits into the ECS / Job System frame.

RPCs are not a replacement for Ghost replication or command streams. Use them for discrete game-flow events, not high-frequency state.

---

2. RPC vs Ghost vs Command

	RPC	Ghost	Command
Transport behavior	Reliable, ordered.	Unreliable with replication optimization.	Unreliable with redundant sends.
Direction	Bidirectional.	Server → Client.	Client → Server.
Frequency	One-shot.	Every network tick as needed.	Every simulation tick.
Use	Game flow, lobby, chat.	Replicated entity state.	Player input.
In-flight limit	Yes, reliable pipeline limit.	No.	No.

Because RPCs are ordered and reliable, frequent RPC traffic can stall behind in-flight packets.

---

3. Define RPCs

3.1 Empty RPC

```
using Unity.NetCode;

public struct PingRpc : IRpcCommand
{
}
```

3.2 RPC with data

```
using Unity.Collections;
using Unity.NetCode;

public struct ChatMessageRpc : IRpcCommand
{
    public FixedString128Bytes Message;
    public int SenderId;
}
```

---

4. Send RPCs

4.1 Client to server

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
public partial struct SendChatSystem : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.RequireForUpdate<NetworkId>();
    }

    public void OnUpdate(ref SystemState state)
    {
        if (!Input.GetKeyDown(KeyCode.Space))
            return;

        var ecb = new EntityCommandBuffer(Allocator.Temp);
        var entity = ecb.CreateEntity();

        ecb.AddComponent(entity, new ChatMessageRpc
        {
            Message = "Hello!",
            SenderId = SystemAPI.GetSingleton<NetworkId>().Value,
        });

        ecb.AddComponent<SendRpcCommandRequest>(entity);
        ecb.Playback(state.EntityManager);
    }
}
```

`ISystem` is a struct. Do not store frame-to-frame state in system instance fields; store state in components or singletons instead.

4.2 Server to one client

```
using Unity.Entities;
using Unity.NetCode;

Entity targetConnectionEntity = default;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new NotifyRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = targetConnectionEntity,
});
```

4.3 Server broadcast

```
using Unity.Entities;
using Unity.NetCode;

var entity = ecb.CreateEntity();
ecb.AddComponent(entity, new GameStartRpc());
ecb.AddComponent(entity, new SendRpcCommandRequest
{
    TargetConnection = Entity.Null,
});
```

`Entity.Null` broadcasts the RPC.

---

5. Receive RPCs

```
using Unity.Collections;
using Unity.Entities;
using Unity.NetCode;
using UnityEngine;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
public partial struct ReceiveChatSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        var ecb = new EntityCommandBuffer(Allocator.Temp);

        foreach (var (req, chat, entity) in
            SystemAPI.Query<RefRO<ReceiveRpcCommandRequest>, RefRO<ChatMessageRpc>>()
                .WithEntityAccess())
        {
            var sourceConnection = req.ValueRO.SourceConnection;
            Debug.Log($"Player {chat.ValueRO.SenderId}: {chat.ValueRO.Message}");

            ecb.DestroyEntity(entity);
        }

        ecb.Playback(state.EntityManager);
```

```
    }
}
```

Always destroy the received RPC entity after processing it. Otherwise the same RPC is processed again every frame.

---

6. RpcQueue

Use RpcQueue only when manual RPC scheduling is required.

```
using Unity.Entities;
using Unity.NetCode;

var rpcQueue = SystemAPI.GetSingleton<RpcCollection>()
    .GetRpcQueue<MyRpc, MyRpc>();
var bufferLookup = SystemAPI.GetBufferLookup<OutgoingRpcDataStreamBuffer>();

if (bufferLookup.HasBuffer(connectionEntity))
{
    var buffer = bufferLookup[connectionEntity];
    rpcQueue.Schedule(buffer, new MyRpc());
}
```

---

7. Manual serialization

Automatic generation is the default. If you disable it, serialization and deserialization must be symmetric.

```
using Unity.Burst;
using Unity.Entities;
using Unity.NetCode;
using Unity.Networking.Transport;

[BurstCompile]
public struct MyCustomRpc : IComponentData, IRpcCommandSerializer<MyCustomRpc>
{
    public int Data;

    public void Serialize(ref DataStreamWriter writer, in RpcSerializerState state,
        in MyCustomRpc data)
    {
        writer.WriteInt(data.Data);
    }

    public void Deserialize(ref DataStreamReader reader, in RpcDeserializerState state,
        ref MyCustomRpc data)
    {
        data.Data = reader.ReadInt();
    }
```

```
    // Execute implementation omitted.
}
```

---

8. IApprovalRpcCommand

IApprovalRpcCommand is a special RPC used only during connection approval. Normal IRpcCommand messages are blocked during Handshake / Approval state.

```
using Unity.Collections;
using Unity.NetCode;

public struct LoginApproval : IApprovalRpcCommand
{
    public FixedString512Bytes AuthToken;
}
```

See 17_Netcode Network Connection & Approval.md for the full approval flow.

---

9. Common patterns

Pattern	Example RPCs
Lobby / matchmaking	JoinLobbyRpc, ReadyRpc.
Game flow	GameStartRpc, RoundEndRpc.
Chat	ChatMessageRpc.
Item trading	TradeRequestRpc, TradeAcceptRpc.
Connection approval	IApprovalRpcCommand implementations.

---

10. Related docs

- 18_Netcode Ghost Snapshot & Synchronization.md — Ghost replication for state.
- 20_Netcode Command Stream & Input.md — command streams for input.
- 17_Netcode Network Connection & Approval.md — approval RPCs.

---

11. Troubleshooting

Symptom	Cause / Fix
RPC is not received	Confirm `SendRpcCommandRequest` was added and the system runs in the intended world.
RPC is processed every frame	Destroy the received RPC entity after handling it.
RPC order appears delayed	Reliable ordered transport can stall behind in-flight packets. Use Ghosts or commands for frequent data.
RPC is blocked during approval	Use `IApprovalRpcCommand`, not `IRpcCommand`.
Manual serialization fails	Ensure `Serialize` and `Deserialize` read/write fields in the same order.

Chapter 25. Netcode Ghost Optimization · Importance · Relevancy

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Large multiplayer worlds cannot send every Ghost to every client every tick. Netcode for Entities uses **Optimization Mode**, **Importance Scaling**, **Relevancy**, and per-Ghost send-rate controls to reduce CPU cost and bandwidth.

The core rule is simple: the server should spend snapshot budget on Ghosts that matter to a specific connection right now.

---

2. Optimization Mode

Set Optimization Mode on `GhostAuthoringComponent`.

Mode	Meaning	Good fit
Dynamic	Default. Optimizes snapshot size both when data changes and when it does not.	Normal moving Ghosts.
Static	Avoids re-sending data when the Ghost has not changed.	Buildings, terrain, static props.

The 1.11.0 line added a Static Ghost optimization that can substantially reduce `GhostUpdateSystem` timing for unchanged static Ghosts.

---

3. Snapshot-size limits

Snapshot budget can be limited at several levels.

Setting	Meaning
`NetworkStreamSnapshotTargetSize`	Per-connection soft byte target.
`GhostSendSystemData.MaxSendEntities`	Maximum entities per snapshot.

Setting	Meaning
`GhostSendSystemData.MaxSendChunks`	Maximum chunks per snapshot.

When budget is limited, higher-importance Ghosts are sent first.

---

4. Importance Scaling

Importance Scaling computes per-Ghost send priority on the server.

4.1 Inputs

`GhostImportance` combines three kinds of data.

Data	Meaning
Per-connection	Connection-specific data such as player position.
Singleton data	Global settings such as tile size.
Per-chunk shared data	Chunk-level scaling parameters.

4.2 Distance-based importance

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

var gridSingleton = state.EntityManager.CreateSingleton(
    new GhostDistanceData
    {
        TileSize = new int3(50, 50, 256),
        TileCenter = new int3(0, 0, 128),
        TileBorderWidth = new float3(1f, 1f, 1f),
    });

state.EntityManager.AddComponentData(gridSingleton, new GhostImportance
{
    ScaleImportanceFunction = GhostDistanceImportance.ScaleFunctionPointer,
    GhostConnectionComponentType = ComponentType.ReadOnly<GhostConnectionPosition>(),
    GhostImportanceDataType = ComponentType.ReadOnly<GhostDistanceData>(),
    GhostImportancePerChunkDataType = ComponentType.ReadOnly<GhostDistancePartitionShared>(),
});
```

Breaking change from the 1.8.0 line: `GhostDistanceImportance` scale functions no longer multiply `baseImportance` by 1000. Existing custom importance code that assumed the old multiplier needs review.

4.3 Automatic `GhostDistancePartitionShared`

```
using Unity.NetCode;

GhostDistancePartitioningSystem.AutomaticallyAddGhostDistancePartitionSharedComponent = true;
```

4.4 Importance Visualizer

The PlayMode Tool includes an Importance Visualizer in the 1.8.0+ line. Use it to inspect the runtime importance distribution instead of guessing from code.

---

5. Ghost Relevancy

Ghost Relevancy lets the server hide or expose specific Ghosts per connection.

Mode	Meaning
`Disabled`	Default. No filtering; all Ghosts are candidates.
`SetIsRelevant`	Only listed Ghosts replicate to that connection.
`SetIsIrrelevant`	Listed Ghosts are excluded from that connection.

5.1 Use cases

Scenario	Implementation idea
Distance culling	Mark distant Ghosts irrelevant.
Fog of war	Hide entities outside vision.
Client-specific Ghosts	Mark a Ghost relevant only for one client.
Team filtering	Hide enemy-only internal data.

5.2 Relevancy example

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;
using Unity.Transforms;

[WorldSystemFilter(WorldSystemFilterFlags.ServerSimulation)]
[UpdateInGroup(typeof(GhostSimulationSystemGroup))]
[UpdateBefore(typeof(GhostSendSystem))]
public partial struct RelevancySystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
```

```
        var relevancy = SystemAPI.GetSingletonRW<GhostRelevancy>();

        relevancy.ValueRW.GhostRelevancyMode = GhostRelevancyMode.SetIsRelevant;
        relevancy.ValueRW.GhostRelevancySet.Clear();

        foreach (var (ghostInstance, transform) in
            SystemAPI.Query<RefRO<GhostInstance>, RefRO<LocalTransform>>())
        {
            foreach (var (connId, connPos) in
                SystemAPI.Query<RefRO<NetworkId>, RefRO<GhostConnectionPosition>>())
            {
                float dist = math.distance(
                    transform.ValueRO.Position,
                    connPos.ValueRO.Position);

                if (dist < 100f)
                {
                    relevancy.ValueRW.GhostRelevancySet.TryAdd(
                        new RelevantGhostForConnection(
                            connId.ValueRO.Value,
                            ghostInstance.ValueRO.ghostId),
                        1);
                }
            }
        }
    }
}
```

This simplified example is O(Ghosts × connections). For large worlds, prefer distance-based importance partitioning and reserve relevancy for rules that partitioning cannot express, such as fog of war or team visibility.

5.3 `DefaultRelevancyQuery`

`DefaultRelevancyQuery` defines Ghost chunks that are always relevant to every connection, such as global game-state Ghosts or UI-related network state.

5.4 Relevancy plus importance

The 1.6.1+ line supports combining Ghost Relevancy and Importance Scaling through `PrioChunks.isRelevant`, enabling a faster relevancy path.

---

6. Prediction CPU optimization

6.1 Prediction Switching

Switch distant or unimportant Ghosts from Predicted to Interpolated.

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.NetCode;

float distance = math.distance(playerPos, ghostPos);
if (distance > predictionRange)
{
    switchQueues.ConvertToInterpolatedQueue.Enqueue(new ConvertPredictionEntry
    {
        TargetEntity = ghostEntity,
        TransitionDurationSeconds = 0.5f,
    });
}
```

6.2 Minimize GhostField write access

Jobs with write access to GhostField components can trigger serialization checks. Use read-only access for fields that do not change in the prediction loop.

6.3 Physics re-simulation

For high-latency cases that require 20+ predicted re-simulation frames:

- Disable multithreaded physics through the `PhysicsStep` singleton when main-thread consolidation is cheaper.
- Use `MaxPredictionStepBatchSizeRepeatedTick` to batch repeated physics ticks.

---

7. `MaxIterateChunks`

`GhostSendSystemData.MaxIterateChunks` limits the number of chunks processed in a single tick.

Setting	Meaning
`MaxIterateChunks`	Limits serialization CPU cost per tick.
`MaxSendChunks`	Limits bandwidth by capping chunks included in a snapshot.

These settings solve different problems. `MaxIterateChunks` spreads CPU work; `MaxSendChunks` constrains snapshot size.

---

8. `MaxSendRate`

`GhostAuthoringComponent.MaxSendRate` limits a Ghost's send frequency in Hz.

```
NetworkTickRate = 60, MaxSendRate = 10
→ This Ghost can be included once every 6 network ticks.
```

Use this for low-frequency or low-importance Ghosts.

---

9. `UseSingleBaseline`

`GhostAuthoringComponent.UseSingleBaseline = true` uses one baseline for delta compression. This can reduce CPU cost, with a possible bandwidth increase.

---

10. Optimization checklist

- [ ] Use **Static** Optimization Mode for Ghosts that rarely change.
- [ ] Set `NetworkStreamSnapshotTargetSize` to cap bandwidth per connection.
- [ ] Implement distance-based Importance Scaling.
- [ ] Apply Relevancy only where partitioning cannot express the rule.
- [ ] Switch distant Ghosts to Interpolated mode.
- [ ] Use `MaxSendRate` for low-frequency Ghosts.
- [ ] Avoid write access to unchanged GhostField data.
- [ ] Inspect runtime priority with the Importance Visualizer.

---

11. Related docs

- `18_Netcode Ghost Snapshot & Synchronization.md` — Ghost serialization and snapshot flow.
- `19_Netcode Prediction & Rollback.md` — prediction switching behavior.
- `24_Netcode Profiler & Debugging.md` — profiling bandwidth and prediction cost.

---

12. Troubleshooting

Symptom	Cause / Fix
A Ghost takes too long to appear	Importance is too low or snapshot budget is too tight.

Symptom	Cause / Fix
Ghosts stop sending after Importance Scaling	Confirm `GhostDistancePartitionShared` is present or use the automatic-add option.
Ghost flickers with Relevancy enabled	Check boundary conditions and rebuild the relevancy set consistently each frame.
Bandwidth is available but Ghosts are still missing	Check `MaxSendEntities` and `MaxSendChunks` limits.

Chapter 26. Netcode Physics Integration & Lag Compensation

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Netcode for Entities integrates with **Unity Physics** for networked physics simulation. Physics behavior depends on whether the Ghost is Interpolated or Predicted.

At least one Predicted Ghost must exist in the scene for predicted physics to run.

---

2. Interpolated vs Predicted physics

2.1 Interpolated Ghosts on clients

- Position and rotation come from server snapshots.
- `Simulate` is disabled, so the physics object behaves as kinematic.
- `PhysicsVelocity` is ignored.

2.2 Predicted Ghosts on clients

- Physics runs inside the prediction loop and can execute multiple times per rendered frame.
- `PhysicsSystemGroup` runs inside `PredictedFixedStepSimulationSystemGroup`.
- The client is expected to produce the same simulation result as the server for predicted physics.

```
Netcode initialization
├─ PhysicsSystemGroup moves under PredictedFixedStepSimulationSystemGroup
└─ FixedStepSimulationSystemGroup moves under PredictedFixedStepSimulationSystemGroup
```

---

3. Predicted physics performance

Predicted physics is expensive because rollback can execute physics repeatedly.

3.1 Server batching

```
using Unity.NetCode;

ClientServerTickRate tickRate = default;
tickRate.MaxSimulationStepBatchSize = 4;
tickRate.MaxSimulationStepsPerFrame = 4;
```

3.2 Client batching

```
using Unity.NetCode;

ClientTickRate clientRate = default;
clientRate.MaxPredictionStepBatchSizeFirstTimeTick = 2;
clientRate.MaxPredictionStepBatchSizeRepeatedTick = 4;
```

Batching can increase the risk of misprediction, so use it as a performance tradeoff rather than a default.

3.3 Quantization

The default Transform / Velocity quantization for physics Ghosts is `1000`.

Higher quantization	Lower quantization
More precise simulation.	Lower bandwidth.
Higher bandwidth.	More precision loss.

---

4. Lag compensation

Because clients predict ahead of the server, the authoritative server sometimes needs to query an older collision world that matches the client's action time.

4.1 Enable lag compensation

Add `NetCodePhysicsConfig` to a SubScene and set `EnableLagCompensation = true`.

4.2 Query an old collision world

```
using Unity.NetCode;
using Unity.Physics;

var physicsHistory = SystemAPI.GetSingleton<PhysicsWorldHistorySingleton>();

physicsHistory.GetCollisionWorldFromTick(
    serverTick,
    interpolationDelay,
    ref physicsWorld,
    out var collisionWorld);

// Use collisionWorld for raycasts or other physics queries.
```

4.3 Collider deep cloning

Setting	Default	Meaning
`DeepCopyDynamicColliders`	`true`	Deep clone dynamic colliders; default in 6.5.0.
`DeepCopyStaticColliders`	`false`	Deep clone static colliders.

The 6.5.0 line enables dynamic Ghost collider deep cloning by default to avoid blob asset assertions.

4.4 Physics history buffer

Version	History size
Up to 1.4.x	16 ticks.
1.5.0+	32 ticks.
Custom	`GhostSystemConstants.SnapshotHistorySize` via compiler define.

---

5. Multi Physics World

Use a separate physics world for client-only effects such as VFX or particles that do not participate in networked simulation.

5.1 Setup

1. Add `NetCodePhysicsConfig` to the SubScene.
2. Set **Client Non Ghost World** to a non-zero value, such as `1`.
3. Add `PhysicsWorldIndex` with the same value to client-only physics objects.

5.2 Custom physics system group

```
using Unity.Entities;
using Unity.NetCode;
using Unity.Physics.Systems;

[WorldSystemFilter(WorldSystemFilterFlags.ClientSimulation)]
[UpdateInGroup(typeof(FixedStepSimulationSystemGroup))]
public partial class VfxPhysicsGroup : CustomPhysicsSystemGroup
{
    public const int WorldIndex = 1;

    public VfxPhysicsGroup() : base(WorldIndex, shareStaticColliders: true)
    {
    }
}
```

`shareStaticColliders: true` lets the custom physics world share static colliders with the main physics world.

---

6. Physics Proxy

Use a Physics Proxy when a Predicted Ghost must interact with a client-only physics entity.

- `CustomPhysicsProxyAuthoring` creates the proxy entity.
- `CustomPhysicsProxyDriver` links the proxy to the Predicted Ghost.
- `SyncCustomPhysicsProxySystem` synchronizes position and rotation.

---

7. `PredictedFixedStepSimulationSystemGroup`

`PredictedFixedStepSimulationSystemGroup` updates only when all required conditions are met.

Requirement	Meaning
`NetworkStreamInGame` singleton exists	The connection is in-game.
At least one Predicted Ghost exists	Predicted physics has something to simulate.
Fixed tick rate runs	Fixed-timestep simulation is active.

Before a network connection is in-game or before predicted Ghosts stream in, predicted physics does not run.

7.1 Physics before connection

If physics must run before a network connection exists, disable predicted physics setup so normal `FixedStepSimulationSystemGroup` physics can run.

```
using Unity.Entities;
using Unity.NetCode;

[UpdateInGroup(typeof(InitializationSystemGroup))]
[CreateAfter(typeof(PredictedPhysicsConfigSystem))]
public partial struct DisablePredictedPhysics : ISystem
{
    public void OnCreate(ref SystemState state)
    {
        state.World.GetExistingSystemManaged<PredictedPhysicsConfigSystem>()
            .Enabled = false;
    }
}
```

---

8. Limitations

Limitation	Meaning
Partial tick unsupported	Physics does not use partial ticks; use Physics Interpolation.
Multi-world debug limits	Unity Physics debug systems do not fully support multi-world setups.
Minimum Predicted Ghost	At least one Predicted Ghost is required for predicted physics to run.

---

9. Related docs

- `19_Netcode Prediction & Rollback.md` — prediction loop behavior.
- `22_Netcode Ghost Optimization · Importance · Relevancy.md` — switching Ghosts away from prediction.
- `24_Netcode Profiler & Debugging.md` — profiling predicted physics.

---

10. Troubleshooting

Symptom	Cause / Fix
Physics does not run	Check for a Predicted Ghost and `NetworkStreamInGame`.
Physics jitters	Check quantization and prediction smoothing.

Symptom	Cause / Fix
Lag-compensation raycast fails	Enable `EnableLagCompensation` and confirm history buffer coverage.
Blob asset assertion	Check collider deep-clone settings; dynamic clone is enabled by default in 6.5.0.
Client-only physics does not run	Match `PhysicsWorldIndex` to the **Client Non Ghost World** value.
Predicted physics CPU is too high	Enable batching and switch unnecessary Ghosts to Interpolated.

Chapter 27. Netcode Profiler & Debugging

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

1. Overview

Netcode for Entities provides a **Netcode Profiler** for analyzing connection state, snapshots, prediction, interpolation, and bandwidth. In the 1.12.0 line, the browser-based Network Debugger was deprecated in favor of the built-in Unity Profiler modules.

---

2. Netcode Profiler

2.1 Access

Unity Profiler includes two Netcode modules on Unity 6.0+.

Module	What to inspect
Server Profiler	Server-side Ghost send, snapshot composition, and bandwidth.
Client Profiler	Client receive, prediction ticks, and interpolation state.

2.2 Key features

Feature	Meaning
Ghost Snapshot View	Tree view of per-Ghost snapshot size and composition.
Tick navigation	Move between frames where snapshots were sent or received.
Search fields	Search Ghost Snapshot TreeView and Prediction Error List.
Client ticks	Visualize Prediction Tick and Interpolation Tick.
Context menu	Inspect Ghost Prefabs and Components from TreeView labels in the 6.5.0 line.
Average per-entity column	Shows average data size per entity.

2.3 `NETCODE_PROFILER_ENABLED`

The 1.12.0 line removed the need for the `NETCODE_PROFILER_ENABLED` scripting define. The Profiler is available without that define.

3. Network Debugger

The browser-based Network Debugger is deprecated in the 1.12.0 line. Use the Netcode Profiler instead.

The old flow was **Multiplayer** → **Open NetDbg**, which opened a browser-based view for snapshot composition, Ghost type distribution, and size analysis.

---

4. Multiplayer PlayMode Window

The Multiplayer PlayMode Window is the main editor workflow for testing client/server setups in Play mode.

Feature	Meaning
Play Mode Type	Choose Client + Server, Client Only, or Server Only.
Thin Client count	Configure dummy clients.
Simulator	Simulate network latency and packet loss.
Importance Visualizer	Visualize Importance Scaling in the 1.8.0+ line.

---

5. Debugging checklists

5.1 Connection

```
□ Application.runInBackground = true
□ Protocol versions match between client and server
□ NetworkStreamInGame is present
□ Firewall and port are open
□ Unity Transport version is compatible
```

5.2 Ghost synchronization

```
□ GhostAuthoringComponent is present
□ GhostField attributes are correct
□ Quantization is appropriate
□ Smoothing is configured
□ Ghost prefab is registered on both sides
```

5.3 Prediction

```
□ Systems run in PredictedSimulationSystemGroup
□ Queries include Simulate
□ Simulation code is deterministic enough for prediction
□ Client and server run the same simulation code
□ Prediction smoothing is registered where needed
```

5.4 Input and commands

```
□ Input is gathered in GhostInputSystemGroup
□ AutoCommandTarget requirements are met, or CommandTarget is set manually
□ Command payload is 1024 bytes or less
□ One-shot input uses InputEvent
```

---

6. Prefab List menu

The 1.11.0+ line includes a Prefab List menu for quickly listing and inspecting registered Ghost Prefabs in the project or scene hierarchy.

---

7. Useful debugging components

Component	Use
`NetworkStreamConnection`	Inspect connection state.
`NetworkId`	Inspect client ID.
`GhostInstance`	Inspect Ghost type and ID.
`PredictedGhost`	Inspect prediction state and applied tick.
`NetworkSnapshotAck`	Inspect snapshot ACK state and estimated RTT.
`GhostOwner`	Inspect Ghost owner ID.

7.1 Entity Inspector

In Entities Hierarchy, select a Ghost entity to inspect:

- live `GhostField` values,
- prediction tick data,
- network connection state.

---

8. Packet dumps

Use packet dumps when low-level network packet analysis is required. In the 1.6.2 line, packet timestamp logs gained `UnityEngine.Time.frameCount`, which makes frame-level correlation easier.

---

9. Common bottlenecks

Area	Where to look
Snapshot bandwidth	Profiler Ghost Snapshot View; inspect per-entity size.
Prediction re-simulation	Profiler Client Tick; inspect predicted tick count.
`GhostSendSystem`	Profiler Timeline; inspect system duration.
Serialization cost	Check GhostField write access and change frequency.
Physics re-simulation	Inspect `PredictedFixedStepSimulationSystemGroup` duration.

---

10. Editor Analytics

The 1.12.0 line collects editor analytics for Netcode Profiler usage patterns.

---

11. Related docs

- `Optimizations and Debugging/03_Profiler · Bottleneck Analysis.md` — general DOTS profiling workflow.
- `22_Netcode Ghost Optimization · Importance · Relevancy.md` — bandwidth controls.
- `23_Netcode Physics Integration & Lag Compensation.md` — predicted physics costs.

---

12. Troubleshooting

Symptom	Cause / Fix
No Netcode Profiler data	Confirm the test is running in a client/server Play Mode and using Unity 6.0+.
Ghost Snapshot View is large but gameplay feels fine	Look for low-priority Ghosts with excessive fields or send rate.

Symptom	Cause / Fix
Prediction errors spike	Check `Simulate` queries, deterministic code paths, and input tick handling.
Network Debugger workflow is missing	It is deprecated; use Unity Profiler Netcode modules.
Packet-level logs are hard to correlate	Use packet dumps with frame-count timestamps in the 1.6.2+ line.

Part III. Optimization and Debugging

Chapter 28. Chunk Layout & TypeManager

1. The 16 KB chunk

Every archetype stores its entities in **16 KB chunks**. The chunk holds:

1. A small header (metadata — archetype id, version numbers, enable-bit masks, change versions).
2. A contiguous **SoA** region: one column per component type, each column an array of values.
3. Padding to the 16 KB boundary.

Chunk capacity — how many entities fit — falls out of the math:

```
capacity ≈ (16 KB − header) / sum(sizeof(component_i))
```

A tight archetype with a few small components holds hundreds of entities. A fat archetype with many large components holds a handful. Chunks with low occupancy waste the bytes that could have held more entities.

---

2. TypeManager — the component registry

`TypeManager` registers every component, buffer, and system at world startup and assigns each a stable **type index**. From the registry, the runtime reads:

Field	What it's used for
`TypeIndex`	Packed identifier used in queries and chunk metadata.
`SizeInChunk`	Bytes per entity for this component in a chunk.
`Alignment`	Column alignment within the chunk.
`ElementSize`	Buffer element size (for `IBufferElementData`).
`HasBlobReferences` / `HasEntityReferences`	Enables validation during serialization and move operations.

You rarely touch `TypeManager` directly. What matters is that **adding new component types, renaming them, or moving them between assemblies triggers a type registry rebuild**, which is where startup time is spent in large projects.

> Entities 1.4 reported `TypeManager.Initialize` becoming ~2× faster in large projects by skipping unnecessary assembly scanning. On 6.5 that optimization is inherited.

---

3. Inspecting chunk layout — the Archetypes window

Window → **Entities** → **Archetypes** lists every archetype in the world. For each row:

- **Entities** — live count.
- **Chunks** — number of chunks in use.
- **Entities / Chunk** — average occupancy.
- **Capacity** — theoretical max per chunk.

What to watch for:

Reading	Interpretation
Many archetypes with 1 entity / chunk	Archetype explosion from over-granular tags. Consolidate.
One archetype with a huge entity count and high occupancy	Healthy — enemies, projectiles, bullets.
Entities / Chunk ≪ Capacity	Fragmentation — occupation is spotty because entities keep leaving for other archetypes (structural churn).
`Capacity = 1`	The archetype is too wide (components too big). Check for oversized managed/shared components.

---

4. Designing archetype-friendly components

Heuristics, in rough order of impact:

1. **Keep unmanaged structs unmanaged.** A single managed `IComponentData` forces side-table storage per entity and breaks Burst.
2. **Don't pad your structs.** Put `float3` before `float`, not after — alignment holes inflate `SizeInChunk`.
3. **Split cold fields off.** If only one system reads a field, move it to a sibling component. Unused columns still cost bytes per entity.

4. **Prefer enableable components over per-frame add/remove.** See `13_Structural Change & Safety.md` — toggling a bit beats reshaping the archetype.
5. **Limit shared component value sets.** Each new value spawns a new chunk bucket.
6. **Tag components cost zero bytes but still add an archetype.** Don't tag every entity with a unique marker.

---

5. Chunk version numbers

Each component column tracks a **change version** — a 32-bit counter bumped whenever any system writes through `RefRW<T>.ValueRW`, `SetComponentData`, or ECB `SetComponent`.

Systems can skip chunks whose version hasn't changed since last run:

```
var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>());
query.SetChangedVersionFilter(typeof(Health));
```

When a query is iterated with a change filter, only chunks bumped since the system last ran are visited. This is how patterns like "on Health change, update UI" become cheap.

Caveats:

- The filter is **chunk-level**, not entity-level. One entity's write dirties the whole chunk.
- Reading through `RefRW<T>.ValueRW` (even if you don't actually mutate) bumps the version.
- Toggling an enableable component also bumps the version.

---

6. BlobAsset storage

`BlobAssetReference<T>` holds large, read-only data (meshes, LOD tables, navigation fields) **outside** the chunk. The chunk only stores a pointer. This avoids bloating chunk capacity when many entities share the same baked data.

Practical tip: if you have a component dominated by a `FixedList` or a `DynamicBuffer` that's identical across all entities of a kind, store the big payload in a blob asset referenced from a lean component instead.

---

7. Examples

7.1 A fat archetype to avoid

```
public struct EnemyState : IComponentData
{
    public float3 Position;
    public quaternion Rotation;
    public float Health;
    public float HealthMax;
    public float Stamina;
    public float StaminaMax;
    public int AIState;
    public float3 LastKnownPlayerPos;
    public float TargetDistance;
    public Entity Target;
    public FixedList512Bytes<float3> PathBuffer;   // 512 bytes per entity!
}
```

One field `PathBuffer` alone eats 512 bytes per entity. Capacity per 16 KB chunk drops into low double digits. Split it:

```
public struct EnemyStats : IComponentData
{
    public float Health, HealthMax, Stamina, StaminaMax;
    public int   AIState;
    public Entity Target;
}

public struct EnemyPathPoint : IBufferElementData
{
    public float3 Value;
}
```

Now the dynamic buffer lives in its own storage and chunk density recovers.

7.2 Change filter for cheap UI updates

```
var query = state.GetEntityQuery(ComponentType.ReadWrite<Health>(),
                                 ComponentType.ReadWrite<HealthBarLink>());
query.SetChangedVersionFilter(typeof(Health));

state.Dependency = new UpdateHealthBarJob { /* ... */ }
    .ScheduleParallel(query, state.Dependency);
```

Only chunks where `Health` changed are visited.

---

8. Troubleshooting

Symptom	Cause / Fix
Slow startup in large project	`TypeManager.Initialize` scanning too many assemblies. Check the Profiler's startup trace; make sure you're on a recent Entities version.
Capacity of 1 for an archetype	Component is too big (buffer capacity, embedded blob). Move bulk data to a `BlobAssetReference<T>` or `IBufferElementData`.
Change filter fires every frame even when "nothing changed"	Someone's reading via `RefRW<T>.ValueRW` without writing. Switch to `RefRO<T>` where possible.
Archetype count keeps growing	Per-entity tags with high cardinality, or shared components keyed on continuous values. Consolidate.
Entities / Chunk much less than Capacity	Structural churn — adds/removes are scattering entities across archetypes. Switch to enableable components.
Managed component dominates CPU profile	Move the managed field to `UnityObjectRef<T>` or split the component; unmanaged paths are much faster.

Chapter 29. Systems · Entity Inspector · Query Window

1. Overview

The three Entities tooling windows you'll live in:

Window	Opens via	Answers
Entities Hierarchy	`Window → Entities → Hierarchy`	"What entities exist? What components do they carry?"
Systems	`Window → Entities → Systems`	"What systems are running, in what order, and how long do they take?"
Query	`Window → Entities → Query`	"Given a component set, which entities match? Which systems query it?"

A fourth, **Archetypes** (`Window → Entities → Archetypes`), is covered in `01_Chunk Layout & TypeManager.md`.

---

2. Entities Hierarchy

Acts as the live Hierarchy but for entities rather than GameObjects.

World picker (top of window)

Switch between `DefaultGameObjectInjectionWorld` (single-player), `ClientWorld`/`ServerWorld` (Netcode), or a test fixture world. Changing the world re-runs the tree against that world's entities.

Tree view

- Each row is an entity; child rows are entities linked via `Parent` / `LinkedEntityGroup`.
- Right-column counts: number of components, number of children.
- Click a row → its components appear in the **Inspector**.

Search

- `c:Health` — filter to entities with a `Health` component.
- `n:Enemy` — name filter.

- `w:Server` — world filter.
- Combine: `c:Health c:Enemy n:boss`.

Inspector

The Inspector for an entity shows:

- **Components** section — each `IComponentData` + its current values. Editable live in Play mode.
- **Relationships** — links to other entities (Parent, Child, LinkedEntityGroup).
- **Enabled state** — per `IEnableableComponent` toggle.

Live editing is **write-through**: changing a field immediately bumps that chunk's change version.

---

3. Systems

The Systems window lists every registered system, grouped by its `SystemGroup`, in the order the scheduler actually ticks them.

Columns to read first

Column	Meaning
Name	The system type.
Namespace	Where it lives (often identifies a package vs your code).
Time (ms)	Main-thread time spent in `OnUpdate`. Updates ~every frame.
Queries	Number of queries the system has touched.
Entity Count	Entities matched by the primary query.

What to look for

Reading	Interpretation
A system shows 0 ms but is green	Skipped by `RequireForUpdate<T>` or `[RequireMatchingQueriesForUpdate]`. Healthy.
A system shows consistent high ms	Actual work. Drill into the Profiler for a breakdown.
A system shows `--` for entity count	No matching query (or the query hasn't been built yet).
Two systems appear out of the expected order	`[UpdateBefore]` / `[UpdateAfter]` chain is wrong, or they're in different groups.

Enable / disable

Each system has a checkbox — untick to skip it in Play mode without recompiling. Great for A/B'ing.

System detail pane

Selecting a system opens a side pane with:

- **Queries** — each query's component set (All/Any/None/Disabled/Present/Absent).
- **Dependencies** — systems this one `[UpdateBefore]`/`[UpdateAfter]`, resolved to the actual schedule.
- **Type Info** — whether `[BurstCompile]` is on the class and on each method.

The **Queries tab now shows Disabled / Present / Absent / None component states** in 1.4+ / 6.x — so enableable queries no longer look ambiguous.

---

4. Query window

Purpose-built for "who matches X?" questions.

Workflow:

1. Add `All` components via the plus button.
2. Add `Any` / `None` if needed.
3. The window lists **matching entities** in the left panel and **systems that run this query** in the right panel.

Use cases:

- "Which systems read `Health`?" → add `Health` to All; every system listed on the right touches it.
- "Why is this entity not matching my system's query?" → paste the system's query, compare component presence against the entity in the Hierarchy window.
- "Are any prefabs slipping into this query?" → prefabs are distinguished by icon in the result list (added in 1.4+).

The Query window also lets you save a query for reuse during the session.

---

5. Journaling — the last-resort diagnostic

`Window → Analysis → Entities Journaling` (enabled via Project Settings → Entities → Journaling) records every structural change, component write, and system update into a ring buffer. Playback views:

- **Operations** — each recorded change with timestamp, source system, and affected entity.
- **Entities** — per-entity timeline of who touched it and when.

Journaling has runtime cost — turn it on only while investigating a bug, off in normal development.

---

6. Field workflow — debugging a missing update

Common bug: "My system processed the entity but the component didn't change." Walkthrough:

1. **Entities Hierarchy** → find the entity, confirm it has the component.
2. **Inspector** → before Play, note the value. Press Play and watch it.
3. If the value doesn't change: **Systems window** → find the relevant system, check **Time (ms)**. If 0, it's skipped.
4. If time > 0 but value still doesn't change: **Query window** → paste the system's query + the entity's components. Verify the entity matches.
5. Still stuck: enable **Journaling** for 1-2 frames. Look for writes to that component from other systems that stomp yours.

---

7. Troubleshooting

Symptom	Cause / Fix
Entities Hierarchy is empty in Play mode	World picker set to the wrong world (e.g. Client only, no Server).
A live field doesn't update	You're viewing a cached entity after a domain reload. Reselect it.
Systems window says 0 ms but my system definitely runs	Profiler may show it under a chunk job's marker if the actual work is in `ScheduleParallel`. Main-thread `OnUpdate` is genuinely 0 in that case — look at job timings in the Profiler.
Query window finds an entity my system doesn't	The system adds a `WithAll<T>()` or change filter that the Query window wasn't told about. Match them exactly.
Journaling window is empty	Journaling isn't enabled in Project Settings, or the buffer rolled over.

Symptom	Cause / Fix
Entities and GameObjects look out of sync	SubScene is open (white icon) — close it (yellow icon) so the baker runs and entities appear.

Chapter 30. Profiler · Bottleneck Analysis

1. Overview

The Unity Profiler (`Window → Analysis → Profiler`) is the primary tool for "why is this frame slow" questions in DOTS. Entities emits enough markers that you can attribute time to specific systems, jobs, and chunks — once you know where to look.

This page walks through reading the profile, the typical DOTS bottleneck patterns, and how to act on them.

---

2. Before you profile

1. **Turn on Deep Profile sparingly.** It is useful for tracking down a specific managed path but distorts timings — never trust Deep Profile for perf numbers.
2. **Enable "Record in Editor & Standalone"** so you capture builds, not just Editor frames.
3. **Set the Profiler to target the player**, not the Editor, when measuring released performance. Editor overhead is significant.
4. **Play for a few seconds before reading**; the first few frames include domain reload and JIT work.

---

3. Reading the CPU Usage module

Top-level markers under a DOTS frame:

```
PlayerLoop
├── Initialization.*
├── Update.ScriptRunBehaviourUpdate              (MonoBehaviour.Update)
├── SimulationSystemGroup                        ← DOTS simulation lives here
│   ├── SystemA.OnUpdate
│   ├── SystemB.OnUpdate
│   └── EndSimulationEntityCommandBufferSystem.Playback
├── FixedStepSimulationSystemGroup
├── PresentationSystemGroup
├── Render
└── WaitForTargetFPS
```

Drill into a `SystemA.OnUpdate` marker to see its subcalls — `ScheduleParallel`, job completion waits, `EntityManager` calls.

---

4. Where the time actually goes — common shapes

4.1 Main-thread `OnUpdate` dominant

If a single `SystemX.OnUpdate` accounts for 10+ ms, the work is running on the main thread. Check:

- Is it using `SystemAPI.Query` in a tight loop that should have been `IJobEntity.ScheduleParallel`?
- Is it a `SystemBase` forced to the main thread by a managed field?
- Is `EntityManager.AddComponent/RemoveComponent` being called per entity?

4.2 Job timings

Jobs show up in the **Timeline** view (or under "Workers" in the Hierarchy view). Look for:

- A job lane fully saturated while others are idle → work isn't spreading across chunks.
- Main-thread waiting on a job (`JobHandle.Complete()` marker) → `.Complete()` called too early; move it later in the frame.
- Huge Burst function names → normal, the symbol is the full generic instantiation.

4.3 `EntityManager` hotspots

Look for markers named:

- `EntityManager.SetArchetype` / `Move Entity` — structural change churn. See `13_Structural Change & Safety.md`.
- `EntityCommandBufferSystem.OnUpdate (Playback)` — expensive ECB playback. Usually means too many commands recorded per frame.
- `TypeManager.Initialize` — only at startup; if it appears per-frame, something is reloading the world.

4.4 `GC.Alloc` markers

Any `GC.Alloc` in the simulation is a regression in DOTS. Sources:

- A `SystemBase` allocating in its `OnUpdate`.

- A `foreach` over a managed collection inside a system.
- Boxing — e.g. `object.ToString()` on a struct.

Burst systems should show zero allocations in the Profiler's Memory module.

---

5. Profiler modules to pin

- **CPU Usage** — timeline and hierarchy of markers.
- **Memory** — allocated heap; watch for climbing `GC.Used Memory`.
- **Entities** (if available) — a DOTS-specific module showing per-system timings over time.

The Entities Profiler module arrived with 1.8.0 / 6.x and aligns with the Systems window on a timeline. If you see multi-frame spikes, this is the fastest way to attribute them.

For networked projects, pair this with `../DOTS Workflows/24_Netcode Profiler & Debugging.md` to inspect Ghost snapshots, prediction ticks, and Netcode-specific bandwidth.

---

6. Common bottlenecks and fixes

Pattern	Symptom in profiler	Fix
Per-entity `Entities.ForEach` (legacy)	Main-thread-heavy `OnUpdate` with many small samples	Port to `IJobEntity.ScheduleParallel`.
Per-frame `AddComponent/RemoveComponent`	`Move Entity` marker dominates	Convert state to an enableable component.
ECB overload	`...ECBS.Playback` taking ms	Batch commands (one per chunk instead of per entity), or raise the batch size.
Shared-component value explosion	`SetSharedComponent` dominates	Bucket the value (quantize to fewer distinct keys).
`.Complete()` called mid-frame	Long `JobHandle.Complete` wait on main thread	Postpone the completion to the end of the frame, or restructure so the main thread doesn't need the result yet.
Burst fallback	Hot method shows unmangled C# name	A captured non-blittable type fell back to IL. Check the Burst Inspector.
Archetype fragmentation	Many small chunks; `EntityManager.*Query` markers high	Archetypes too granular — consolidate tags / use enableable components.

Pattern	Symptom in profiler	Fix
Bake-time regression	Bake markers during Play	Domain reload re-triggered baking. Usually Editor-only; confirm in a build.

---

7. Frame budget worked example

Imagine a 16.6 ms budget (60 Hz). After ruling out Render:

- `SimulationSystemGroup`: 6 ms
  - `EnemyAISystem.OnUpdate`: 2.1 ms (Burst, parallel jobs on 4 workers)
  - `MovementSystem.OnUpdate`: 1.4 ms (Burst, parallel)
  - `EndSimulationECBS.Playback`: 1.8 ms ← suspicious
  - everything else: 0.7 ms

`1.8 ms` of ECB playback for what should be a small number of structural changes is usually the biggest lever. Check command counts:

1. Entities Journaling for 1 frame → counts of `Instantiate/AddComponent/DestroyEntity` per system.
2. If a system records hundreds of commands every frame, batch them (one command per chunk, or switch to a parallel recorder with chunk-index sort keys).

---

8. Standalone Profiling

For a real build:

1. **File → Build Profiles → Development Build + Autoconnect Profiler**.
2. Player runs, Profiler captures from the remote process.
3. Results are more honest about what your players will see. Editor measurements can be 2-5× slower for reasons unrelated to your code.

---

9. Troubleshooting

Symptom	Cause / Fix
Profile shows no DOTS markers	Profiler module is filtering them out — enable "Scripts" and "Jobs" in the module settings.
Frame is slow but no hot marker	Could be render-thread wait. Enable the GPU module and the timeline view.
Deep Profile shows different hot spots	Deep Profile distorts allocations and call counts. Trust the non-deep profile for relative timing.
Burst method shows managed call in stack	Fallback — Burst couldn't compile it. Check the Burst Inspector output.
`GC.Collect` marker spikes	Managed allocations somewhere in a system. Narrow down with Memory module → Snapshot.
System time oscillates wildly between frames	Work grows with entity count and chunks are entering/leaving the query each frame. Check structural churn.

Chapter 31. Managed Object Reference Audit

1. Overview

Managed Unity object references are one of the easiest ways to accidentally pull an ECS system out of the fast path. This audit helps find places where component data, systems, or rendering glue hold `UnityEngine.Object` references in ways that block Burst, jobs, or clean SubScene baking.

Use this after the core ECS migration, or whenever a profiler trace shows managed work where you expected data-oriented code.

---

2. What to search for

Pattern	Why it matters
`class .*: IComponentData`	Managed component data; valid but usually not ideal for hot paths.
`UnityEngine.Object`	Broad marker for managed Unity object references.
`GameObject`, `Transform`, `Component`	Runtime scene-object references usually do not belong in simulation data.
`Mesh`, `Material`, `Texture`, `AudioClip`	Often should be `UnityObjectRef<T>` or presentation-side cached data.
`Resources.Load`	Runtime asset lookup; often better handled by baking/content systems.
`ScriptableObject`	Useful authoring source, but runtime jobs should read baked values or blobs.

Keep the audit scoped to ECS-facing code. MonoBehaviour-only UI or editor tooling does not need to be forced into DOTS patterns.

---

3. Triage rules

Finding	Likely action
Managed component in a hot simulation query	Convert to unmanaged `IComponentData`, `BlobAssetReference<T>`, or `UnityObjectRef<T>`.

Finding	Likely action
Managed component used only by presentation	Keep it if it is isolated from Burst/job simulation paths.
ScriptableObject read every frame	Bake the needed values into a component or blob asset.
Mesh/material reference in ECS data	Use `UnityObjectRef<Mesh>` / `UnityObjectRef<Material>` and dereference in presentation code.
GameObject/Transform stored in component data	Replace with ECS data or move the reference to a hybrid presentation bridge.

---

4. Audit example

```
// Suspicious: managed component data used as per-entity visual state.
public class EnemyVisual : IComponentData
{
    public Material Material;
}
```

Prefer:

```
using Unity.Entities;
using UnityEngine;

public struct EnemyVisual : IComponentData
{
    public UnityObjectRef<Material> Material;
}
```

Then dereference in a main-thread presentation system or cache the resolved material in a renderer registry.

---

5. Profiler signs

Profiler symptom	What to check
Simulation system allocates every frame	Managed object lookup or LINQ/collection allocation in `OnUpdate`.
Burst is disabled for a system that should be data-only	Managed component or managed API access in the query path.
Main thread has many tiny rendering lookups	Cache `UnityObjectRef<T>.Value` results outside the per-entity loop.

Profiler symptom	What to check
SubScene rebakes more often than expected	Authoring scripts or object references are creating unnecessary dependencies.

---

6. Validation

1. Re-run the search patterns in §2.
2. Confirm hot simulation systems query only unmanaged components.
3. Check Burst Inspector for systems/jobs expected to compile with Burst.
4. Profile a representative scene and confirm managed allocations are gone from the hot path.
5. Enter Play Mode from a clean Editor launch to verify SubScene baking still resolves the references.

---

7. Troubleshooting

Symptom	Cause / Fix
Converted component still cannot be queried in Burst	A remaining field is managed. Check nested structs too.
`UnityObjectRef<T>.Value` returns null in Play Mode	The referenced asset was not baked or loaded. Check the Baker and SubScene/content workflow.
Visual system is still slow after conversion	You are dereferencing per entity every frame. Cache by shared asset or bake a compact lookup key.
ScriptableObject changes do not affect runtime data	The values were baked. Reimport/rebake the SubScene or add proper baking dependencies.
Editor-only references fail in player builds	Move editor APIs behind `#if UNITY_EDITOR` and bake runtime-safe data.

Part IV. Migration

Chapter 32. Entities 1.x → 6.5 Overview

1. What you're signing up for

Upgrading from Entities 1.x to 6.5 is mainly **three DOTS migrations** on one package-distribution transition:

1. **Editor/package distribution upgrade.** You move to Unity 6000.4+ (`com.unity.entities` 6.4) or 6000.5+ (Entities 6.5), where Entities is a Core Package embedded in the Editor.
2. **ECS data cleanup.** Review managed Unity object references in ECS-facing data and migrate hot-path component data toward unmanaged components, baked values, `BlobAssetReference<T>`, or `UnityObjectRef<T>`.
3. **ECS API migration.** Legacy APIs marked obsolete in Entities 1.4 — `Entities.ForEach`, `IAspect` — are still obsolete on 6.5. Also: `ComponentLookup.GetRefRWOptional` → `TryGetRefRW`.

Each migration is done in its own sub-document in this folder. This page is the **map** — when to do which, in what order.

> **Naming watch-out:** The Entities **package** version (1.4 / 6.4 / 6.5) is distinct from the Unity Editor version (6000.3 / 6000.4 / 6000.5). They align by convention from 6.4 onward but are tracked in separate release notes. This document treats them as two axes; the Changelog page (`../Changelog/Entities 1.4 → 6.5 Key Changes.md`) does the same.

2. Recommended order

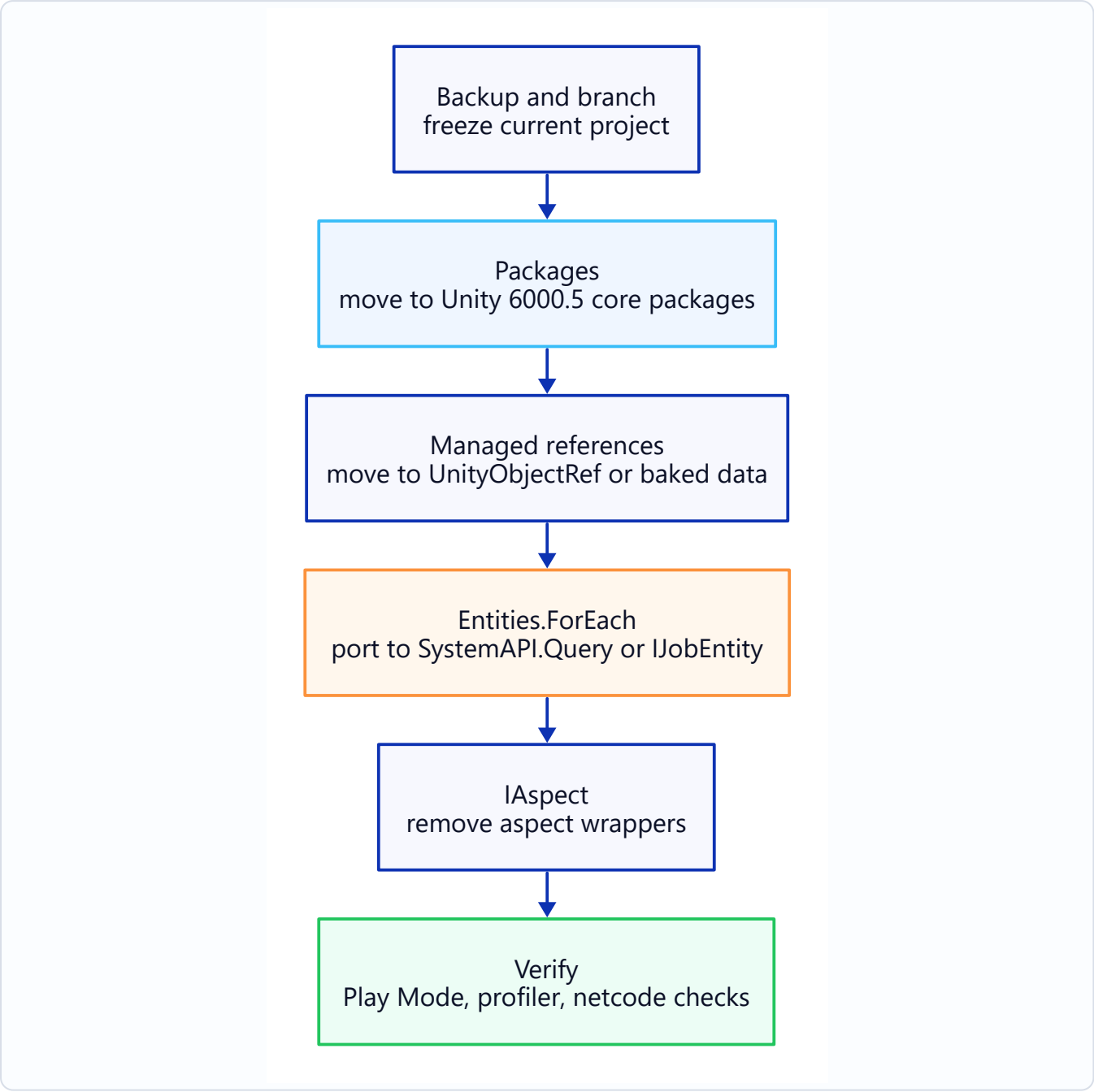

Figure: Entities 1.x to 6.5 Migration Order.

Why this order:

- **Editor first** because nothing else compiles against 6.x APIs until the Editor knows about them.
- **Package Manager → Core Package next** — without this, you have duplicate / conflicting package resolution.
- **Managed reference cleanup before iteration refactors** — it keeps the data model clear before rewriting systems and jobs around it.
- **foreach / IAspect last** — these are ECS-internal; they don't block the rest of the codebase.

3. What stays the same

A surprising amount — which makes the migration less scary than it looks.

Unchanged	Notes
`IComponentData`, `IBufferElementData`, `ISharedComponentData`	Same interfaces, same semantics.
`ISystem` / `SystemBase`	Same lifecycle hooks, same attributes.
`EntityManager` API surface	`CreateEntity`, `AddComponent`, etc. — identical.
`SystemAPI.Query`	Introduced in 1.x and stays the central iteration API.
`IJobEntity`, `IJobChunk`	Same shape; `IJobEntity` is the recommended default.
`EntityCommandBuffer` + ECBS	Same.
Burst, Jobs, Collections, Mathematics	Same.
Netcode for Entities concepts (Ghost, RPC, Prediction)	Same architecture; the package just distributes differently.

If your 1.x code already uses `IJobEntity` and `SystemAPI.Query` (not legacy `Entities.ForEach`), steps 4 and 5 of the migration mostly disappear for you.

---

4. What changes, at a glance

Area	1.x	6.5
Distribution	`com.unity.entities` UPM package	Core Package (no manifest entry)
Version number	1.4.x	6.5.x (aligns with Unity 6000.5)
Managed Unity object references in ECS data	Often stored in managed components	Prefer baked data, blobs, or `UnityObjectRef<T>` where appropriate
Component iteration (legacy)	`Entities.ForEach` marked obsolete in 1.4	Still obsolete, still compiles with warnings; removal planned for a future major release
Aspects	`IAspect` marked obsolete in 1.4	Still obsolete, still compiles; removal planned for a future major release
Optional refs	`GetRefRWOptional` / `GetRefROOptional` deprecated	`TryGetRefRW` / `TryGetRefRO`

Full changelog for the period is in `../Changelog/Entities 1.4 → 6.5 Key Changes.md`.

---

5. Backup strategy

Non-negotiable before starting:

1. **Branch in git.** Name it descriptively (`migrate/entities-6.5`). Do not migrate on `main`.
2. **Commit `manifest.json` and `packages-lock.json` in their pre-migration state.** They are the source of truth for reverting if resolution breaks.
3. **Tag the pre-migration commit.** `git tag pre-entities-6.5` makes reverts trivial.
4. **Keep the project open in the old Editor version until you're ready.** Opening in 6000.5 once changes the `ProjectVersion.txt` and forces the migration path.

---

6. Per-step checklists

6.1 Editor upgrade

- Install Unity 6000.5.x via Hub.
- Do **not** yet open the project. Commit any pending work first.
- Open the project in the new Editor — it will offer to upgrade `ProjectVersion.txt`. Accept.
- Let the initial import finish; expect warnings about deprecated APIs and missing packages. That's what the next steps fix.

6.2 Package cleanup → `02_Package Manager → Core Package.md`

- Remove `com.unity.entities`, `com.unity.collections`, `com.unity.mathematics`, `com.unity.entities.graphics` from `manifest.json`.
- Delete `packages-lock.json` and let the Editor regenerate.
- Also remove Netcode for Entities if your project uses it (6.5 is a Core Package version).

6.3 Managed object references → `03_Managed Object References → UnityObjectRef.md`

- Audit ECS-facing code for managed Unity object fields.
- Convert hot-path component data to unmanaged ECS data where possible.

- Use `UnityObjectRef<T>` for ECS-side references to Unity assets/objects.
- Use `BlobAssetReference<T>` for large immutable gameplay data.

6.4 Legacy ForEach → `04_foreach → IJobEntity.md`

- For systems that still use `Entities.ForEach` / `Job.WithCode` — port to `IJobEntity` + `SystemAPI.Query`.

6.5 Aspect removal → `05_IAspect Removal.md`

- Inline aspect methods into the call site using `RefRW<T>` / `RefRO<T>` parameters.
- Delete the `IAspect` struct.

---

7. Testing after migration

Smoke tests to run before declaring the migration done:

1. **Play mode on a representative SubScene.** Entities exist, systems tick, nothing throws on start.
2. **Build a standalone player.** Build-only breakages (usually serialization) appear here.
3. **Run your existing gameplay regression tests.** If performance matters, capture a baseline Profiler trace from before migration and diff.
4. **Check save/load.** Raw `Entity` handles are world-local and not stable. Save game-owned IDs, content keys, or authored data instead.
5. **Run on a target platform.** Some platform-specific backends (IL2CPP quirks, AOT) shake out here.

---

8. When to roll back

Clean rollback triggers:

- Editor won't open the project at all (rare; usually `ProjectVersion.txt` mismatch).
- `packages-lock.json` resolution produces a nonsense graph and deletion doesn't fix it.
- A critical package you depend on (third-party) isn't yet compatible with 6.5.

Plan: `git checkout pre-entities-6.5`, reinstall the older Unity version, verify the game still runs on the prior version.

Migrations that have partially completed but are stuck should be abandoned cleanly and restarted from the tag — do not ship half-migrated code.

---

9. Troubleshooting

Symptom	Cause / Fix
Editor won't open — "Project was saved in a newer version"	Someone opened it in 6000.5 previously; revert `ProjectSettings/ProjectVersion.txt` to the older string.
After manifest cleanup, compile error "Unity.Entities assembly not found"	The Editor is older than 6000.4 — Core Package isn't available. Upgrade the Editor or add the UPM package back.
Hundreds of deprecation warnings	Expected. Treat them as the migration punch list — each points at something to fix.
Build succeeds, game crashes on start	Usually an ECB or serialization issue introduced by removing a component. Diff the baked EntityScene asset size; huge reductions suggest a component was silently dropped.
Performance regresses after migration	Usually a `SystemBase` was introduced that shouldn't have been (managed field), or an `IJobEntity` fell back from Burst. Check the Profiler and Burst Inspector.

Chapter 33. Package Manager → Core Package

1. What changes

On Unity 6000.4+, Entities (and its companion DOTS packages) become **Core Packages**: they install from the Editor itself instead of downloading from a remote registry, and their version is fixed by the Editor. You **still add them through the Package Manager**, and they stay listed as dependencies in your project — being a Core Package changes *where the package comes from and who picks its version*, not whether your project has to request it. What changes when you migrate a 1.x project:

- **`Packages/manifest.json`** — the `com.unity.entities*` entries **stay**, but their pinned **1.x** versions are replaced by the Editor's Core versions (re-add them from the Package Manager).
- **`Packages/packages-lock.json`** — the affected packages now resolve with `"source": "builtin"`.
- **Package Manager window** — the packages still appear under "In Project"; you can no longer pick their version (it follows the Editor).

Your C# code does **not** change. Assembly names (`Unity.Entities`, `Unity.Collections`, etc.) are the same; `.asmdef` references are unchanged.

---

2. Which packages move

Moved to **Core Package** status on 6000.4+:

UPM id (before)	After
`com.unity.entities`	Core Package, ships with Editor
`com.unity.collections`	Core Package
`com.unity.mathematics`	Core Package
`com.unity.entities.graphics`	Core Package
`com.unity.netcode` (Netcode for Entities)	Core Package in 6.5

Stay **on Package Manager**:

- `com.unity.physics` (Unity Physics)

- `com.unity.charactercontroller` (Character Controller)

Built into the engine (unchanged):

- Jobs system
- Burst (`com.unity.burst` is still exposed as a UPM entry in some setups, but on 6000.4+ it's tied to the Editor version; safest to leave manifest as-is).

---

3. Editing `manifest.json`

Open `Packages/manifest.json`. It looks something like:

```
{
  "dependencies": {
    "com.unity.entities": "1.4.2",
    "com.unity.collections": "2.4.5",
    "com.unity.mathematics": "1.3.2",
    "com.unity.entities.graphics": "1.4.2",
    "com.unity.netcode": "1.12.0",
    "com.unity.physics": "1.3.14",
    "com.unity.render-pipelines.universal": "17.0.4",
    ...
  }
}
```

On a 6000.4+ Editor those pinned **1.x** versions no longer resolve — the registry no longer serves them for this Editor. Do **not** simply delete the lines (that removes the dependency, and a Core Package is *not* auto-added back). Instead, point each Core Package at the Editor's version. Two ways, both of which keep the packages as dependencies:

Recommended — re-add through the Package Manager. For each Core Package, remove its stale `1.x` pin, then add it back via **Window** → **Package Manager** → **Unity Registry** → **Install** (or **+** → **Add package by name...**, e.g. `com.unity.entities`). The Package Manager writes the Editor's Core version into `manifest.json` for you, and pulls in the dependent Core Packages automatically.

By hand. Replace each Core Package's `1.x` version with the Editor's Core version (for example `6.5.0` on Unity 6000.5):

```
 {
   "dependencies": {
-    "com.unity.entities": "1.4.2",
+    "com.unity.entities": "6.5.0",
-    "com.unity.entities.graphics": "1.4.2",
+    "com.unity.entities.graphics": "6.5.0",
-    "com.unity.netcode": "1.12.0",
+    "com.unity.netcode": "6.5.0",
     "com.unity.physics": "1.3.14",
```

```
    "com.unity.render-pipelines.universal": "17.0.4",
    ...
  }
}
```

`com.unity.collections` and `com.unity.mathematics` come in as dependencies of Entities, so you usually don't need to list them yourself. Leave `com.unity.physics` and anything else that is still a registry (UPM) package as-is.

Save the file.

---

4. Rebuilding `packages-lock.json`

Two options:

4.1 Let the Editor regenerate it

Close the Editor, delete `Packages/packages-lock.json`, reopen. Unity regenerates it. This is the cleanest option.

4.2 Edit in place

Open `packages-lock.json` and update the stale entries for the packages above (delete them and let the Editor re-resolve). The Editor reconciles on next open. Slightly faster if you have a complex lock file, but more error-prone.

Both result in the final lock file containing entries like:

```
"com.unity.entities": {
  "version": "6.5.0",
  "depth": 0,
  "source": "builtin",
  "dependencies": {}
}
```

Note that `"source": "builtin"` is the tell that the package is now a Core Package (supplied by the Editor), while `"depth": 0` shows it is still a **direct dependency** declared in `manifest.json` — the entry did not go away.

---

5. Verifying the result

1. **Package Manager window** (`Window → Package Manager` → "In Project") — Entities, Collections, Mathematics, Entities Graphics (and Netcode for Entities if you use it) should be listed, with their version fixed by the Editor (no version dropdown).
2. **Editor menus** — `Window → Entities → Hierarchy/Systems/Archetypes` exist and work.
3. **Compile** — press Play; the project should enter Play mode with no compile errors referencing missing assemblies.

---

6. Troubleshooting

Symptom	Cause / Fix
`Unable to find package 'com.unity.entities@1.x.y'` after upgrading the Editor	The manifest still pins a 1.x version. Update it to the Core version (re-add via the Package Manager), then delete `packages-lock.json` and let the Editor regenerate.
`The type or namespace Unity.Entities could not be found`	Entities isn't added to the project (add it via the Package Manager), or you're on an Editor older than 6000.4 (upgrade it).
Package Manager shows a stale or conflicting Entities version	A leftover `1.x` pin is still in `manifest.json`. Update it to the Core version (re-add via the Package Manager) and regenerate `packages-lock.json`.
Third-party asset won't compile because it pins `com.unity.entities@1.x`	The asset needs updating. Either wait, fork, or stay on the 1.x line for this project.
Unity Physics breaks because it wanted a specific Entities version	Upgrade Unity Physics to a version compatible with Entities 6.5. Check the package's changelog on docs.unity3d.com.
`packages-lock.json` regeneration fails	Network issue or a pinned private registry. Check Editor logs for the specific resolution failure.
Netcode for Entities still resolves to an old `1.x` version	On 6000.5+, Netcode is a 6.5 Core Package. Update `com.unity.netcode` to the 6.5 version (re-add via the Package Manager) and regenerate.

Chapter 34. Managed Object References → UnityObjectRef

1. Overview

When migrating older ECS code, a common problem is managed object references leaking into runtime component data. In Entities 6.5, keep gameplay components unmanaged where possible and use `UnityObjectRef<T>` when ECS data must point at a Unity object such as a mesh, material, texture, audio clip, or authored ScriptableObject.

This page is scoped to DOTS component design. It does not cover engine-wide Unity object identifier APIs.

---

2. What to look for

Search for components that store managed Unity objects directly:

Pattern	Why to review it
`class` components implementing `IComponentData`	Managed components are allowed but slower and main-thread constrained.
`UnityEngine.Object` fields inside component data	Usually indicates a managed component or invalid unmanaged component design.
`GameObject`, `Transform`, `Material`, `Mesh`, `Texture`, `AudioClip` fields	Decide whether this belongs in authoring, presentation, or `UnityObjectRef<T>`.
Runtime systems calling `Resources.Load` per entity	Move asset resolution to baking or a central content system.

---

3. Before and after

Before — managed component

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisual : IComponentData
```

```
{
    public Mesh Mesh;
    public Material Material;
}
```

This works, but it makes the component managed. Managed components reduce Burst/job flexibility and are usually a poor fit for per-entity gameplay data.

After — unmanaged component with `UnityObjectRef<T>`

```
using Unity.Entities;
using UnityEngine;

public struct ProjectileVisual : IComponentData
{
    public UnityObjectRef<Mesh> Mesh;
    public UnityObjectRef<Material> Material;
}
```

Bake the references once:

```
using Unity.Entities;
using UnityEngine;

public class ProjectileVisualAuthoring : MonoBehaviour
{
    public Mesh Mesh;
    public Material Material;
}

public class ProjectileVisualBaker : Baker<ProjectileVisualAuthoring>
{
    public override void Bake(ProjectileVisualAuthoring authoring)
    {
        var entity = GetEntity(TransformUsageFlags.Dynamic);
        AddComponent(entity, new ProjectileVisual
        {
            Mesh = new UnityObjectRef<Mesh>(authoring.Mesh),
            Material = new UnityObjectRef<Material>(authoring.Material)
        });
    }
}
```

---

4. Runtime access pattern

Keep simulation jobs purely data-oriented. Dereference Unity objects only where managed Unity APIs are allowed.

```
using Unity.Entities;
using UnityEngine;
```

```
public partial class ProjectileVisualUploadSystem : SystemBase
{
    protected override void OnUpdate()
    {
        foreach (var visual in SystemAPI.Query<RefRO<ProjectileVisual>>())
        {
            Mesh mesh = visual.ValueRO.Mesh.Value;
            Material material = visual.ValueRO.Material.Value;

            if (mesh == null || material == null)
                continue;

            // Upload to a rendering registry, material property block cache, etc.
        }
    }
}
```

For high-volume rendering, avoid per-frame dereferencing per entity. Cache resolved objects in a presentation-side registry keyed by a compact ECS value.

---

5. When not to use `UnityObjectRef<T>`

Case	Better choice
Pure gameplay data	Store primitive / math / blob data directly in `IComponentData`.
Large read-only tables	`BlobAssetReference<T>`.
Entity prefab baked into the same SubScene	`Entity` prefab reference.
Prefab streamed through content workflows	`EntityPrefabReference`.
Complex managed behavior per entity	Keep it outside the simulation path or isolate it in a managed presentation system.

---

6. Migration checklist

- [ ] Find managed component data used only to store Unity object references.
- [ ] Replace those fields with `UnityObjectRef<T>` where an unmanaged component is appropriate.
- [ ] Move object assignment into Bakers.
- [ ] Keep `.Value` dereferencing out of Burst jobs.
- [ ] Cache dereferenced objects when many entities share the same asset.
- [ ] Use `BlobAssetReference<T>` for large immutable gameplay data instead of ScriptableObject reads at runtime.

---

7. Troubleshooting

Symptom	Cause / Fix
Component cannot be used in a Burst job	It is still managed. Convert Unity object fields to `UnityObjectRef<T>` or split presentation data out.
`.Value` is null	The referenced asset is not loaded or was destroyed. Check the authoring reference and content loading path.
System allocates every frame	You are resolving or loading assets per entity. Bake references and cache resolved managed objects.
Gameplay system depends on a ScriptableObject at runtime	Bake the needed values into component data or a blob asset.
Build strips an asset	Ensure the asset is referenced through baking/content workflows, not only by a runtime string path.

Chapter 35. foreach → IJobEntity Migration

1. Status — obsolete, not yet removed

The legacy component-iteration APIs are **obsolete** on Entities 1.4+ / 6.5 but still compile:

Legacy	Status in 6.5	Replacement
`Entities.ForEach`	**Obsolete** since 1.4; deprecation warnings on every use; Unity's changelog says it will be removed in a future major release	`SystemAPI.Query<...>` (main thread) or `IJobEntity` (jobs)
`Job.WithCode`	**Obsolete** alongside ForEach	Plain `IJob` struct, or inline in `OnUpdate`
`IJobForEach` (pre-1.0)	Removed long before 1.4	`IJobEntity`

You're here either because you opened a 1.x project in 6000.5+ and the compiler is warning loudly, or because you want to clear these warnings before a future major package release removes the API. The refactor is mechanical once you see the pattern.

---

2. The two replacement APIs

- **`SystemAPI.Query<T, U, ...>`** — main-thread iteration inside `OnUpdate`. Simple, no job overhead, ideal for small workloads or when you need managed APIs.
- **`IJobEntity`** — source-generated parallel job. Drop-in if you're ready for multi-threaded work, which is where DOTS pays off.

Background: `../DOTS Workflows/11_IJobEntity · SystemAPI.Query.md`.

---

3. Port — `Entities.ForEach` → `SystemAPI.Query`

3.1 Before

```
public partial class HealthRegenSystem : SystemBase
{
    protected override void OnUpdate()
```

```
    {
        float dt = Time.DeltaTime;

        Entities
            .WithAll<Alive>()
            .WithNone<Dead>()
            .ForEach((ref Health health, in Regen regen) =>
            {
                health.Value = math.min(health.Value + regen.Rate * dt, health.Max);
            })
            .ScheduleParallel();
    }
}
```

3.2 After — `SystemAPI.Query` (main thread)

```
[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;

        foreach (var (health, regen) in
                 SystemAPI.Query<RefRW<Health>, RefRO<Regen>>()
                          .WithAll<Alive>()
                          .WithNone<Dead>())
        {
            health.ValueRW.Value = math.min(
                health.ValueRO.Value + regen.ValueRO.Rate * dt,
                health.ValueRO.Max);
        }
    }
}
```

Notes on the shape change:

- `SystemBase` → `ISystem` (cheaper, Burst-able). You may need to keep `SystemBase` if the old code captured managed fields.
- `ref Health` → `RefRW<Health>` tuple element. Read via `.ValueRO`, write via `.ValueRW`.
- `in Regen` → `RefRO<Regen>`.
- `Time.DeltaTime` → `SystemAPI.Time.DeltaTime`.
- Filters (`WithAll`/`WithNone`) move onto the `Query` builder.

---

4. Port — `Entities.ForEach.ScheduleParallel` → `IJobEntity`

When the ForEach was scheduled in parallel, the job form is a direct 1:1 port and keeps the parallelism.

4.1 After — `IJobEntity`

```
[BurstCompile]
public partial struct HealthRegenJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref Health health, in Regen regen)
    {
        health.Value = math.min(health.Value + regen.Rate * DeltaTime, health.Max);
    }
}

[BurstCompile]
public partial struct HealthRegenSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state)
    {
        new HealthRegenJob { DeltaTime = SystemAPI.Time.DeltaTime }
            .ScheduleParallel();
    }
}
```

Filters go on the job struct as attributes:

```
[WithAll(typeof(Alive))]
[WithNone(typeof(Dead))]
public partial struct HealthRegenJob : IJobEntity { ... }
```

Or pass a query at schedule time:

```
var q = SystemAPI.QueryBuilder()
    .WithAll<Health, Regen, Alive>()
    .WithNone<Dead>()
    .Build();
new HealthRegenJob { DeltaTime = dt }.ScheduleParallel(q);
```

---

5. Captures and their replacements

Legacy `Entities.ForEach` allowed capturing locals with `.WithReadOnly`, `.WithDisposeOnCompletion`, etc. In `IJobEntity`, captures are **fields on the job struct** — Burst knows exactly what's being passed.

```
// BEFORE
var buffer = new NativeArray<int>(count, Allocator.TempJob);
Entities.ForEach((ref Foo f) => { /* use buffer */ })
    .WithDisposeOnCompletion(buffer)
    .ScheduleParallel();

// AFTER
[BurstCompile]
partial struct FooJob : IJobEntity
{
    [ReadOnly] public NativeArray<int> Buffer;

    void Execute(ref Foo f) { /* use Buffer */ }
}

var buffer = new NativeArray<int>(count, Allocator.TempJob);
state.Dependency = new FooJob { Buffer = buffer }.ScheduleParallel(state.Dependency);
buffer.Dispose(state.Dependency);   // chained dispose replaces WithDisposeOnCompletion
```

Attribute replacements:

- `.WithReadOnly(x)` → `[ReadOnly] public NativeArray<T> X;`
- `.WithDisposeOnCompletion(x)` → `x.Dispose(handle)` after scheduling.
- `.WithoutBurst()` → `[WithoutBurst]` on the job struct (but strongly consider removing the reason you needed it).
- `.WithEntityAccess()` → add `Entity entity` parameter to `Execute`.

---

6. When a `SystemBase` is still required

If the original system had a genuine managed field (Input System, UI Toolkit, external service), you can't mechanically convert to `ISystem`. Two options:

1. **Keep it as `SystemBase`** and use `SystemAPI.Query` inside `OnUpdate`. Single-threaded.
2. **Split** into a `SystemBase` that bridges the managed world and an `ISystem` that does the Burst work — see `../DOTS Workflows/08_System — ISystem vs SystemBase.md`.

---

7. Validation

After porting a system:

1. **Unit-test the behaviour.** Port should not change logic; if output differs, the refactor introduced a bug.
2. **Profile.** Parallel ForEach and `IJobEntity.ScheduleParallel` should be within 5% of each other for equivalent work. A large regression means Burst isn't kicking in — inspect the Burst Inspector.
3. **Delete the old system file.** Resist keeping a `#if LEGACY` fallback — it rots.

---

8. Troubleshooting

Symptom	Cause / Fix
`warning: Entities.ForEach is obsolete`	Expected on 6.5 — the code still compiles. Port as above to clear the warning before a future major release removes it.
`error: 'SystemAPI' does not contain 'Query'`	System is not `partial`, or the struct doesn't implement `ISystem` correctly.
`IJobEntity` compiles but doesn't run	Missing `[BurstCompile]` on the struct and on `OnUpdate`; source generator doesn't schedule non-partial types.
Job reads stale values	Captured a local before `.Schedule*` was called. Fields must be set on the job struct before scheduling.
Perf regresses after port	Check for Burst fallback (Burst Inspector), or a managed type sneaked into a field.
Compile-time only issues after porting	Cross-check that you removed captured locals via attributes and not via lambda-closure syntax, which only applied to the ForEach API.

Chapter 36. IAspect Deprecation — Migrating Off Aspects

1. Status — deprecated, not yet removed

`IAspect` was marked **obsolete in Entities 1.4** and remains obsolete on the 6.x line. It still compiles, still runs, and is still listed as a valid `IJobEntity.Execute` parameter kind on the 6.5 API reference — but the compiler raises a deprecation warning, and Unity has stated the type will be **removed in a future major release**. New code should not use it; existing code should migrate at an opportunistic pace.

`IAspect` was introduced to let systems declare a "view" over a set of components and use it as a single argument in `Entities.ForEach` or `IJobEntity`. In practice it added a layer of source generation and a parallel type system that didn't pay for itself:

- Writing an aspect duplicated the component parameter list.
- Debuggers stepped through generated wrapper code.
- `RefRW<T>` / `RefRO<T>` on `IJobEntity` and `SystemAPI.Query` achieve the same "pass components as a bundle" ergonomics without the extra type.

Because the surface is still present, your code does not break on the Editor upgrade — this migration is about cleaning up warnings and preparing for 2.0.

---

2. Mechanical refactor

2.1 Starting point

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public readonly partial struct MovementAspect : IAspect
{
    public readonly RefRW<LocalTransform> Transform;
    public readonly RefRO<Velocity>       Velocity;
    public readonly RefRO<Speed>          Speed;

    public void Step(float dt)
    {
```

```
        Transform.ValueRW.Position += Velocity.ValueRO.Direction * Speed.ValueRO.Value * dt;
    }
}

public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var m in SystemAPI.Query<MovementAspect>())
            m.Step(dt);
    }
}
```

2.2 After — direct component parameters

```
using Unity.Entities;
using Unity.Mathematics;
using Unity.Transforms;

public partial struct MovementSystem : ISystem
{
    public void OnUpdate(ref SystemState state)
    {
        float dt = SystemAPI.Time.DeltaTime;
        foreach (var (transform, velocity, speed) in
                 SystemAPI.Query<RefRW<LocalTransform>, RefRO<Velocity>, RefRO<Speed>>())
        {
            transform.ValueRW.Position += velocity.ValueRO.Direction * speed.ValueRO.Value * dt;
        }
    }
}
```

Two changes:

1. The aspect type is gone. Its fields become the tuple elements of the `SystemAPI.Query`.
2. The method that lived on the aspect (`Step`) is inlined. If it's too big to inline, extract a **static helper** that takes the same parameters — that keeps it Burst-friendly:

```
static void Step(ref LocalTransform t, in Velocity v, in Speed s, float dt)
{
    t.Position += v.Direction * s.Value * dt;
}
```

Calling it:

```
Step(ref transform.ValueRW, velocity.ValueRO, speed.ValueRO, dt);
```

---

3. Aspects in `IJobEntity`

3.1 Before

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;
    void Execute(MovementAspect aspect) => aspect.Step(DeltaTime);
}
```

3.2 After — components as Execute parameters

```
[BurstCompile]
public partial struct MoveJob : IJobEntity
{
    public float DeltaTime;

    void Execute(ref LocalTransform transform, in Velocity velocity, in Speed speed)
    {
        transform.Position += velocity.Direction * speed.Value * DeltaTime;
    }
}
```

Note that `IJobEntity` uses plain `ref` / `in` parameters, not `RefRW<T>` / `RefRO<T>`. (The generator wraps the components into chunk-array accesses under the hood.)

---

4. Aspects that wrapped a DynamicBuffer

```
// BEFORE
public readonly partial struct PathAspect : IAspect
{
    public readonly DynamicBuffer<Waypoint> Path;
    public void Advance(int n) => Path.RemoveRange(0, n);
}

// AFTER — access buffer directly
foreach (var (path, e) in SystemAPI.Query<DynamicBuffer<Waypoint>>().WithEntityAccess())
{
    path.RemoveRange(0, 1);
}
```

In jobs, `DynamicBuffer<T>` is a valid `Execute` parameter — nothing extra needed.

---

5. Cross-entity aspects

Occasionally an aspect wrapped a `ComponentLookup<T>` for "I need to peek at another entity's component." Replacement:

```
// BEFORE
public readonly partial struct AITargetAspect : IAspect
{
    public readonly RefRO<Target> Target;
    [ReadOnly] public readonly ComponentLookup<Health> HealthLookup;
    public float TargetHealth =>
        HealthLookup.TryGetComponent(Target.ValueRO.Entity, out var h) ? h.Value : 0f;
}

// AFTER — declare the lookup on the system / job directly
public partial struct AISystem : ISystem
{
    private ComponentLookup<Health> _healthLookup;

    public void OnCreate(ref SystemState state)
    {
        _healthLookup = state.GetComponentLookup<Health>(true);
    }

    public void OnUpdate(ref SystemState state)
    {
        _healthLookup.Update(ref state);

        foreach (var (target, e) in SystemAPI
                     .Query<RefRO<Target>>()
                     .WithEntityAccess())
        {
            float hp = _healthLookup.TryGetComponent(target.ValueRO.Entity, out var h)
                ? h.Value : 0f;
            // ...
        }
    }
}
```

This pattern is also lighter than the aspect version because the lookup is shared across entities.

---

6. When an aspect was complex

If the aspect had many fields, encapsulated state, or participated in non-trivial inheritance — that's a sign to consider a genuine refactor, not just a port. Often the code reads better as two systems that pass data through components, rather than one system with a god-aspect.

---

7. Troubleshooting

Symptom	Cause / Fix
Compile warning `IAspect is obsolete`	Expected on 6.5. Port as above to clear the warning; the code still runs until removal in a future major version.
`type or namespace IAspect not found`	You are on a version where `IAspect` has been removed. Complete the port — no fallback remains.
Query tuple has too many types and doesn't compile	`SystemAPI.Query` supports a large tuple; splitting a very wide query into two queries or moving to `IJobChunk` usually reads better.
Job no longer Burst-compiles	An aspect method may have captured a type Burst didn't like. Re-check the `Execute` parameters directly.
Cross-entity aspect ported naively runs slowly	`ComponentLookup<T>.Update(ref state)` must be called in `OnUpdate` before use; forgetting it makes lookups invalid.
DynamicBuffer aspect won't port	`DynamicBuffer<T>` is valid as a `SystemAPI.Query` type and `IJobEntity.Execute` parameter directly — no wrapper needed.
Tests that asserted on the aspect type fail	Delete or rewrite those tests against the new signature; there's nothing to keep.

Appendix. Changelogs

Appendix A. Entities 1.4 → 6.5 Key Changes

> A scoped summary of the official `com.unity.entities` package changelog entries that matter when moving an ECS project from the 1.4 line to the 6.5 Core Package line. This page intentionally covers Entities package changes only.

1. Version tracks

Entities 6.x aligns with Unity Editor 6000.x and ships as a Core Package. Older Entities 1.x versions were Package Manager packages.

Track	Distribution	Example
Entities 1.x	Package Manager (`com.unity.entities`)	1.4.2 in the official 6.5 changelog history.
Entities 6.x	Core Package embedded in the Editor	6.4.0 / 6.5.0.

This distinction is about package distribution and versioning. It is not a license to pull unrelated UnityEngine or Editor-wide migrations into DOTS programming docs.

---

2. Entities 6.5.0

The official package changelog entry for Entities 6.5.0 is:

> "Placeholder for 6.5.0"

Treat 6.5.0 as the Unity 6000.5-aligned package version. The package changelog does not list new Entities API surface for this release.

---

3. Entities 6.4.0

The official package changelog entry for Entities 6.4.0 says:

> "Package is now a core package embedded in Unity."

Practical effects:

Area	Change
Installation	On Unity 6000.4+ you still add `com.unity.entities` from the Package Manager, but it installs from the Editor bundle — there is no version to choose.
Version choice	The Entities version comes from the Editor install.
Lock file	`packages-lock.json` marks the package as built-in/core rather than a registry dependency.
Upgrade workflow	Replace any pinned 1.x version of `com.unity.entities` (re-add it from the Package Manager) and let the Editor resolve it to the Core version.

See `../Migration/02_Package Manager → Core Package.md`.

---

4. Entities 1.4.2

The official changelog history included with `com.unity.entities@6.5` lists 1.4.2 before the Core Package transition.

Changed

Change	Why it matters
Burst dependency updated to 1.8.24	A dependency bump for projects still on the 1.4 line.
USS classes changed from percentage to pixel sizing	Editor UI styling detail.
Font-size field changed from percentage to pixel sizing	Editor UI styling detail.

Fixed

Fix	Why it matters
Incorrect source generation for `public partial SystemBase`	Relevant if older 1.4 code hit source-generation errors.
`SGICE002` with `SystemAPI` inside `Entities.ForEach` plus `WithDeferredPlaybackSystem`	Useful context while migrating away from `Entities.ForEach`.
World serialization memory leak	Runtime/editor stability.

Fix	Why it matters
Inspector initialization after exiting Play Mode with an entity selected	Editor tooling stability.

---

5. 1.4.0 pre-release entries worth migrating for

The 1.4.0 preview/pre-release history is where the important ECS API direction appears.

Area	Official change	Migration target
Iteration	`Entities.ForEach` marked obsolete; future major removal planned.	Use `IJobEntity` or `SystemAPI.Query`.
Aspects	`IAspect` marked obsolete; future major removal planned.	Use direct component/query APIs.
Optional component refs	`ComponentLookup.GetRefRWOptional()` / `GetRefROOptional()` deprecated.	Use `TryGetRefRW<T>()` / `TryGetRefRO<T>()`.
Chunk buffers	`ArchetypeChunk.GetBufferAccessorRO<T>()` / `GetBufferAccessorRW<T>()` added.	Request the access mode you actually need.
Untyped buffers	`GetUntypedBufferAccessorReinterpret<T>()` added.	Use only when source/destination element sizes and buffer capacities are safely aliasable.
System handles	`SystemTypeIndex` can be retrieved from `SystemHandle`.	Better low-level system access.
Query tooling	Disabled, Present, Absent, None columns; Dependency tab; prefab distinction in Query window.	Better inspection/debugging of queries.
Object refs in ECS	Manual documentation for `UnityObjectRef` added.	Use `UnityObjectRef<T>` for managed Unity object references in unmanaged ECS data.

---

6. Migration checklist

- [ ] On Unity 6000.4+, re-add the DOTS Core Packages from the Package Manager, replacing any pinned 1.x versions in `manifest.json`.
- [ ] Replace remaining `Entities.ForEach` with `IJobEntity` or `SystemAPI.Query`.
- [ ] Replace `IAspect` usage with direct component access.
- [ ] Replace `GetRefRWOptional` / `GetRefROOptional` with `TryGetRefRW<T>()` / `TryGetRefRO<T>()`.
- [ ] Review managed Unity object references and use `UnityObjectRef<T>` or baked data where appropriate.
- [ ] Recheck Systems, Query, and Entity Inspector windows after migration.

---

7. References

- Entities 6.5 Manual
- Entities 6.5 Changelog

Appendix B. Netcode for Entities 1.4 → 6.5 Key Changes

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

> A migration reader for Netcode for Entities changes accumulated from the 1.4 line to the 6.5 Core Package line. Netcode 6.5.0 moved into the Unity 6000.5 Editor as a built-in package, so later entries move toward Editor release notes rather than this package changelog.

1. Version highlights

6.5.0 (2026-02-05) — built-in Editor package

Area	Change
Distribution	Netcode moved to a built-in Editor package starting with Unity 6000.5.
Entities dependency	Changelog line lists `com.unity.entities` **1.4.2**. Keep this distinct from the DOTS manual's Unity 6000.5 / Entities Core Package target.
Profiler	Ghost Snapshot View TreeView labels gained context menus for quick prefab/component inspection.
Source Generator	IDE-side Source Generator execution was disabled for Rider and Visual Studio, improving IDE performance.
Breaking	Disconnect ECB changed from `BeginSimulationECBS` to `NetworkGroupCommandBufferSystem`.

1.12.0 (2026-02-01)

Area	Change
Profiler	Tick-based navigation, search fields, and client Prediction / Interpolation tick display.
Entities dependency	Bumped from 1.4.2 to **1.4.4**.
Deprecated	Browser-based Network Debugger deprecated in favor of Netcode Profiler.
Removed	`NETCODE_PROFILER_ENABLED` is no longer required.
Breaking	`GhostAuthoringComponentBaker` `TransformUsageFlags` behavior changed.

1.11.0 (2025-12-12)

Area	Change
Prefab List menu	Quick Ghost Prefab inspection.
GameObject Ghost	Work started for GameObject-layer Ghost support.
Static Ghost optimization	Static Ghost job timing reduced substantially for unchanged data.
Ghost Migration	`IncludeInMigration` supports non-Ghost data migration.
Bool serialization	Packed / unpacked bool serialization costs improved.
API breaking	`PrefabDebugName.Name` fully deprecated.

1.10.0 (2025-11-09)

Area	Change
Zero-sized components	World state save supports zero-sized components.
Source Generator	IDE-side execution disabled; reflected in 6.5.0 behavior.
Fixes	`PredictedGhostSpawnSystem` assert, Sleep mode warning spam, `AlwaysRun` prediction-loop exception, and related fixes.

1.9.x (2025-09 to 2025-10)

Area	Change
Ghost optimization docs	Dedicated Ghost optimization documentation added.
Stabilization	1.8.0 Profiler Modules and Importance Visualizer stabilized.

1.8.0 (2025-08-17)

Area	Change
Importance Visualizer	Added to PlayMode Tool.
Forced Input Latency	`ClientTickRate.ForcedInputLatencyTicks` added.
Profiler Modules	Experimental Server / Client Profiler modules for Unity 6+.
Breaking	`GhostDistanceImportance` scale functions no longer multiply `baseImportance` by 1000.
Transport	Updated to `com.unity.transport` **2.5.3**.

1.7.0 (2025-07-29)

Area	Change
Host Migration	`ENABLE_HOST_MIGRATION` define removed; Host Migration enabled by default.
Host Migration API	Class and method names refactored; use the current package documentation/samples for exact type names.
Serialization	Ghost component serialization improved for Host Migration.
Transport	Updated to `com.unity.transport` **2.1.0**.

1.6.1 to 1.6.2 (2025-05 to 2025-07)

Area	Change
Predicted ECB	Added begin/end ECB systems for `PredictedSimulationSystemGroup`.
Predicted spawning	Added `PredictedSpawningSystemGroup` after `EndPredictedSimulationCommandBufferSystem`.
Host Migration	Experimental feature hidden behind `ENABLE_HOST_MIGRATION`.
Reconnect tracking	`NetworkStreamIsReconnected` component added.
Snapshot history	`GhostSystemConstants.SnapshotHistorySize` configurable by compiler define.
Relevancy + Importance	Fast-path support for combining Ghost Relevancy and Importance Scaling.

1.5.0 (2025-04-22)

Area	Change
Thin Clients	`AutomaticThinClientWorldsUtility` added for runtime Thin Client creation.
Serialization	FixedList and unsafe fixed-buffer replication support.
Baselines	`GhostAuthoringComponent.UseSingleBaseline` added for CPU savings.
Integer serialization	byte / short-specific templates.
Lag Compensation	Physics History Buffer increased from 16 to **32** ticks.
Relay	Relay package path shifted toward `com.unity.services.multiplayer` v1.1.0.

1.4.0 (2024-11-14)

Area	Change
Send rate	`GhostAuthoringComponent.MaxSendRate` limits Ghost send frequency.
Chunk iteration	`GhostSendSystemData.MaxIterateChunks` limits chunks processed per tick.
`NetCodeConfig`	Added several Unity Transport `NetworkConfigParameters`.

Area	Change
Snapshot ACK	ACK history capacity became configurable.
Fixes	Snapshot ACK issues past 256 ticks, physics interpolation jitter, and related fixes.

---

2. Cumulative breaking changes

Version	Breaking change
6.5.0	Disconnect ECB changed from `BeginSimulationECBS` to `NetworkGroupCommandBufferSystem`.
1.12.0	`GhostAuthoringComponentBaker` `TransformUsageFlags` behavior changed.
1.11.0	`PrefabDebugName.Name` fully deprecated.
1.8.0	`GhostDistanceImportance` multiplier behavior changed.
1.7.0	Host Migration API class and method names refactored.
1.5.0	Command validation tightened and handshake timeout behavior changed.
1.4.0	`MaxSendChunks` scope changed.

---

3. Trends to remember

1. **Profiler-first debugging** — the old browser Network Debugger is deprecated; use Netcode Profiler modules.
2. **Host Migration became default** — experimental in 1.6.1, enabled by default in 1.7.0+.
3. **IDE performance improved** — Netcode Source Generators no longer run inside IDE analysis.
4. **Serialization became more efficient** — bool packing, byte/short templates, and single-baseline options reduce CPU/bandwidth pressure.
5. **Core Package transition** — Netcode 6.5.0 ships with Unity 6000.5; future changes move toward Editor release notes.

---

4. Related docs

- `../DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
- `../DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md`
- `../DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md`
- `../DOTS Workflows/24_Netcode Profiler & Debugging.md`

---

5. References

- Netcode for Entities 6.5 Manual
- Netcode for Entities 6.5 Changelog
- Netcode for Entities 1.12 Changelog

Reference. Glossary

Glossary

Unity 6000.5 · Entities 6.5.0 · Netcode for Entities 6.5.0

One-line definitions for every non-obvious term used in this manual. Cross-links point to the doc where each term is defined in depth.

1. Core nouns — the five things ECS is built from

Term	Definition	See
Entity	An opaque 64-bit ID (`struct Entity`, `Unity.Entities` namespace). No data, no behaviour — just a key used to look up components.	`DOTS Workflows/03_ECS Core Concepts.md`
Component	Data attached to an entity. Several kinds — see §2.	`DOTS Workflows/05_Component Types.md`
Archetype	The set of component types an entity has. Entities with the same archetype share storage.	`DOTS Workflows/03_ECS Core Concepts.md`
Chunk	A 16 KB block of entities sharing an archetype. Components stored column-major (SoA) within the chunk.	`Optimizations and Debugging/01_Chunk Layout & TypeManager.md`
World	Container of entities and systems. The runtime starts with a "Default World".	`DOTS Workflows/03_ECS Core Concepts.md`

---

2. Component kinds

Kind	What it is
`IComponentData`	Unmanaged struct component — the default.
`IBufferElementData`	Element type for a `DynamicBuffer<T>` attached to an entity.
`ISharedComponentData`	Value-keyed component — entities with the same value share a chunk bucket.
`ICleanupComponentData`	Component that survives `DestroyEntity` so a system can clean up before the entity fully disappears.
Tag component	Empty `IComponentData` struct — zero bytes, used purely as a query filter.

Kind	What it is
Chunk component	Per-chunk data (one instance per chunk, not per entity).
`IEnableableComponent`	Component whose presence toggles via a per-chunk bitmask — **no structural change**.

Details: `DOTS Workflows/05_Component Types.md` · `DOTS Workflows/06_Enableable Component.md`.

---

3. Systems and jobs

Term	What it is
`ISystem`	Modern struct-based system interface. Burst-compatible. **Preferred for new systems.**
`SystemBase`	Legacy class-based system interface. Use only when a system needs main-thread managed access.
SystemGroup	Ordered container of systems. Main groups: `InitializationSystemGroup`, `SimulationSystemGroup`, `PresentationSystemGroup`.
`IJobEntity`	Job that iterates entities matching a query. Source-generated, Burst-compiled.
`IJobChunk`	Job that iterates chunks (not individual entities) — use for per-chunk aggregation.
`SystemAPI`	Static helper surface inside systems: `Query<>()`, `GetComponentLookup<T>()`, `Time`, `GetSingleton<T>()`, etc.

Details: `DOTS Workflows/08_System — ISystem vs SystemBase.md` · `DOTS Workflows/11_IJobEntity · SystemAPI.Query.md` · `DOTS Workflows/12_IJobEntity vs IJobChunk.md`.

---

4. Authoring → runtime

Term	What it is
Baker	Converts an authoring GameObject + MonoBehaviour into ECS components at SubScene save/build time.
SubScene	Authoring container — GameObjects inside it are baked into entities and serialised to an EntityScene asset.
EntityScene	The baked asset produced from a SubScene — loaded into a World at runtime.

Details: `DOTS Workflows/01_Baker Pattern & SubScene.md`.

---

5. Structural change safety

Term	What it is
Structural change	Adds/removes a component or creates/destroys an entity. Moves entities between chunks → expensive; not safe mid-iteration.
`EntityCommandBuffer` (ECB)	Records structural changes during iteration, plays them back at a safe boundary.
`EntityCommandBuffer.ParallelWriter`	Thread-safe ECB variant for parallel jobs; each call takes a `sortKey` for deterministic playback.
Deferred entity	An entity created via `ecb.CreateEntity()` / `Instantiate()` that doesn't exist yet — placeholder index resolves at ECB playback.
Sort key	Integer passed to each parallel ECB command; `[ChunkIndexInQuery]` is the default.

Details: `DOTS Workflows/13_Structural Change & Safety.md` · `DOTS Workflows/14_EntityCommandBuffer · Deferred Entity.md` · `DOTS Workflows/15_ParallelWriter · Deterministic Playback.md`.

---

6. Entity reference types

These are the DOTS reference types used in ECS data and baking workflows.

Type	What it is
`Entity`	ECS handle (`Unity.Entities`). Just an ID in the ECS world.
`EntityPrefabReference`	Reference to a baked entity prefab asset for content/streaming workflows (`Unity.Entities.Serialization`).
`UnityObjectRef<T>`	ECS-side reference to a `UnityEngine.Object` — lets entities point at managed assets (materials, meshes) without boxing.

Details: `DOTS Workflows/04_Entity References — Entity · EntityPrefabReference · UnityObjectRef.md`.

---

7. Netcode for Entities

Term	What it is	See
Netcode for Entities	Unity's DOTS multiplayer netcode layer for server-authoritative games with client prediction.	`DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
Server World	Authoritative ECS world that runs the server simulation and sends Ghost snapshots.	`DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
Client World	ECS world that receives snapshots, sends commands, predicts local gameplay, and presents interpolated state.	`DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
Thin Client World	Lightweight dummy client world used for testing load and connection behavior.	`DOTS Workflows/16_Netcode Client-Server World & Bootstrap.md`
`NetworkStreamConnection`	Component on a connection entity that stores the transport connection handle.	`DOTS Workflows/17_Netcode Network Connection & Approval.md`
`NetworkId`	Server-assigned connection ID after approval.	`DOTS Workflows/17_Netcode Network Connection & Approval.md`
`NetworkStreamInGame`	Component that enables gameplay data exchange; without it, snapshots and commands do not flow.	`DOTS Workflows/17_Netcode Network Connection & Approval.md`
Ghost	Server-authoritative networked entity replicated to clients through snapshots.	`DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md`
Snapshot	Serialized Ghost state sent from server to client, usually over unreliable transport.	`DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md`
`GhostField`	Attribute that marks a component field for Ghost serialization.	`DOTS Workflows/18_Netcode Ghost Snapshot & Synchronization.md`
`PredictedGhost`	Component identifying Ghosts that participate in client prediction.	`DOTS Workflows/19_Netcode Prediction & Rollback.md`
`Simulate`	Enableable tag used by Netcode to mark which predicted entities should run on the current prediction tick.	`DOTS Workflows/19_Netcode Prediction & Rollback.md`
`IInputComponentData`	High-level input component type with generated command-buffer management.	`DOTS Workflows/20_Netcode Command Stream & Input.md`
`ICommandData`	Lower-level command stream type where tick mapping and buffer access are managed manually.	`DOTS Workflows/20_Netcode Command Stream & Input.md`

Term	What it is	See
`InputEvent`	One-shot input marker for events such as jump/fire that must register exactly once on the target tick.	`DOTS Workflows/20_Netcode Command Stream & Input.md`
`IRpcCommand`	Reliable one-shot RPC message type.	`DOTS Workflows/21_Netcode RPC.md`
`IApprovalRpcCommand`	Special RPC type allowed during connection approval.	`DOTS Workflows/21_Netcode RPC.md`
Ghost Importance	Server-side priority score that decides which Ghosts fit into snapshot budget first.	`DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md`
Ghost Relevancy	Per-connection filtering that decides whether a Ghost should replicate to a specific client.	`DOTS Workflows/22_Netcode Ghost Optimization · Importance · Relevancy.md`
Lag Compensation	Server-side lookup of historical collision worlds so it can validate client actions at the client's perceived time.	`DOTS Workflows/23_Netcode Physics Integration & Lag Compensation.md`

---

8. Legacy / obsolete (do NOT appear in new code)

Term	Status on 6.5	Migration target
`Entities.ForEach`	Obsolete since 1.4; still compiles with warnings; removal planned for a future major release.	`SystemAPI.Query<>()` or `IJobEntity` — `Migration/04_foreach → IJobEntity.md`
`IAspect`	Obsolete since 1.4; still compiles with warnings; removal planned for a future major release.	Direct component parameters (`RefRW<T>` / `RefRO<T>`) — `Migration/05_IAspect Removal.md`
`ComponentLookup.GetRefRWOptional`	Obsolete.	`TryGetRefRW` — see `Migration/01_Entities 1.x → 6.5 Overview.md`

---

9. Supporting concepts

Term	What it is
Burst	Unity's native compiler for jobs and `[BurstCompile]`-marked structs. Produces native SIMD code.

Term	What it is
Change version	32-bit counter bumped when a system writes to a component column in a chunk. Enables `SetChangedVersionFilter` to skip unchanged chunks.
SoA (Structure of Arrays)	Storage layout where each field becomes its own array — cache-friendly for scalar iteration. Chunks use SoA.
`RefRW<T>` / `RefRO<T>`	Wrapper types for accessing components in `SystemAPI.Query` with read-write / read-only intent.
`EnabledRefRW<T>` / `EnabledRefRO<T>`	Wrapper for reading/writing an enableable component's toggle bit without triggering a structural change.
TypeManager	The runtime registry that assigns each component a stable type index. Rebuilt at startup; large projects feel this.
`BlobAssetReference<T>`	Pointer to a large, read-only data blob stored outside the chunk. Avoids bloating chunk capacity.